学校代码：10730　　密级：公开
分类号：TP302

兰州大学

硕 士 学 位 论 文

（ 专 业 学 位 ）

论文题目（中文）：基于内-外积算法的异步多模式矩阵加速器架构研究与实现

论文题目（外文）：Research and Implementation of Asynchronous Multi-mode Matrix Accelerator Architecture Based on Inner-outer Product Algorithm

作 者 姓 名：张灵壮
学 科 专 业：电子信息·计算机技术
研 究 方 向：计算机体系结构
教 育 类 型：学历教育
指 导 教 师：何安平 副教授
合 作 导 师：马浚 工程师

论文工作时间段：2024 年 7 月 至 2025 年 3 月

答 辩 日 期：2025 年 5 月

校址: 甘肃省兰州市城关区天水南路 222 号

原 创 性 声 明

本人郑重声明：本人所呈交的学位论文，是在导师的指导下独立进行研究所取得的成果。学位论文中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

论文作者签名：_____ 日 期：_____

关于学位论文使用授权的声明

本人在导师指导下所完成的论文及相关的职务作品，知识产权归属兰州大学。本人完全了解兰州大学有关保存、使用学位论文的规定，同意学校保存或向国家有关部门或机构送交论文的纸质版和电子版，允许论文被查阅和借阅；本人授权兰州大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本学位论文。本人离校后发表、使用学位论文或与该论文直接相关的学术论文或成果时，第一署名单位仍然为兰州大学。

本学位论文研究内容：

☑可以公开

□ 不宜公开，已在学位办公室办理保密申请，解密后适用本授权书。

（请在以上选项内选择其中一项打“√”）

论文作者签名：_____ 导师签名：_____

日 期：_____ 日 期：_____

基于内-外积算法的异步多模式矩阵加速器架构研究与实现

中文摘要

低阶张量运算广泛应用于数值计算、机器学习、图神经网络和线性代数求解等领域，其中稀疏矩阵乘法（SpGEMM）和稀疏矩阵向量乘法（SpMV）是两大重要计算操作。然而，稀疏矩阵的高稀疏度、规模大和元素分布不规则带来了高效访存、计算负载均衡和格式化结果存储的挑战。现有 CPU 在处理高稀疏度矩阵时存在吞吐率瓶颈，导致计算性能受限。近年来，专用矩阵加速器具备良好的内存层次结构和小量的指令译码开销，成为提升稀疏矩阵乘法的一大方法。目前，常见的硬件加速器均是基于同步电路设计，随着电路规模的急剧增加，同步电路面临时钟偏移等问题，难以应对并发度极高的矩阵计算。由于异步电路取消全局时钟，具有模块化好、功耗低等优势，且适合并发性高的计算，所以本文将异步电路设计方法、矩阵乘算法与稀疏矩阵存储策略相结合，探索解决稀疏高的低阶张量计算能效低下的解决方案。本研究工作的主要工作如下：

（1）本研究工作采用粗粒度异步微流水线设计方法进行矩阵加速器架构设计，取消了全局时钟。粗粒度异步微流水线中的链式结构（Fifo）、分支（Split）、融合（Merge）三种核心结构，使得复杂算法映射成硬件电路的难度降低。多模式矩阵加速器基于粗粒度异步微流水线设计，模块之间通过脉冲通信，数据与事件绑定保证模块的正常工作，降低功耗的同时保证设计的性能。

（2）本研究提出基于 BCSR 格式的存储方案，并设计内-外积矩阵乘算法。在对存储格式、分块策略的分析结论基础上，使用 BCSR 格式存储采用 4×4 分块策略后的规模 128×128 的稀疏矩阵，同时结合片内存储每行 128 比特的实际情况，提出了一种便于代码库实现的矩阵和向量的存储方案。结合内积算法和外积算法的优势，设计了一种基于“内积-外积”融合算法的矩阵计算方式。

（3）本研究结合异步电路设计方法、存储格式和矩阵乘算法，提出支持 11 种计算操作的多模式矩阵加速器架构。首先，定义了 11 条微码，支持矩阵乘法、矩阵向量乘法和数乘矩阵在内的 11 种计算模式。接着，提出包含矩阵控制模块（CU_M）、三级流水矩阵计算单元（MXU）、数乘单元（SMU）、压缩单元（COMPR）和解压单元（DECOMPR）的矩阵计算架构。最后，基于 110*nm* 制程完成多模式矩阵加速器的 ASIC 前端设计流程。仿真结果表明矩阵的稀疏度大于 99.0% 时，相较于 Intel MKL，本架构在矩阵乘法、列向量 × 行向量和行向量 × 列向量 3 种模式下实现了平均 2.5×、2.5× 和 3× 的平均加速。同时，在稀疏度 0%

和 99.9% 下，基于 SAIF 文件的功耗分析结果表明异步多模式稀疏矩阵加速器面向高度稀疏的低阶张量计算具有较好的功耗优势。

关键词：稀疏矩阵乘法，矩阵加速器架构，内-外积融合算法，异步电路设计，低功耗

RESEARCH AND IMPLEMENTATION OF ASYNCHRONOUS MULTI-MODE MATRIX ACCELERATOR ARCHITECTURE BASED ON INNER-OUTER PRODUCT ALGORITHM

Abstract

Low-order tensor operations is widely applied in numerical computation, machine learning, graph neural networks, and linear algebra solvers, where generalized sparse matrix-matrix multiplication (SpGEMM) and sparse matrix-vector multiplication (SpMV) serve as two critical computational primitives. However, the high sparsity, large-scale nature, and irregular element distribution of sparse matrices pose significant challenges in efficient memory access, computational load balancing, and formatted result storage. Conventional CPUs exhibit throughput bottlenecks when processing highly sparse matrices, leading to constrained computational performance. In recent years, dedicated matrix accelerators have emerged as a promising approach to enhance sparse matrix operations, leveraging their optimized memory hierarchy and minimal instruction decoding overhead. Most existing hardware accelerators adopt synchronous circuit designs, which increasingly suffer from clock skew issues as circuit scales expand rapidly. In contrast, asynchronous circuits eliminate global clocks and offer advantages such as modularity, low power consumption, and high concurrency support. This study innovatively integrates asynchronous circuit design methodologies, matrix multiplication algorithms, and sparse matrix storage strategies to address the low energy efficiency in high-sparsity low-order tensor computations. The main contributions of this work include:

(1) This study employs a coarse-grained asynchronous micropipeline design methodology for matrix accelerator architecture, eliminating the global clock. The three core structures in the coarse-grained asynchronous micropipeline are chained structure (First In First Out, FIFO), branching, and fusion significantly reduce the complexity of mapping sophisticated algorithms to hardware circuits. The multi-mode matrix accelerator, based on this coarse-grained asynchronous micropipeline design, utilizes pulse communication between modules. By binding data with events, it ensures proper module operation while reducing power consumption without compromising design performance.

(2) This study proposes a storage scheme based on the Block Compressed Sparse Row (BCSR) format and designs an inner-outer product matrix multiplication algo-

rithm. Building upon the analysis of storage formats and blocking strategies, we employ the BCSR format to store 128×128 sparse matrices with a 4×4 blocking strategy. Considering the practical constraint of 128 bit on-chip memory per row, we present an efficient storage scheme for matrices and vectors that facilitates library implementation. By combining the advantages of both inner-product and outer-product algorithms, we develop a novel matrix computation method based on a fused "inner-outer product" algorithm.

(3) This study proposes a multi-mode matrix accelerator architechture supporting 11 computational operations, based on asynchronous circuit design methodologies,storage formats and matrix multiplication. First, we define 11 microinstructions to support various computation modes including matrix multiplication, matrix-vector multiplication, and scalar-matrix multiplication. Subsequently, we present a novel matrix computation architecture which includes a matrix control unit (CU_M), a three-stage pipelined matrix computation unit (MXU), a scalar multiplication unit (SMU), a compression unit (COMPR) and a decompression unit (DECOMPR). The ASIC frontend design flow of this multi-mode matrix accelerator was implemented using 110*nm* technology. Simulation results demonstrate that when matrix sparsity exceeds 99.00%, our architecture achieves average speedups of 2.5×, 2.5×, and 3× compared to Intel MKL for three computation modes: matrix multiplication, column vector × row vector, and row vector × column vector, respectively. Furthermore, power analysis based on SAIF files reveals that the asynchronous multi-mode sparse matrix accelerator exhibits significant power efficiency advantages for highly sparse low-order tensor computations, with consistent performance across sparsity levels at 0% and 99.9%.

Keywords: Generalized Sparse Matrix-Matrix Multiplication, Matrix Accelerator Architecture, Inner-outer Product Fusion Algorithm, Asynchronous Circuit Design, Low Power Consumption

目 录

中文摘要 I

Abstract III

第一章 绪论 1

1.1 研究背景和意义 1

1.2 国内外研究现状 2

1.2.1 矩阵乘加速器研究现状 2

1.2.2 稀疏矩阵向量乘加速器研究现状 4

1.3 本文主要工作与创新点 5

1.4 文章组织结构 6

第二章 相关技术原理 7

2.1 矩阵乘算法 7

2.1.1 内积算法 7

2.1.2 外积算法 7

2.1.3 Strassen 算法 8

2.1.4 Gustavson 算法 9

2.1.5 Cannon 算法 9

2.2 稀疏矩阵存储格式 10

2.2.1 COO 存储格式 11

2.2.2 CSR、CSC 存储格式 11

2.2.3 BCSR、BCSC 存储格式 12

2.2.4 ELL 存储格式 12

2.2.5 DIA 存储格式 13

2.2.6 HYB 存储格式 14

2.2.7 Bitmap 存储格式 14
2.2.8 Fibertree 存储格式 15
2.3 异步电路设计方法 16
2.3.1 异步控制器 16
2.3.2 NCL 异步电路设计方法 18
2.3.3 粗粒度异步微流水线 19
2.4 本章小结 21
第三章 矩阵加速器的存储格式、分块策略与算法分析 22
3.1 稀疏矩阵存储格式分析 22
3.2 矩阵分块策略分析 23
3.3 基于 BCSR 格式的矩阵、向量存储方案 24
3.4 矩阵乘法算法分析 26
3.4.1 分块矩阵乘法分析 26
3.4.2 子矩阵乘法分析 27
3.4.3 基于 BCSR 格式的内积-外积的矩阵乘法 28
3.5 本章小结 32
第四章 异步多模式矩阵加速器架构设计 33
4.1 系统架构 33
4.2 控制单元（CU_M）与矩阵计算微码 34
4.2.1 矩阵计算微码 34
4.2.2 控制单元 35
4.3 存储（Memeroy）与访存接口（Access Interface） 35
4.3.1 存储（Memeroy） 35
4.3.2 访存接口（Access Interface） 36
4.4 三级流水矩阵计算模块（MXU） 37
4.4.1 第一级流水：BCSR 解析器（BCSR Parser） 38

4.4.2 第二级流水：计算核心（Computing Core） 42
4.4.3 第三级流水：结果回写模块（Result Writer） 46
4.5 数乘模块（SMU） 49
4.5.1 BCSR 搬移模块（BCSR Mover） 49
4.5.2 计算与结果回写模块（Compute & Result Writer） 50
4.6 矩阵、向量压缩单元（COMPR）与解压单元（DECOMPR） 50
4.6.1 矩阵、向量压缩单元（COMPR） 51
4.6.2 矩阵、向量解压单元 52
4.7 本章小结 54
第五章 异步多模式矩阵加速器的实现与验证 56
5.1 异步多模式矩阵加速器测试方案设计 56
5.1.1 异步片上系统整体架构 56
5.1.2 测试数据集 57
5.1.3 基于 SAIF 文件的功耗分析 58
5.2 异步多模式矩阵加速器的 ASIC 实现与分析 59
5.2.1 异步电路系统 ASIC 前端设计流程 59
5.2.2 综合后仿真验证 62
5.2.3 性能分析对比 65
5.2.4 面积报告与功耗分析 70
5.3 异步片上系统实现、分析 73
5.3.1 异步片上系统功能测试 73
5.3.2 异步 SoC 面积、功耗报告对比 74
5.4 本章小结 76
第六章 结论 77
6.1 总结 77
6.2 展望 78

参考文献 80
在学期间的研究成果 86
致　　谢 87

图 目 录

图 2-1 内积算法示意图 7
图 2-2 外积算法示意图 8
图 2-3 Gustavson 算法示意图 9
图 2-4 Cannon 算法示意图 10
图 2-5 COO 存储格式示意图 11
图 2-6 CSR、CSC 存储格式示意图 12
图 2-7 BCSR、BCSC 存储格式示意图 13
图 2-8 ELL 存储格式示意图 13
图 2-9 DIA 存储格式示意图 14
图 2-10 HYB 存储格式示意图 14
图 2-11 Bitmap 存储格式示意图 15
图 2-12 Fibertree 存储格式示意图 15
图 2-13 C-element 示意图 16
图 2-14 Click 示意图 17
图 2-15 GasP 示意图 18
图 2-16 阈值门与加权阈值门示意图 19
图 2-17 NCL 系统框架与 DI 寄存器结构图 20
图 2-18 FIFO、Merge 和 Split 结构示意图 21

图 3-1 128×128 矩阵下 COO、CSR 和非压缩格式的存储空间与非零元素个数关系 23
图 3-2 基于 BCSR/BCSC 格式的矩阵、向量存储方案 26

图 4-1 多模式矩阵加速器整体架构 33
图 4-2 存储微架构 36
图 4-3 访存接口拓扑结构 37
图 4-4 三级流水矩阵计算单元架构 37
图 4-5 BCSR Parser 内部事件流图 39
图 4-6 PtrFetcherA 微架构 39
图 4-7 IdxFetcherB 微架构 40
图 4-8 BitArray Generator 微架构图 41
图 4-9 BlkMatcher 微架构图 41

图 4-10 DataFetcherA 微架构图 43
图 4-11 Data Allocator、PE Array 微架构 44
图 4-12 Result Merger 微架构 46
图 4-13 结果回写模块微架构图 47
图 4-14 数乘模块架构 49
图 4-15 索引生成模块架构图 50
图 4-16 计算与结果回写模块架构图 51
图 4-17 矩阵、向量压缩模块架构图 52
图 4-18 块遍历模块架构图 52
图 4-19 矩阵、向量解压单元架构图 53
图 4-20 解压单元有效块取数器微架构 54
图 4-21 解压单元数据回写模块微架构 55

图 5-1 异步片上系统架构图 56
图 5-2 测试数据集生成代码 58
图 5-3 ASIC 下基于 SAIF 文件的功耗分析流程 59
图 5-4 基于 SAIF 文件的功耗分析脚本 59
图 5-5 异步电路系统 ASIC 前端流程 60
图 5-6 粗粒度异步微流水线搭建 61
图 5-7 多模式矩阵加速器 RTL 设计 61
图 5-8 乘累加器最长路径约束语句 62
图 5-9 多模式矩阵加速器形式化验证结果 63
图 5-10 多模式矩阵加速器 Testbench 的主要逻辑 63
图 5-11 稀疏矩阵乘法综合后仿真波形 64
图 5-12 稀疏矩阵乘法计算结果匹配 64
图 5-13 压缩单元回写模块仿真波形 65
图 5-14 数乘单元模块仿真波形 65
图 5-15 对于不同稀疏度本架构与 Intel MKL 库在 4 种模式下的加速比折线图 69
图 5-16 多模式矩阵加速器面积报告 70
图 5-17 多模式矩阵加速器 DC 功耗报告 71
图 5-18 多模式矩阵加速器基于 SAIF 文件的稠密乘法平均功耗 72
图 5-19 矩阵库函数代码 74
图 5-20 行向量乘列向量模式测试程序 74
图 5-21 Testbench 中模拟指令、数据初始化过程代码段 75

图 5-22　异步 SoC 执行行向量乘列向量计算程序综合后仿真结果 75

表 目 录

表 2-1 双轨逻辑..... 18
表 2-2 四轨逻辑..... 19

表 3-1 不同分块策略下，平均有效块占比和平均零值填充率占比统计... 24
表 3-2 5 种矩阵乘算法下，4×4 规模矩阵乘法统计..... 28

表 4-1 矩阵加速器支持的数据类型编码..... 34
表 4-2 矩阵加速器计算微码..... 35

表 5-1 多模式矩阵加速器的仿真性能统计 _1..... 67
表 5-2 多模式矩阵加速器的仿真性能统计 _2..... 68
表 5-3 在 0% 与 99.9% 稀疏度下，多模式矩阵加速器不同计算模式的
平均功耗统计..... 72
表 5-4 矩阵加速器芯片基本参数..... 76

第一章 绪论

1.1 研究背景和意义

张量代数（Tensor Algebra）是向量、矩阵和高阶张量间的计算，普遍存在于数值计算[1]。通常，高阶张量间的运算通过将高阶张量分解为低阶张量（即，矩阵、向量）间的运算来完成。低阶张量运算在机器学习[2-3]、图神经网络[4-5]、线性代数求解器[6-7]、环境分析[8-9]等众多应用领域中扮演重要角色，因其丰富的应用场景受到国内外学者广泛的关注。

矩阵乘法、矩阵向量乘法是张量代数中两个常见的计算。现代科学与工程应用中的矩阵通常是不规则、稀疏的，如计算物理问题中由相对较少的微分项导出的矩阵、深度学习训练过程中的输入矩阵等。因此，稀疏矩阵乘法（General Sparse Matrix-Matrix Multiplication, SpGEMM）、稀疏矩阵向量乘法（Sparse Matrix-Vector Multiplication, SpMV）作为两个重要的计算基础，在现代科学与工程应用中占据大量计算时间。

稀疏矩阵的稀疏度高、规模大以及元素分布不规则等特征给计算架构提出三大挑战：（1）高效访问稀疏矩阵中的非零元素；（2）减少对中间结果操作；（3）格式化结果并写回内存中。高稀疏度是指矩阵中的非零元素远小于矩阵的维度积。早在 20 世纪 60 年代，研究者们就已发现在计算机使用二维数组处理稀疏矩阵十分低效。2003 年，OSKI 库[10]形式化稀疏矩阵表示方案，并确定许多优化方案，为稀疏代数的专用软件库奠定基础。然而，中央处理器（Central Processing Unit, CPU）内存吞吐率瓶颈使得软件实现的单核性能具有欺骗性，通常只达到峰值性能的 10%[11]。图形处理器（Graphics Processing Unit, GPU）在稀疏矩阵向量乘法和稠密矩阵乘法上表现出令人满意的计算效率，但是稀疏矩阵乘法情况则更加复杂。在计算的过程中，在任何位置都可能出现非零元素，需要复杂内存管理和负载均衡[12]。尽管 GPU 的峰值理论吞吐量超过 4 $TFLOPS$，但是当矩阵稀疏度降至 0.1% 以下时，计算单元明显未得到充分利用，实际性能通常达到低于 1 $GFLOPS$[13]。

在未来科技竞争中，关键领域对高能效的计算需求愈发迫切，特别是依赖电池供电的前沿应用，像边缘计算，可穿戴设备等领域。如何能源受限的应用领域中有效加速稀疏矩阵乘法、稀疏矩阵向量乘法，是学术界与工业界一直在研究的关键问题之一。随着摩尔定律的逐渐失效，设计专用矩阵加速器将成为提高稀疏矩阵乘法性能和改善能效的主要方法之一。专用矩阵加速器能够更好地利用中间内存层次结构，确保数据靠近计算核心，同时能够节约访存、乘累加等指令的

译码时间开销。

目前常见的专用硬件加速器大多是基于同步时钟电路设计，随着计算规模增加尤其是在完成不规则矩阵运算时，时钟的统一调度将变得越来越困难[14]。时序上，同步芯片内部需要满足组合逻辑时延严格小于时钟周期。功耗上，同步芯片无法满足极低功耗应用场景的需求。芯片规模的急剧增大使得同步时钟电路设计面临着时钟偏移[15]、时钟抖动[16]等问题带来的实质性挑战。异步电路不依赖时钟，通过握手信号控制电路工作。相较于同步电路，异步电路避免由时钟导致的相关问题，具有低功耗、模块化好、抗电磁干扰性强以及性能好等优势[17]，并且适合并发性高的计算，因此尝试用异步电路实现稀疏矩阵乘法运算是一个不错的选择。此外，有关矩阵计算架构的研究工作，大多聚焦于某一特定的运算模式，诸如稀疏矩阵乘法、稀疏矩阵向量乘法等。然而在滤波算法[18]中，同时涉及矩阵加（减）和矩阵相乘运算。实际的科学计算领域中，单一的计算模式难以契合系统需求，往往需要多种计算模式协同配合计算得到最终的结果[19]。

本文基于粗粒度异步微流水线提出一种面向规模为 128×128 矩阵，长度是 128 的向量的支持多种低阶张量计算的高能效多模式矩阵计算架构，并在专用集成电路技术（Application Specific Integrated Circuit，ASIC）前端流程下实现，并完成功能验证。最后在 ASIC 前端设计流程下验证多模式矩阵计算架构与异步 RISC-V CPU 和互联构建片上系统（System on Chip, SoC），证明该方案能够有效提升整个 SoC 的算力水平，满足低功耗领域的计算需求。

1.2 国内外研究现状

近年来，众多加速稀疏矩阵乘法、稀疏矩阵向量乘法的硬件加速器被国内外研究者们提出。针对不同的应用领域，不同的专用硬件加速器在矩阵算法、存储结构和系统架构等方面创新优化，采用硬件描述语言（Hardware Description Language, HDL）实现，最终在 FPGA 或 ASIC 上实现加速稀疏矩阵乘法、稀疏矩阵向量乘法。设计专用硬件加速器，解决 CPU 在相关应用领域中的算力不足问题，弥补 GPU 在面向高稀疏度的矩阵乘计算时效率低问题。

1.2.1 矩阵乘加速器研究现状

2016 年谷歌推出第一代 TPU[20]（Tensor Processing Unit），主要要用于模型的推理（inference）。从第二代 TPU 开始到 2024 年发布的第六代 TPU[21]，TPU 的核心用途已经转到 AI 的模型训练。脉动阵列是 TPU 的核心部件，其规模从 256×256 到改进到 128×128，有效提高脉动阵列的利用率[22-24]。

Subhankar Pal 等人于 2018 年提出基于外积算法的 OuterSPACE[13]。它由单

程序多数据（Single Program Multiple Data, SPMD）的处理单元，分布式内存，高速 crossbar 型总线以及高带宽存储（High Bandwidth Memory, HBM）构成。其中，单程序多数据处理单元通过二级缓存从主存中获取数据。

2020 年，Zhekai Zhang 等提出 SpArch[25]，该加速器通过对输入矩阵进行压缩减少中间结果数量，将矩阵乘外积算法中的乘法和合并阶段流水线化提升性能，行预取器（row prefetcher）改善因矩阵压缩导致矩阵 B 取数速度的降低，霍夫曼树结构的调度器降低内存拥塞。Eric Qin 等人提出面向 DNN 训练中不规则、非结构化的稀疏矩阵乘法的 SIGMA[26]，采用 bitmap 格式记录稀疏矩阵中的非零元素的位置信息，设计前向加法网络（Forwarding Adder Network, FAN）用于部分结果累加。Nitish Srivastava 等人提出的 MatRaptor[27]，首次探索基于 Gustavson 算法的稀疏矩阵加速器，并提出硬件友好的 C^2SR 存储格式。由于输入矩阵采用 C^2SR 格式进行压缩，MatRaptor 中的处理单元能够向量化访问数据，同时并行处理矩阵 A 中的 2-8 行，充分利用内存带宽（在矩阵乘法 $C = AB$ 中，矩阵 A 表示矩阵乘中的输入左矩阵，矩阵 B 表示矩阵乘中的输入右矩阵）。

2021 年，Guowei Zhang 等人提出基于 Gustavson 算法和 CSR 格式的稀疏矩阵加速器 GAMMA[28]，内部的调度单元（Scheduler）向 32 个处理单元分发矩阵 A 不同行中的非零元素，处理单元线性绑定矩阵 A 中的行用于计算矩阵 C 中对应的行，处理单元高速、并行地根据计算需求从缓存结构 FIBERCACHE 中取出矩阵 B。Julián Pavón 等提出采用向量架构的 VIA（Vector Indexed Architecture）[29]，同时为 VIA 设计丰富的指令集用于加速低阶张量计算。VIA 的硬件部分极少，主要是智慧便签存储器（Scratchpad Memory, SPM）和向量单元。VIA 支持的指令能够较为容易地集成到不同向量指令集架构（Vector Instruction Set Architecture）。

2023 年，Gerasimos Gerogiannis 等人提出 SPADE[30]，SPADE 处理单元通过和 CPU 之间的紧密耦合避免 CPU 和 SPADE 之间的数据搬运。兰州大学异步电路与系统团队刁文麒等人首先提出一种矩阵计算异步系统[14]，并在 $110nm$ 制程工艺下进行实现。该工作基于 COO 格式的双向压缩策略存储稀疏矩阵，支持八种矩阵、向量、标量相关的计算类型，采用“发送-中继-接收”的异步微流水线模控制、传输数据，软件层根据矩阵密度选择合适的计算策略。工业软件布局布线后，芯片的面积为 17.3 mm^2，平均功耗为 4.68 mW。实验表明在能效上该芯片具有较好的表现。

2024 年，Hesam Shabani 等人提出的 HIRAC[31]是应用于 DNN 应用的软硬件协同架构加速器。HIRAC 层次化，网络复杂度低的内部结构，通过 SorPack 算法将稀疏矩阵转成稠密矩阵来提高处理单元的利用率和系统整体性能。兰州大学异步电路与系统团队张鸿锐等人提出基于“内积-Strassen”混合算法的异步矩

阵加速器架构[17]。该架构基于粗粒度异步微流水构建，采用 BCSR 格式、BCSC 格式存储稀疏矩阵，支持 10 种矩阵类的运算，能够满足不同应用场景的计算需求。该工作在 FACE-VUP 平台上完成原型验证与性能评估，仿真结果表明，与 intel MKL 函数库和 Eigen 库相比，当矩阵的稀疏度小于 0.8% 时，稀疏矩阵乘法实现了 3.2 倍和 8.2 倍的平均加速比。

2025 年，中国科学院上海高等研究院祝永新团队提出一种创新硬件架构——Bit-Cigma。该架构通过对比特稀疏性的优化以及对浮点运算流程的重构，具备灵活兼容多种数据类型的能力[32]。

1.2.2 稀疏矩阵向量乘加速器研究现状

2019 年，Fazle Sadi 等人的工作[33]是基于内积算法的定制化加速器。该加速器通过元数据压缩技术 VLDI（Variable Length Delta Index）有效存储大规模的图结构，以及采用多路并行合并策略的预排序器在合并阶段提高内存带宽的利用率，加速稀疏矩阵向量乘法。

2020 年，Bahar Asgari 等人提出的 Alrescha[34]是一种创新的、自适应的矩阵预处理方法，用于简化对向量的访问。Alrescha 通过集成一个支持固定长度的内积单元和一个小的可重构的内积单元拥有在运行时快速、轻量可重构的特性。Alrescha 同时对数据进行预处理将非零元素的存储和运算的步骤匹配来消除数据的解压代价。

2021 年，Xinfeng Xie 等人提出 SpaceA[35]是一个存内计算（Processing-in-memory, PIM）的稀疏矩阵向量乘法加速器。SpaceA 将非零元素分发到不同的存储块到达负载均衡，内部集成内容寻址存储器（Content Addressable Memory, CAM）充分利用输入向量的数据局部性特征，平衡未完成的内存访问请求隐藏访问非本地块的内存时延。Siying Feng 等人采用软硬件协同的框架 CoSPARSE[36]选择最适合给定问题的软硬策略实现硬件加速。CoSPARSE 专注于软件层，该层根据矩阵的维度和稀疏度、向量的稀疏度选择内积或外积算法实现稀疏矩阵向量乘；硬件层主要决定采用何种内存子系统利用数据的共享模式。

Yuechen Lu 等人在 2023 年提出 DASP 算法[37]，通过稠密矩阵乘累加单元（Matrix Multiply-Accumulate，MMA）加速稀疏矩阵向量乘法。DASP 算法由两个主要部分组成：一种新的数据结构，可以有效地用于 MMA 单元的计算模式，以及相应的稀疏矩阵向量乘算法。

2024 年，Soojin Hwang 等人提出基于内积、支持三种模式计算的加速器 Cerberus[38]，根据矩阵、向量的稀疏度从 bitmap 和 CSR 格式中选取最佳的数据存储格式处理稀疏矩阵向量乘法，同时支持稠密稀疏矩阵向量乘法。

1.3 本文主要工作与创新点

本文主要面向对低阶张量计算存在低功耗的需求的相关应用场景，探索基于延迟限定型异步电路机制的矩阵加速器的可行性。从异步电路设计方法、矩阵乘算法、稀疏矩阵存储格式等方面，设计一种高能效的矩阵计算架构，提供一种完整、可行的解决方案。

本文的主要工作和创新点如下：

（1）取消全局时钟。异步电路设计相较于同步电路设计更容易将复杂算法映射到硬件电路上。粗粒度异步微流水线包含 Fifo、分支、融合三种核心结构，大幅简化异步电路设计复杂度。多模式矩阵加速器基于粗粒度异步微流水线设计，模块之间通过事件（脉冲信号）通信，数据与事件绑定保证模块的正常工作，降低功耗的同时保证设计的性能。

（2）提出基于 BCSR 格式的存储方案，并设计内-外积矩阵乘算法。针对规模 128×128 的稀疏矩阵，采用 4×4 分块策略后，在 BCSR 格式基础上，结合 Sram 每行大小 128 比特的实际情况，设计了有利于代码库实现的矩阵和向量的存储方案。结合内积算法和外积算法的优势，设计了一种基于“内积-外积”融合算法的矩阵计算方式。利用内积算法的输出复用性，降低结果矩阵或向量压缩格式的生成难度。利用外积算法的输入复用性，降低局部取数频率，提高局部的计算性能，从而提升整体的计算性能。

（3）提出支持 11 种计算操作的多模式矩阵加速器架构。首先，定义了 11 条微码，支持矩阵乘、矩阵加以及稀疏矩阵压缩、解压在内的 11 种计算模式。接着，提出由矩阵控制单元、三级流水矩阵计算单元、数乘单元、压缩单元和解压单元组成的矩阵计算架构。不同于 CPU 通过执行批量指令来控制、完成矩阵计算操作，矩阵加速器内的控制单元通过解析微码驱动对应的模块来完成相应的计算操作，去除指令译码的时间开销，提升低阶张量计算性能。设计支持 7 种计算的 BCSR 解析器，使得矩阵、向量相关的多种计算能够以较低的代价部署在同一矩阵计算单元上。压缩、解压单元用于原始矩阵和压缩后的矩阵之间进行相互转换，补足多模式矩阵加速器的功能。最后，基于 110*nm* 制程完成多模式矩阵加速器的 ASIC 前端设计流程。仿真结果表明矩阵的稀疏度大于 99.0% 时，相较于 Intel MKL，本架构在矩阵乘法、列向量 × 行向量和行向量 × 列向量 3 种模式下实现了平均 2.5×、2.5× 和 3× 的平均加速。同时，在稀疏度 0% 和 99.9% 下，基于 SAIF 文件的功耗分析结果表明异步多模式稀疏矩阵加速器面向高度稀疏的低阶张量计算具有较好的功耗优势。

1.4 文章组织结构

本文共六章，章节的主要安排如下：

第一章，绪论。首先介绍了本文的研究背景以及国内外相关领域的研究现状，阐明异步矩阵计算架构的研究意义，接着简要地说明了本文的主要工作和创新点，最后对全文的组织结构进行说明。

第二章，相关技术原理。该章节从矩阵乘算法、稀疏矩阵存储格式和异步电路设计方法三个方面进行介绍。首先介绍 5 种常见的矩阵乘法，并分析其输入输出复用性。其次介绍 8 种稀疏矩阵存储格式，简要分析了它们的存储效率。最后从异步电路控制器、NCL 异步电路设计方法和粗粒度异步微流水线 3 个方面介绍异步电路设计方法。

第三章，矩阵加速器的存储格式、分块策略与算法分析。首先分析第二章提及的 8 种稀疏矩阵存储格式的优缺点，选择 BCSR 格式存储稀疏矩阵。接着，统计不同分块策略下平均有效块占比与平均块内非零值占比并分析。紧接着提出基于 BCSR 格式的矩阵、向量存储方案。最后分析了分块矩阵乘法、子矩阵乘法，提出基于 BCSR 格式的内积-外积的矩阵乘法。

第四章，多模式矩阵加速器架构设计。首先介绍了矩阵计算整体架构，依次详细介绍矩阵计算架构内的控制单元模块、三级流水矩阵计算模块、数乘模块和压缩、解压模块的实现原理和具体细节。

第五章，多模式矩阵加速器的实现与验证。首先从 SoC 整体架构、测试数据集和功耗分析方案三个维度介绍多模式矩阵加速器的测试方案。接着，按照 ASIC 前端设计流程采用 110*nm* 工艺实现、验证多模式矩阵加速器，从性能、面积和功耗三个方面比较分析。最后，异步验证矩阵计算 SoC 的可行性。

第六章，结论。对本文工作进行总结，针对不足之处提出具有可行性的改进方向。最后针对未来需要持续深入探索的方向给出前瞻性的规划与展望。

最后是参考文献、攻读硕士期间的科研成果以及致谢。

第二章　相关技术原理

2.1　矩阵乘算法

1858 年，英国数学家阿瑟·凯莱（Arthur Cayley）发表的《矩阵论的研究报告》，打开了矩阵计算的大门。矩阵乘作为矩阵论中的基本运算之一，在众多领域应用广泛。时至今日，已有多种矩阵乘算法被提出。根据它们的算法特征，将其应用在不同的地方。给定矩阵 $A \in R^{m\times n}$，矩阵 $B \in R^{n\times p}$，矩阵 A 和矩阵 B 相乘结果为矩阵 $C \in R^{m\times p}$，用于描述此节的矩阵算法。本节介绍 5 种常见的矩阵乘算法。

2.1.1　内积算法

内积（Inner Product）作为最常见的矩阵乘算法。结果矩阵 C 由矩阵 A 中的每行逐次乘以矩阵 B 中的每列得到，即 $C_{i,j} = a_i \cdot b_j$，其中 a_i 为矩阵 A 中的第 i 行，b_j 为矩阵 B 中第 j 列。考虑如果每次只能计算长度为 n 的向量点乘的情况。对于 m 行的矩阵 A，需要访问 m 次矩阵 B，故内积算法的输入复用性较差。每次的行列点乘运算，都可以得到结果矩阵 C 中的 1 个元素值，因此外积算法的输出复用性较好。当 $m = n = p = 3$ 时，内积算法如图2-1所示。

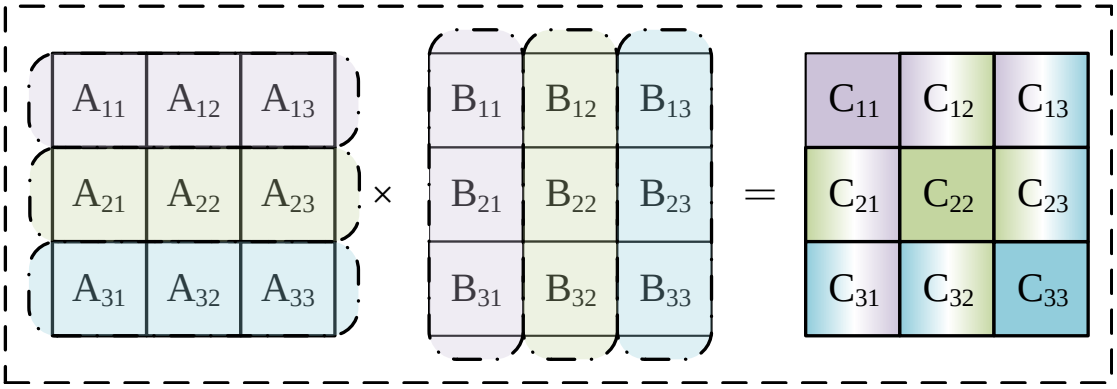


图 2-1　内积算法示意图

2.1.2　外积算法

外积算法下，结果矩阵 C 由矩阵 A 分解得到的列向量与矩阵 B 分解得到对应的行向量作外积乘法。数学公式表示为 $C = \sum_{i=0}^{n-1} C_i = \sum_{i=0}^{n-1} a_i \otimes b_i$。其中 a_i 是矩阵 A 中第 i 个列向量，b_i 是矩阵 B 中第 i 个行向量，C_i 是列向量 a_i 和行向量 b_i 外积乘法结果。如果每次只能计算长度为 n 的数乘向量运算，那么外积算法下，矩阵 A 和矩阵 B 只需要访问 1 次即可，因此外积算法的输入复用性较好。计算

过程中需要频繁访问中间结果矩阵，所以外积算法的输出复用性较差。图2-2为 $m=n=p=3$ 时，外积算法示意图。

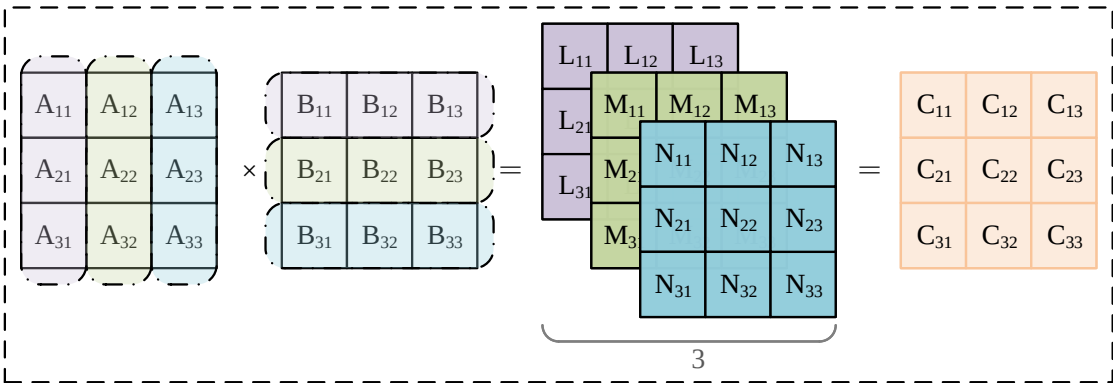


图 2-2　外积算法示意图

2.1.3　Strassen 算法

德国数学家 Volker Strassen 于 1969 年提出的 Strassen 算法[39]，将矩阵乘算法复杂度从 $\Theta(n^3)$ 缩小到 $\Theta(n^{2.81})$，其核心思想是分而治之（divide-and-conquer）。

当 $m=n=p=2$ 时，结果矩阵 C 在 Strassen 算法下的计算步骤如公式 2-1 至公式 2-10 所示[40]，可分为“加法-乘法-加法”3 个阶段，分别是公式 2-1 至 2-5、公式 2-6 至 2-8 和公式 2-9 至 2-10，共需要 18 次加法和 7 次的乘法，相较于内积算法多了 14 次加法和减少 1 次乘法。对于规模相当大的矩阵，结果矩阵 C 的计算主要被乘法支配，Strassen 算法能够有效减少乘法次数[41]，提升传统 CPU 上的矩阵乘法速度。考虑每次只能完成 m 次乘法和 m 次加法的场景。那么，Strassen 算法计算结果矩阵 C，需要多次访问矩阵 A 和矩阵 B，同时结果矩阵 C 依赖计算过程中产生的中间结果，故其输入复用性和输出复用性都比较差。

$$S_1 = B_{12} - B_{22}, \quad S_2 = A_{11} + A_{12} \tag{2-1}$$

$$S_3 = A_{21} + A_{22}, \quad S_4 = B_{21} - B_{11} \tag{2-2}$$

$$S_5 = A_{11} + A_{22}, \quad S_6 = B_{11} + B_{22} \tag{2-3}$$

$$S_7 = A_{12} - A_{22}, \quad S_8 = B_{21} + B_{22} \tag{2-4}$$

$$S_9 = A_{11} - A_{21}, \quad S_{10} = B_{11} + B_{12} \tag{2-5}$$

$$P_1 = A_{11} \times S_1, \quad P_2 = S_2 \times B_{22} \tag{2-6}$$

$$P_3 = S_3 \times B_{11}, \quad P_4 = A_{22} \times S_4 \tag{2-7}$$

$$P_5 = S_5 \times S_6, \quad P_6 = S_7 \times S_8, \quad P_7 = S_9 \times S_{10} \tag{2-8}$$

$$C_{11} = P_5 + P_4 - P_2 + P6, \quad C_{12} = P_1 + P_2 \tag{2-9}$$

$$C_{21} = P_3 + P_4, \quad C_{22} = P_5 + P_1 - P_3 - P_7 \tag{2-10}$$

2.1.4 Gustavson 算法

Gustavson 算法[42]在 1978 年被瑞典学者 Fred G Gustavson 提出，也被称为 row-wise 积方法，其数学表达式为 $c_i = \sum_{j=0}^{m-1} A_{i,j} \cdot b_j$。其中 c_i 是结果矩阵 C 中的第 i 个行向量，$A_{i,j}$ 是矩阵 A 第 i 行中的非零元素，b_j 是矩阵 B 中第 j 行向量。结果矩阵 C 中的第 i 行由矩阵 A 第 i 行中的非零元素乘以矩阵 B 中对应行中所有的非零元素得到的中间向量累加后得到，矩阵 A 中的非零元素 $A_{i,j}$ 所在的列索引 j 即为矩阵 B 中需要进行乘法的行向量索引。采用 Gustavson 算法，将计算任务合理分配后，可以并行计算结果矩阵 C 中的行[27]。

如果每次只能计算长度为 n 的数乘向量，与内积算法相相同，对于 m 行的矩阵 A，需要访问 m 次矩阵 B，因此 Gustavson 算法的输入复用性较差，输出复用较好。当 $m = n = p = 2$ 时，Gustavson 算法如图2-3所示。

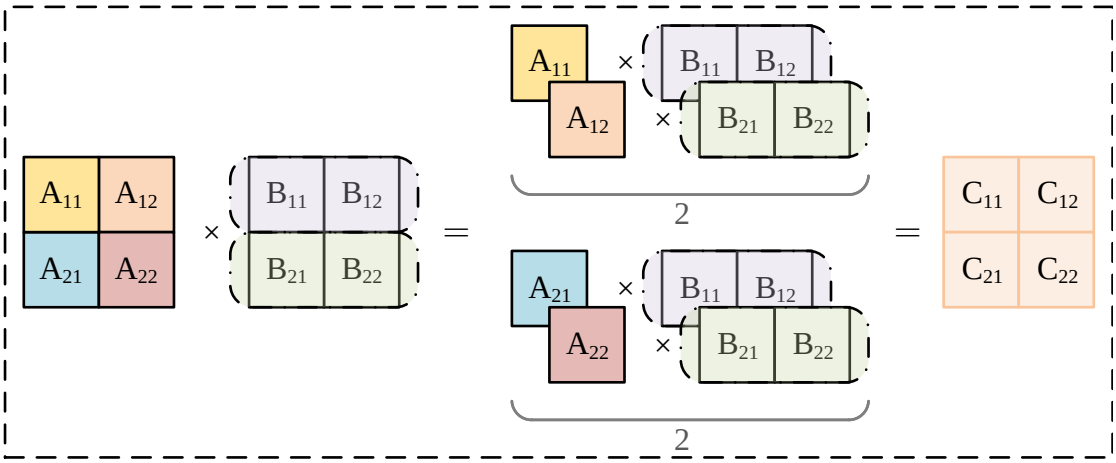


图 2-3　Gustavson 算法示意图

2.1.5 Cannon 算法

在计算机科学中，Cannon 算法[43]是一种用利用二维网格完成矩阵乘法的分布式算法，由 Lynn Elliot Cannon 于 1969 年首次提出。Cannon 算法不同于 2.1 节至 2.4 节提到的算法，它需要足够的计算单元，并充分利用这些计算单元。数据在计算单元中的移动，使得数据被暂存、复用。数据的每次移动，计算单元能够完成一次的乘累加计算。当所有的数据输入计算单元后，即可完成复杂的矩阵乘法。

假设 $m = p = n$，即矩阵 A、矩阵 B 和矩阵 C 均为 n 维方阵。Cannon 算法要求对矩阵 A 和矩阵 B 进行预处理，将其分别转换为维度为 $n \times (2n-1)$ 和 $(2n-1) \times n$ 的输入矩阵。当 $n = 3$ 时，Cannon 算法如图2-4中所示，9 个计算单元（$cell$）组

成的简单 2 维 mesh 型方阵，每个 *cell* 中有 3 个寄存器，分别是 $l(i,j)$、$t(i,j)$ 和 $v(i,j)$，$l(i,j)$ 暂存从左侧传来的数据，$t(i,j)$ 暂存从上方传来的数据，$v(i,j)$ 暂存当前计算单元上的中间结果。输入矩阵的预处理、近似于 2 倍的存储（$2-\frac{1}{n}$）和 n^2 个乘累加单元，使得 Canon 算法在这种场景下拥有较好的输入复用性和输出复用性。

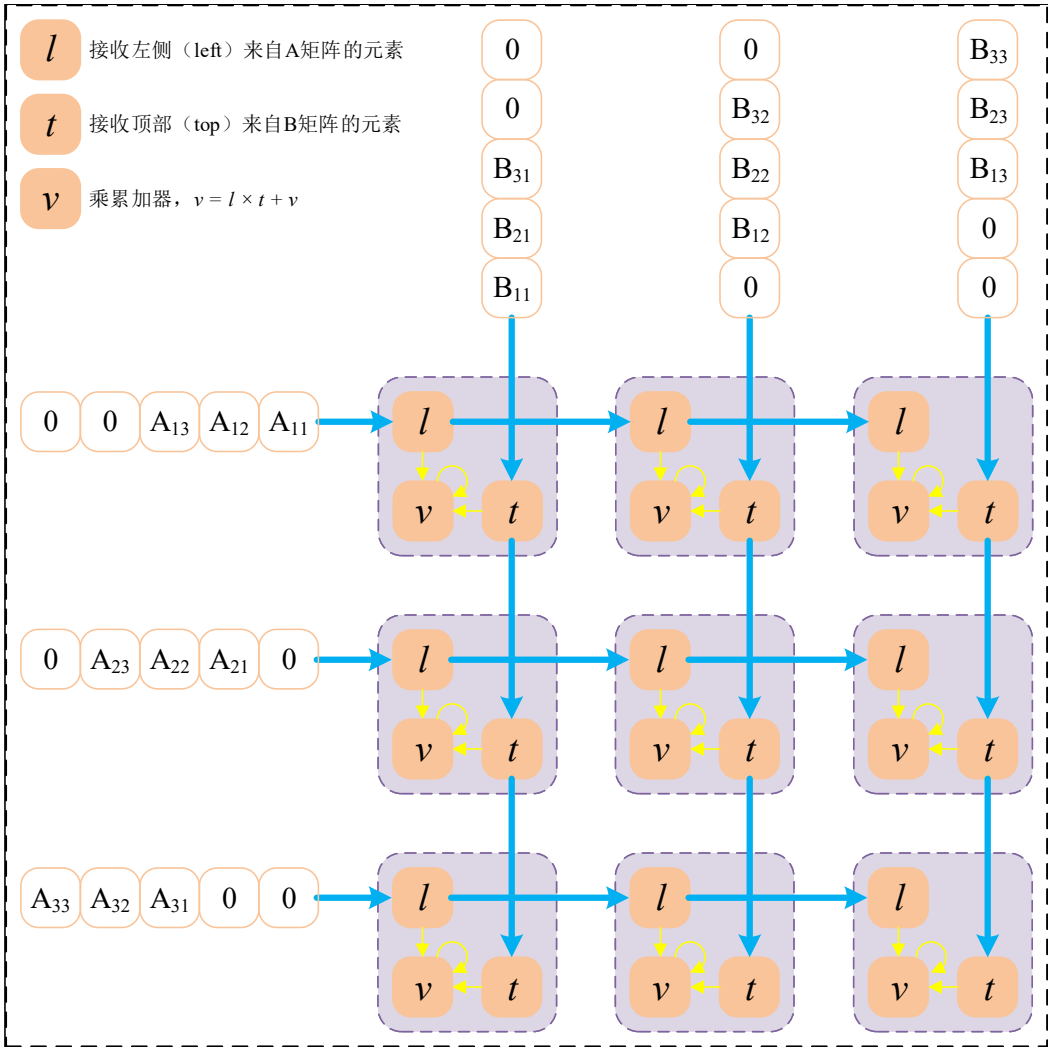


图 2-4　Cannon 算法示意图

2.2　稀疏矩阵存储格式

一般矩阵存储方案（即，二维数组）存储稀疏矩阵是低效且高功耗的，因为大部分的内存被用来存储零值元素，且零值元素参与运算。通过采用稀疏矩阵存储格式，提高存储的利用效率。假设稀疏矩阵 $A \in R^{m \times n}$，记非零元素个数为 N，用于此节存储空间的分析。本节将介绍 COO、CSR、BCSR、ELL、DIA 和 HYB 六种常见的稀疏矩阵存储格式。

2.2.1　COO 存储格式

COO 存储格式（Coordinate）[44]由三元组 $(row_idx,\ col_idx,\ value)$ 构成，三元组的长度等于非零元素的个数。其中，$value$ 存储稀疏矩阵中的非零元素，row_idx, col_idx 存储非零元素行索引（row index）与列索引（column index）构成的坐标。这是一种最为直观稀疏矩阵的存储方案。存储稀疏矩阵 A 需要存储空间 $(size_{value}+size_{row_idx}+size_{col_idx})\times N$，存储关系是 $\Theta(3N)$。图2-5为 $m=n=4$ 下的 COO 存储格式示意图。

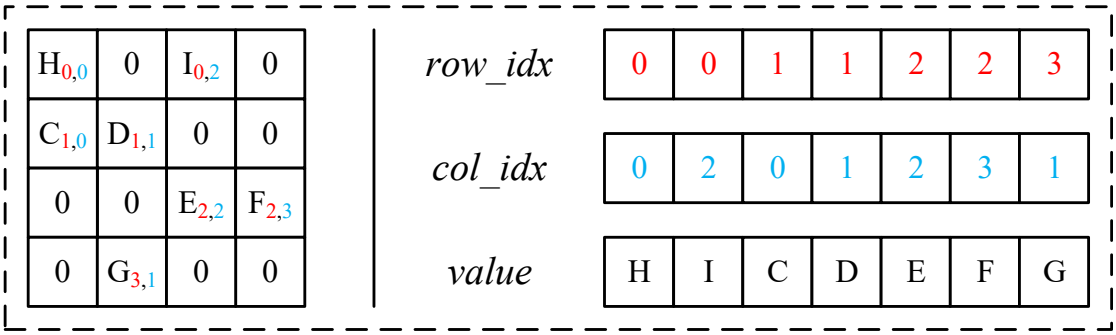


图 2-5　COO 存储格式示意图

2.2.2　CSR、CSC 存储格式

CSR 存储格式（Compressed Spase Row）[45]是由行指针（row pointer）、列索引和非零元素构成的三元组 $(row_ptr,\ col_idx,\ value)$。其中，$col_idx$ 中存储非零元素的列索引，$value$ 为非零元素值，$row_ptr[i]$、$row_ptr[i+1]$ 分别表示第 i 行非零元素与对应的列索引值在数组 $value, col_idx$ 中的起始位置、结束位置。通常数组 $value$ 和 col_idx 的从索引 0 开始存放有效数据，那么第 1 个行指针的值恒为 0，第 $m+1$ 个行指针的值等于非零元素个数 N。row_ptr 长度固定等于 $m+1$，col_idx 和 $value$ 的长度等于 N。因此，存储稀疏矩阵 A 需要存储空间 $size_{row_ptr}\times(m+1)+(size_{col_idx}+size_{value})\times N$，存储关系近似 $\Theta(2N+m)$。图2-6（a）为 $m=n=4$ 时的 CSR 存储格式示意图。

不同于 CSR 存储格式是按照行主序进行压缩，CSC 存储格式（Compressed Sparse Column）是按照列主序进行压缩，由列指针（column pointer）、行索引和非零元素组成的三元组 $(col_ptr,\ row_idx,\ value)$。图2-6（b）是同一稀疏矩阵在 CSC 格式下的存储示意图。其中 col_ptr 长度固定等于 $n+1$，row_idx 和 $value$ 的长度等于 N。因此，存储稀疏矩阵 A 则需要存储空间 $size_{col_ptr}\times(n+1)+(size_{row_idx}+size_{value})\times N$，存储关系是 $\Theta(2N+n)$。

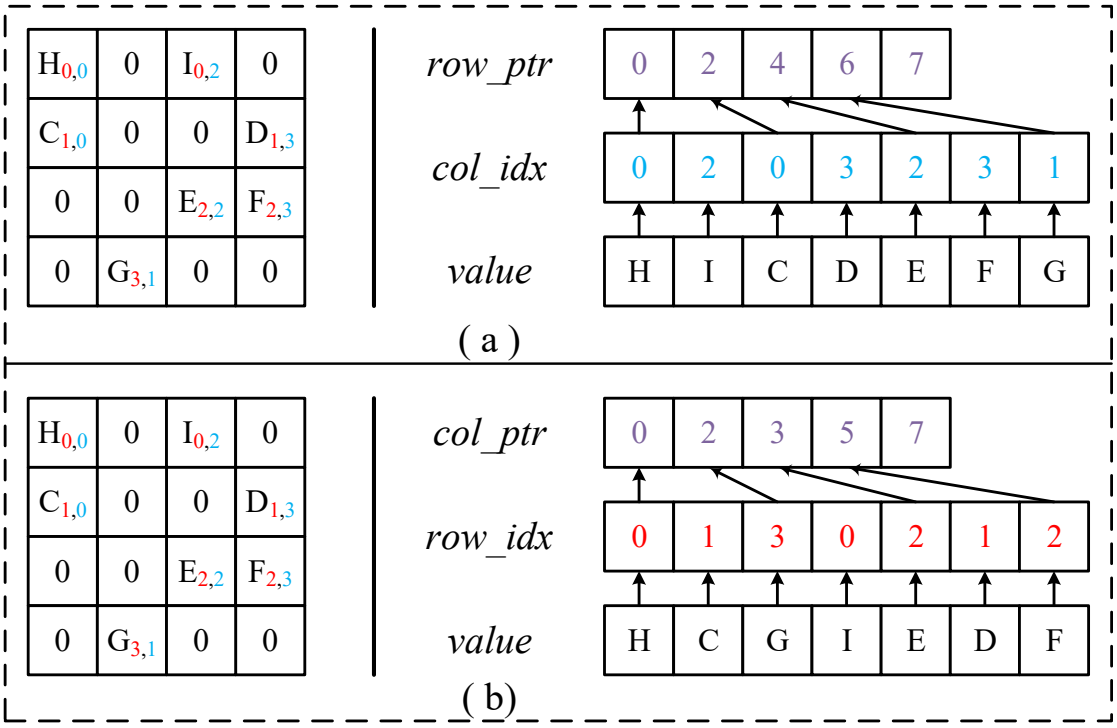


图 2-6　CSR、CSC 存储格式示意图

2.2.3　BCSR、BCSC 存储格式

BCSR 存储格式（Block Compressed Sparse Row）[44]将输入矩阵进行分块，存储存在非零元素的子矩阵。BCSR 格式同样由三元组 (row_ptr, col_idx, $value$) 构成，其中 $value$ 存放子矩阵。假设块大小为 $r \times c$，如果 $r = c = 1$，此时 BCSR 存储格式等同于 CSR 存储格式。如果有效块的个数是 k，那么需要存储空间 $size_{value} \times (r \times c \times k) + size_{row_idx} \times \lceil \frac{m}{r} \rceil + size_{col_idx} \times k$ 存储稀疏矩阵 A，存储关系是 $\Theta((rc+1)k+n)$。

BCSC 是 BCSR 的转置的格式，与 CSR 和 CSC 之间的关系相同。图2-7展示同一矩阵分别在 BCSR 和 BCSC 下的存储示意图，其中 $m = n = 6$，子矩阵大小是 2×2。

2.2.4　ELL 存储格式

ELL 存储格式（ELLPACK）[46]使用二元组 (col_idx, $value$) 存储稀疏矩阵，其中 col_idx 和 $value$ 都是二维数组，col_idx 存储非零元素的列索引，$value$ 存储非零元素。记稀疏矩阵 A 中非零元素最多行的非零元素个数为 max，若采用 ELL 存储格式存储稀疏矩阵 A，那么 col_idx 和 $value$ 的大小为 $n \times max$。非零元素个数不足 max 的行，$value$ 数组需要填充 0，col_idx 中对应的索引位置填充 -1。所以 ELL 存储格式存储稀疏矩阵 A 耗费存储空间 $(size_{col_idx} + size_{value}) \times n \times max$，存储关系是 $\Theta(n \cdot max)$。可以看出，ELL 数组在存储非零元素分布不均匀的稀疏

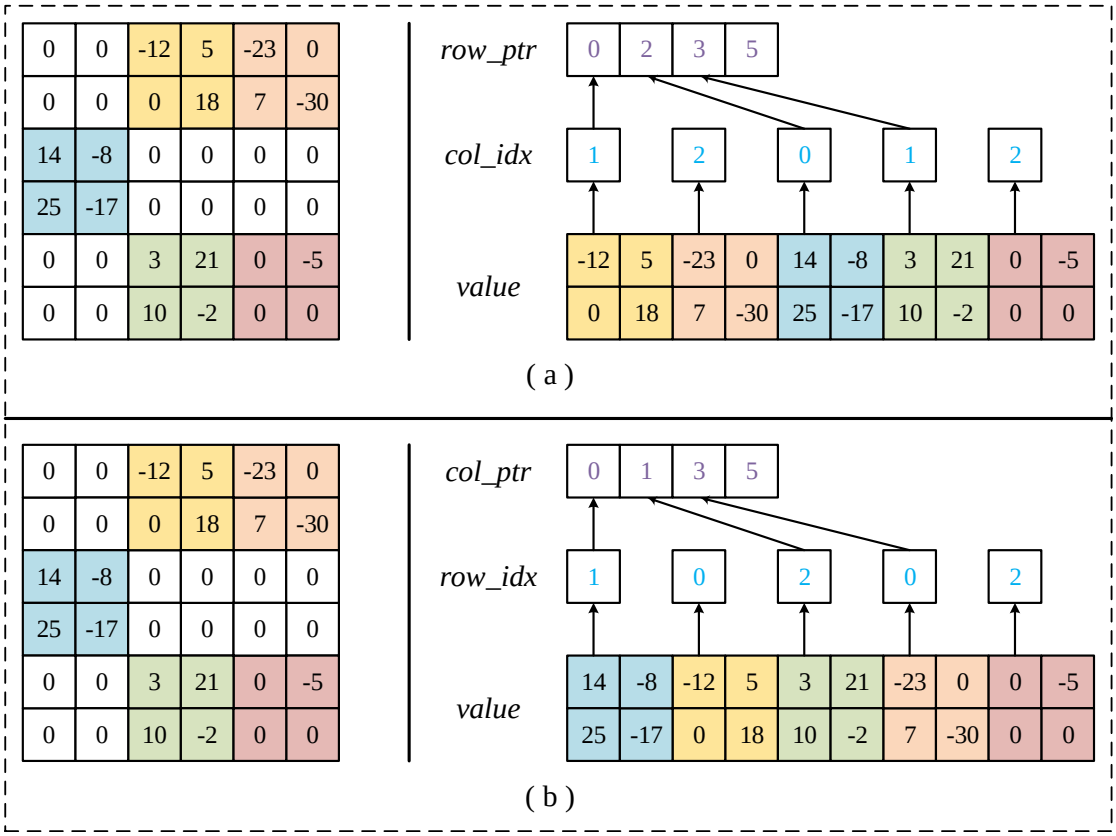


图 2-7　BCSR、BCSC 存储格式示意图

矩阵，存储效率低下。当 $m=n=4$ 时，图2-8是 ELL 存储格式示意图。

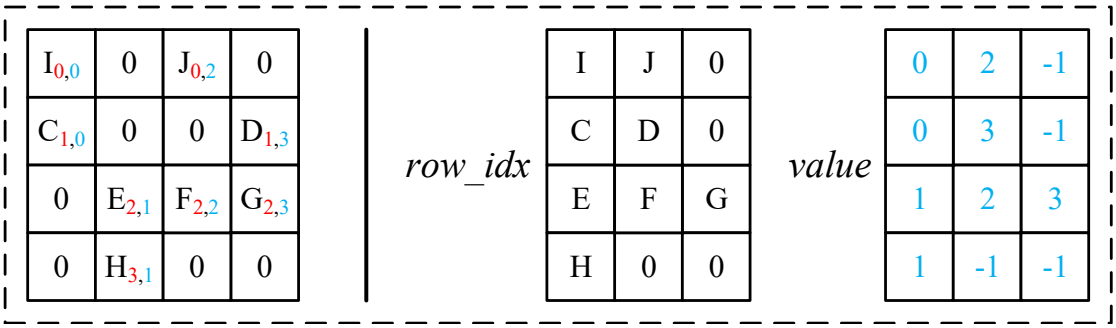


图 2-8　ELL 存储格式示意图

2.2.5　DIA 存储格式

DIA 存储格式（Diagonal）[47] 使用 *value* 和 $offsets$ 两个数组存储稀疏矩阵。其中 *value* 是存储非零元素二维数组，不存在非零元素的位置需要补零。一维数组 $offsets$ 用于存储每个对角线与中央主对角线的偏移量，规定主对角线的偏移量是 0，主对角线的上方和下方分别规定为正偏移量和负偏移量。DIA 存储格式主要是为大部分非零元素密集、规则分布在对角线附近的稀疏矩阵设计。如果沿对角线只存在很少的非零元素，DIA 格式可能会浪费大量的内存空间，降低计算

效率。图2-9展示非零元素 H 位置不同的两个稀疏矩阵在 DIA 格式下的存储示意图。图2-9（b）相较于图2-9（a）DIA 存储格式引入更多的零值填充，且存储效率极低。如果 d 表示有一个非零元素的输入稀疏矩阵的对角线数，则 DIA 格式所需的内存大小是 $size_{value}\times min(m,n)\times d+size_{offset}\times d$，存储关系近似 $\Theta(min(m,n)d)$。

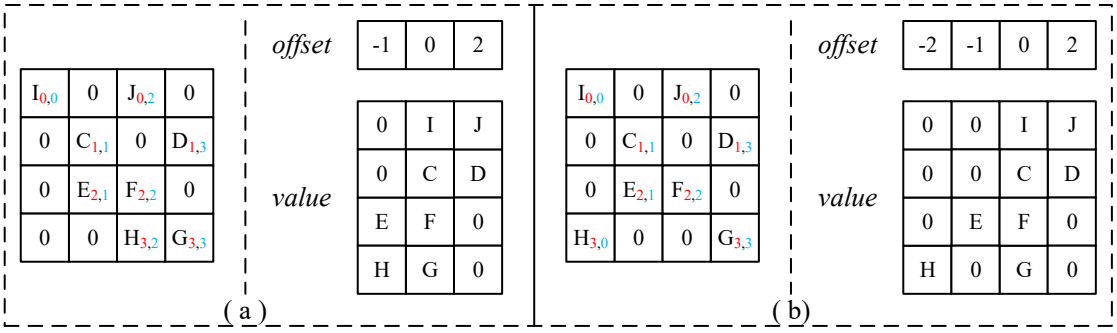


图 2-9　DIA 存储格式示意图

2.2.6　HYB 存储格式

HYB（Hybrid）是一种混合型存储格式，由 ELL 和 COO 两种格式组成[48]，如图2-10所示。HYB 格式需要指定一个阈值 k，即 ELL 格式内每行最多能存多少个非零元素。ELL 格式无法存储的部分用 COO 存储格式进行存储。ELL 格式每行至多存储 k 个非零元素，而 COO 存储不能被 ELL 格式存储的其余非零元素。图2-10中阈值 k 等于 2，稀疏矩阵 A 的维度 $m=n=4$。HYB 格式所需的空间为 $size_{value}\times(m\times k+X)+size_{index}\times(m\times k+2X)$，其中 X 是 COO 格式处理的非零值的数量。在通常情况下，ELL 格式填充零元素的个数要远大于超出阈值 k，因此阈值的选取是影响 HYB 格式存储效率的关键因素。

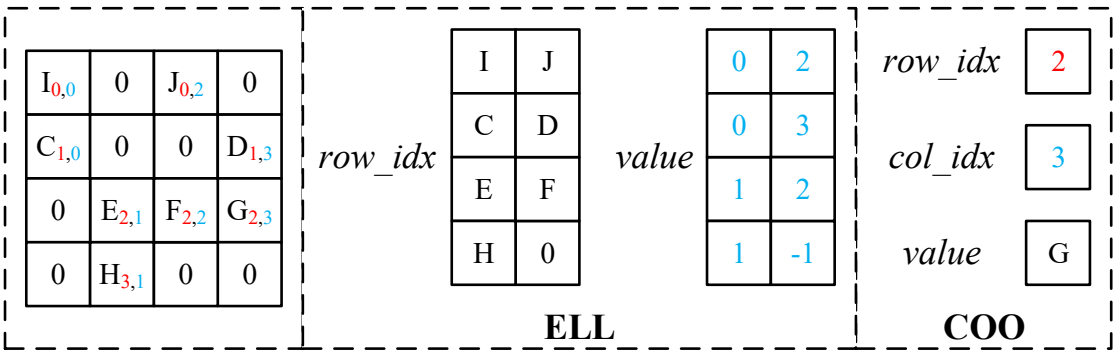


图 2-10　HYB 存储格式示意图

2.2.7　Bitmap 存储格式

Bitmap 存储格式使用和稀疏矩阵相同维度的二维矩阵 *Bitmap* 来记录稀疏矩阵中非零元素的存在情况，*Bitmap* 中的每个元素的大小为 1 比特（bit），如

图2-11所示。Bitmap 格式存储稀疏矩阵 A 需要存储空间 $m \times n + size_{value} \times N$。

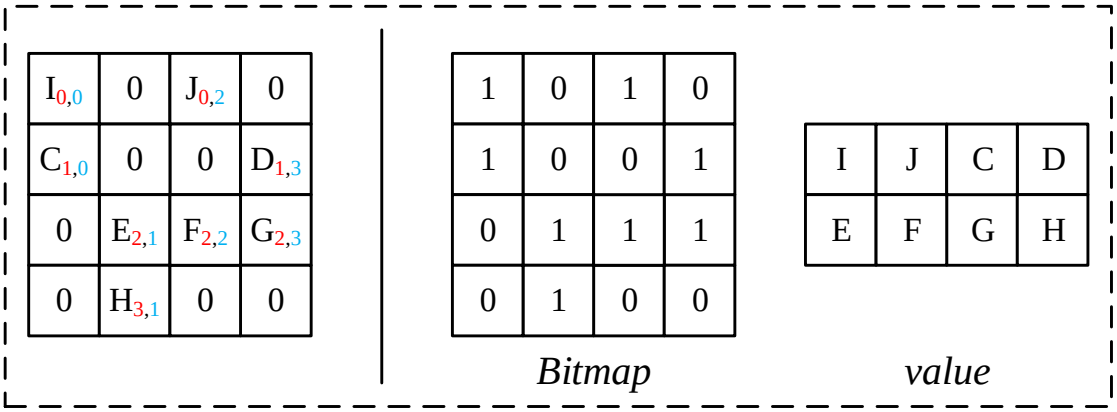


图 2-11　Bitmap 存储格式示意图

2.2.8　Fibertree 存储格式

Fibertree 格式是一种用树结构表示张量的存储格式，每一层对应张量中的一个被标记的秩（rank）。以二维张量稀疏矩阵 A 为例，它的秩分别是 m 和 n。Fibertree 中每层的顺序表示秩的顺序，若按照行主序使用 Fibertree 存储稀疏矩阵 A，那么秩的顺序记作 $[m, n]$。每层包含一个或多个 $fiber$，$fiber$ 是一个在当前 $rank$ 上有着相同索引的元素的集合。当 $fiber$ 是中间节点时，它指向下一层对应的 $fiber$。当 $fiber$ 时叶节点时，它负载一个非零元素。图2-12是 m, n 等于 4 的稀疏矩阵 A 用 Fibertree 格式存储示意图。Fibertree 存储稀疏矩阵所需的存储空间近似为 $size_{value} \times N + size_{node} \times (m + N)$，其中 node 指代 Fibertree 中的索引节点，指代图2-12中用椭圆圈住的每一个索引。

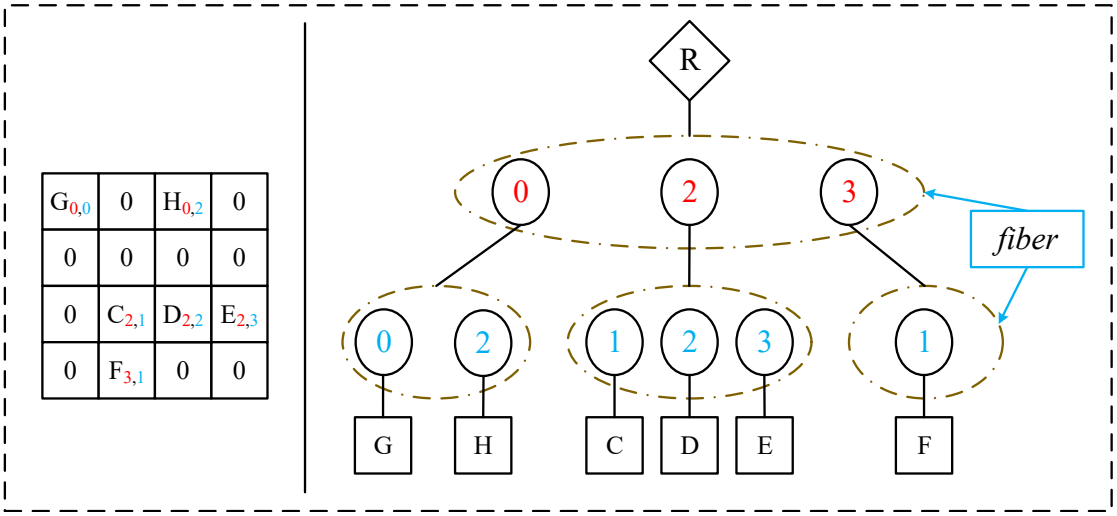


图 2-12　Fibertree 存储格式示意图

2.3　异步电路设计方法

同步时钟架构在过去的几十年中一直是数字电路设计的焦点。与同步电路相比，异步无时钟电路的功耗更小、噪声更低、电磁干扰更小，同时不会降低电路性能。异步电路设计展现出的这些优势，重新引起人们对异步电路设计的兴趣。[49]异步电路可以划分为两大类：限定延迟模型和延迟非敏感模型。本节将从异步控制器、NCL（NULL Convention Logic）设计方法和粗粒度异步微流水线三个方面进行介绍。

2.3.1　异步控制器

异步控制器在异步电路中扮演核心角色，通过握手协议完成模块间的通信交互。C-element[50]、Click[51]和 GasP[52]作为三种主流的异步控制器，广泛使用于诸多异步电路设计中。

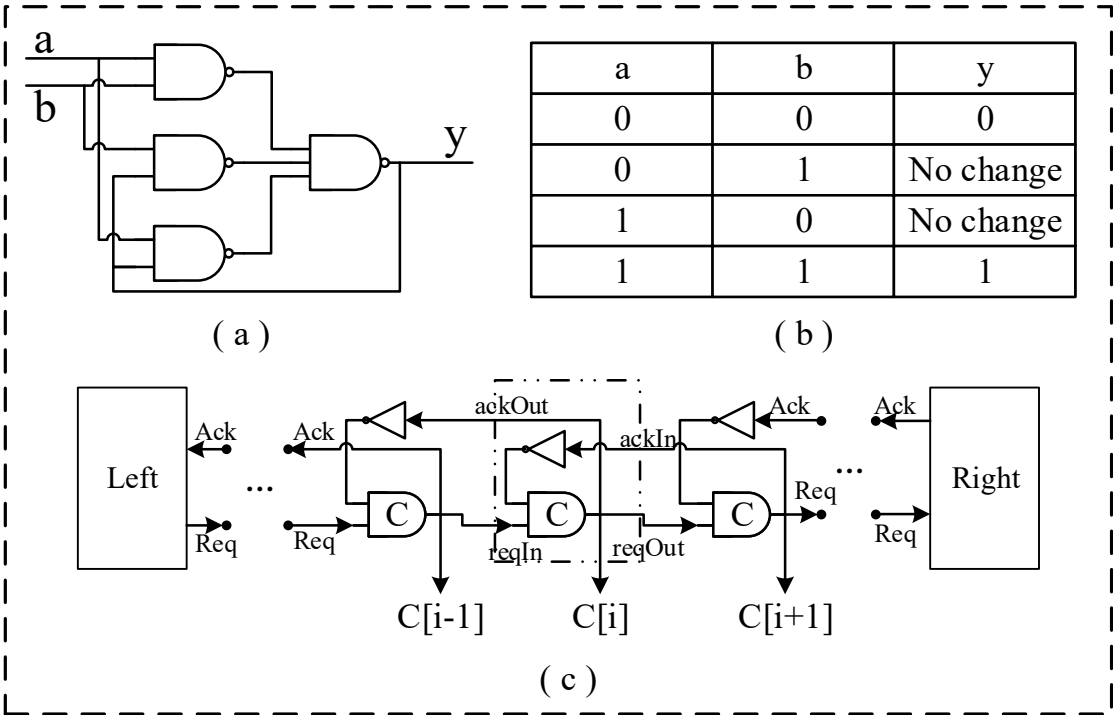


图 2-13　C-element 示意图

（1）C-element 控制器：该控制器由 Muller 在 19 世纪 60 年代左右提出，是最早的异步控制器。它是一个类似于异步复位锁存器的状态保持单元，有多种实现形式，图2-13(a) 是基于标准单元的一种实现。C-element 控制器有两个输入端 a,b 和一个输出端 y，其逻辑表达式为 $y=(a\&b)|(a\&c)|(b\&c)$，真值表如图2-13(b)。当输入 $a,b=0$ 时，输出 $y=0$。当输入 $a,b=1$ 时，输出 $y=1$。其它输入组合下，输出 y 保持不变。图2-13(c) 是由 C-element 控制器和反相器组成的电路，被称为 Muller 流水线[53]。初始状态下所有的 C-element 信号被初始化成 0。以 C[i] 作为

参照物，当 C[i] 的 *reqIn* 从 0 到 1，则说明 C[i-1] 传来一次的请求。C[i] 处理完当前请求后，将请求传递给 C[i+1]，同时给 C[i-1] 应答，拉高 *ackOut*。C[i-1] 收到应答后，拉低 C[i] 的 *reqIn*；C[i] 的 *ackIn* 端口收到 C[i+1] 的应答后，将 *reqOut* 拉低。该 Muller 流水线基于四相握手协议，其变种与扩展构成众多异步电路设计的基石。

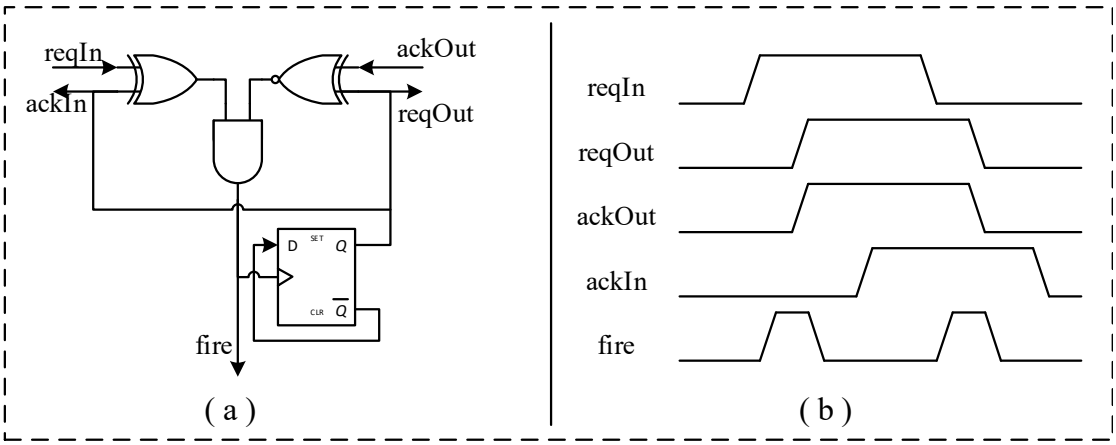


图 2-14　Click 示意图

（2）Click 控制器：Willem Mallon 等人提出采用两相握手协议的 Click 控制器。Click 由异或门、同或门、与门和非门共 4 个标准单元和 1 个 D 触发器组成，共有 2 个输入和 3 个输出，其电路结构如图2-14(a) 所示。不同于四相握手协议，两相握手协议不需要归零的操作，无需对相邻控制器发来的应答作出响应。因此相同情况下，基于两相握手协议的控制器能够较好地节省不必要的事件和能量。在两相握手协议中，电平从 0 到 1 和 1 到 0 都表示一个事件。图2-14(b) 是 Click 的时序逻辑图。在 Click 构成的异步微流水线中，Click 中的信号 *reqIn*、*reqOut*、*ackIn* 和 *ackOut* 负责与级联的 Click 进行通信，Click 产生脉冲信号 *fire* 触发寄存器组驱动数据在电路中的传输。初始化后，Click 的输入、输出信号均处于低电平状态。当 Click 的输入端 *reqIn* 发生从 0 到 1 的跳变，代表 Click 接收到一次事件，随后 Click 的 *fire* 端会产生一个脉冲，拉高 *reqOut* 将事件传递给下一个 Click，拉高 *ackIn* 给上一个 Click 应答。

（3）GasP 控制器：Ivan Sutherland 等人于 2001 提出 GasP 控制器，该控制器采用模拟电路搭建，遵循四相握手协议。如图2-15所示，GasP 电路有 1 个输入信号 *pred*、2 个输出信号 *succ* 和 *fire*，其中 *pred* 和 *succ* 信号均为双向状态线既表示请求也表示应答。状态线有 FULL 和 EMPTY 两种状态，FULL 表示当前该信号线上有数据，不再接收新数据；EMPTY 表示当前该信号线上无数据，可以接收新数据。GasP 只有在 *pred* 为 FULL，*succ* 为 EMPTY 时产生 $fire$ 信号进行数据传输。虽然 GasP 在速度、面积和功耗方面有较好的表现，但在不同的制程工艺

下，需要定制 GasP，相较于能够使用标准单元搭建的异步控制器难度较大[54]。

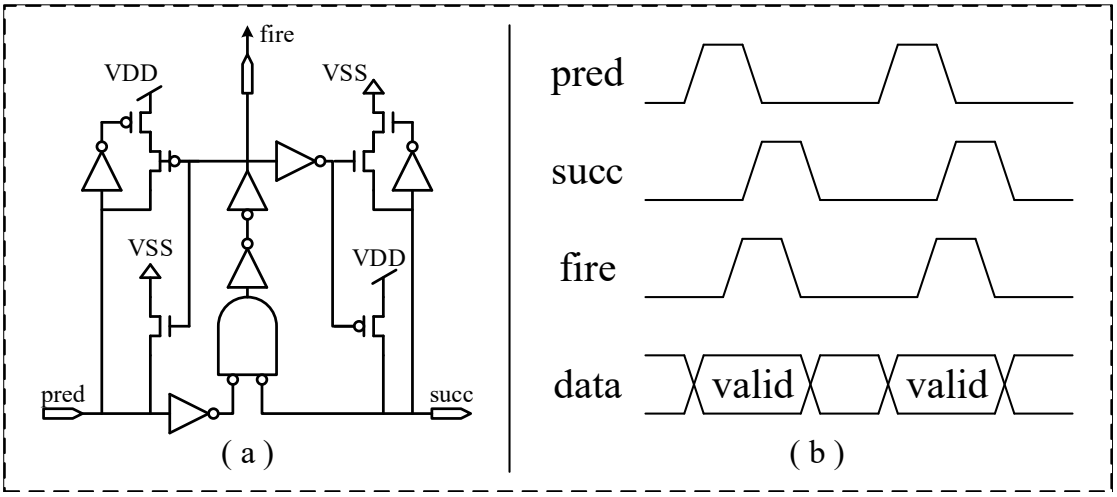


图 2-15　GasP 示意图

2.3.2　NCL 异步电路设计方法

NCL 是一种延迟非敏感的异步无时钟电路设计方法，有着“构造即正确”的特点，即无需进行时序分析就能保证电路正确工作[49]。NCL 电路利用双轨或四轨编码来实现延迟非敏感，因此对于相同功能的电路，NCL 实现的电路面积往往大于单轨逻辑电路。以 1bit 的信号 D 为例，在双轨逻辑下使用 D^0 和 D^1 两条导线表示集合 DATA0、DATA1、NULL，其中，DATA0 表示布尔逻辑 0，DATA1 表示布尔逻辑 1，NULL 表示当前的数据无效。对于一个四轨信号 Q，包含 Q^0、Q^1、Q^2 和 Q^3 四条导线，构成集合 DATA0、DATA1、DATA2、DATA3、NULL。四轨信号 Q 可以表示 2bit 的布尔逻辑信号，DATA0-DATA3 依次对应 $00_2 - 11_2$。双轨编码 D 与四轨编码 Q 均为独热码编码，表示有效数据时，只能有一个导线为 1。双轨编码 D 和四轨编码 Q 的具体表示如表 2-1和表 2-2所示，其中四轨编码 Q 只列出有效编码。

表 2-1　双轨逻辑

	DATA0	DATA1	NULL	Illegal
D^0	1	0	0	1
D^1	0	1	0	1

NCL 电路是由具有迟滞性状态保持特性的阈值门实现，这种设计方法最早由特修斯逻辑公司（Theseus Logic, Inc.）提出[55]。27 个基础阈值门[49]是 NCL 电路基石，实现四变量内的所有布尔函数。常见的一种阈值门记作 $THmn$，其中 $1 \le m \le n$，表示该阈值门有 n 个输入。另一种阈值门被称为加权阈值门，记

表 2-2　四轨逻辑

	DATA0	DATA1	DATA2	DATA3	NULL
Q^0	1	0	0	0	0
Q^1	0	1	0	0	0
Q^2	0	0	1	0	0
Q^3	0	0	0	1	0

作 $THmnWw_1w_2\ldots w_R$，其中权重的值满足 $m \geq w_R \geq 1$，带权重的输入个数 R 满足 $1 \leq R \leq n$。权重 $w_1, w_2 \ldots, w_R$ 依次对应输入 $input1, input2 \ldots, inputR$。当至少有 m 个输入被拉高时，阈值门的唯一输出将被置 1。当所有的输入都被拉低时，其输出才会被拉低。图2-16(a) 表示阈值门 $TH34$，图2-16(b) 表示加权阈值门 $TH34W2$。

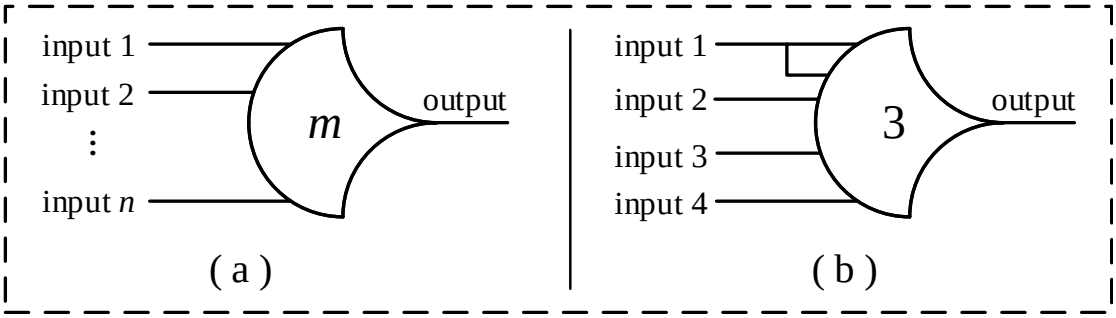


图 2-16　阈值门与加权阈值门示意图

如图2-17(a) 所示，NCL 系统的框架由 DI 寄存器以及夹在它们间的 DI 组合逻辑组成，输入由局部握手信号和完备性检测控制。NCL 系统至少包含输入和输出两级 DI 寄存器组。两个相邻寄存器层级通过请求（K_i）和应答信号 (K_o) 来交互。应答信号与完备性检测电路结合起来生成请求信号，发送到前一级寄存器。应答信号由异或阈值门逻辑生成，当寄存器输出为 DATA 时 K_o 为 rfn（$requestfornull$）（即逻辑 0），输出为 NULL 时 K_o 为 rfd（$requestfordata$）（即逻辑 1）。当 K_i 为 rfd 时 DI 寄存器组传递 DATA，K_i 为 rfn 时 DI 寄存器组传递 NULL。图2-17(b) 表示单比特双轨 DI 寄存器，阈值门符号中的 n 表示复位后为逻辑 0，如果 n 的位置为 d 表示复位后为数据。

2.3.3　粗粒度异步微流水线

可组合性是异步微流水线最重要的特质之一，使得复杂的功能如同编程一般能够从最简单模块开始搭建，更复杂的系统能够被层次化地从已有设计中构建[56]。到目前为止已有众多基于异步微流水线的复杂体系结构被提出、实现，包含片上网络架构[57,54,58]、安全加密芯片[59-60]、可重构芯片[61]、矩阵加速器[14,17]、

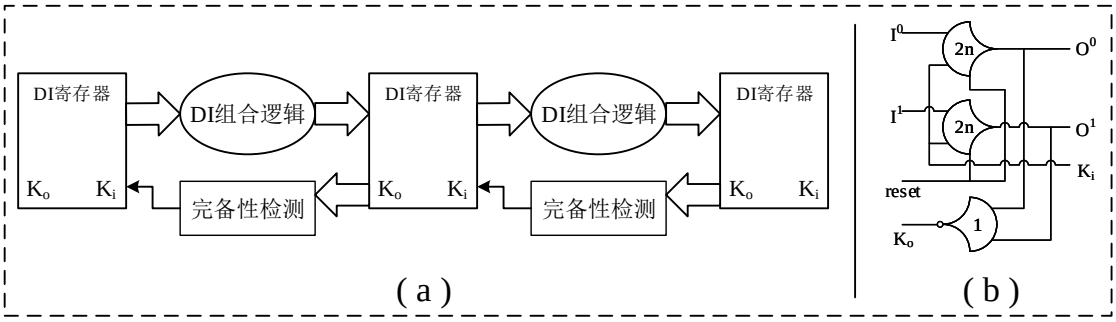


图 2-17　NCL 系统框架与 DI 寄存器结构图

机器学习加速器[62-65]、类脑芯片[66-67]和 RISC-V CPU[68-69]等。随着半导体工艺的发展，电路系统的集成度愈高，内部模块并发工作、通信与控制逻辑密切相关。异步控制器级联易于构建如先进先出（First In First Out, FIFO）这样的简单结构，设计复杂的电路系统却是较为繁琐。因此一些特殊结构被提出，用于搭建复杂电路中的控制逻辑，例如基于 C-element 的 Fork、Join、Merge 结构[53]，6 种“Link-Joint”结构[70]。兰州大学异步电路与系统团队归纳总结国内外异步电路方法学成果，提出符合约束数据绑定（Bounded Bundle Data, BBD）[71]的“发送-中继-接收”模型[14]，进而提出粗粒度异步微流水线结构[69]。

“发送-中继-接收”模型包含发送者（Sender）、中继者（Relay）、带允许机制的中继者（pmtRelay）和接收者（Receiver）四种基础结构。发送者结构置于异步微流水线的起始位置，在接收到外部事件之后，将事件转换成中继者能够识别的事件，驱动中继者。中继者结构置于异步微流水线中间，其本质是异步控制器。中继者输出局部脉冲信号驱动寄存器组进行数据拷贝，输出握手信号将事件传输到邻接组件。在允许信号有效的前提下，带允许机制的中继者才能够正常工作，否则将阻塞事件的传递。接收者结构位于当前异步微流水线的结束位置，在收到握手事件后，中继者不会立刻应答与之级联的中继者，而是进入等待状态，直到释放事件的出现，至此，异步微流水线的一轮执行结束。

基于“发送-中继-接收”模型构建的粗粒度异步微流水线包含 FIFO、Split 和 Merge 三种类型的结构。

图2-18(a) 是 FIFO 类结构，主要包含 Fifo、LastFifo 和 PmtFifo 三种结构。Fifo 结构收到激活事件后，会依次产生多个脉冲信号 $fire$ 控制数据流动，最后一个脉冲事件产生后经过一定的延迟后产生给上一个微流水线的应答和给下一个微流水线的请求。与 Fifo 不同，LastFifo 无需下一级的释放事件即可复位。PmtFifo 中插入了带允许机制的中继者，当 pmt 为 0 时，异步微流水线将暂停产生脉冲信号 $fire$；当 pmt 为 1 时，异步微流水线才能正常工作产生脉冲信号。

图2-18(b) 是 2 路的 Merge 类结构，主要包含 ConfMerge、WaitMerge 和 Ar-

bitMerge 三种结构。在 ConfMerge 中各路输入是互斥关系，适用于同一时刻只会有其中一路输入到来的情况，ConfMerge 将到来事件和数据发送到相邻模块。WaitMerge 是各路事件全部到来后，再往下一级发送事件和数据。ArbitMerge 用于存在多个输入可能会同时到来的情景，能够根据预置的各个输入的优先级情况，有序将各个输入的数据和事件输出到下一级。

图2-18(c) 是 2 路的 Split 类结构，主要包含 NaturSplit 和 SelSplit 两种结构。NanurSplit 负责将一路数据和事件复制、传输到多个分支。SelSplit 中包含有效信号 $valid$，以 SelSplitN 为例，表示 N 路的条件分支，存在 $valid_0, valid_1 \cdots, valid_{N-1}$ 共 N 个有效信号，当激活事件到来时，数据和事件只从 $valid$ 信号为 1 的分支传输出去。

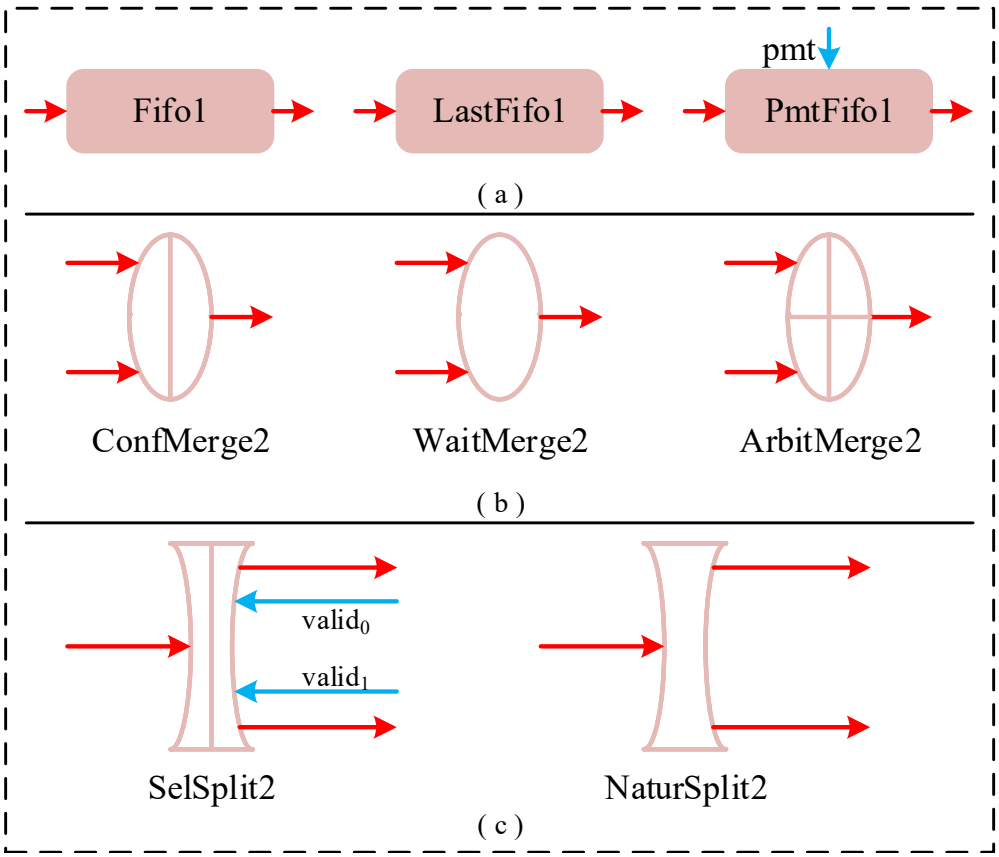


图 2-18　FIFO、Merge 和 Split 结构示意图

2.4　本章小结

首先，本章的第一节介绍了 5 种常用的矩阵算法并从输入、输出复用性的角度进行简要地分析。其次，第二节介绍了 8 种稀疏矩阵的存储格式并分析了它们的存储效率。最后，从异步电路控制器、NCL 异步电路设计方法和粗粒度异步微流水线 3 个维度对异步电路设计进行了较为全面的介绍。

第三章 矩阵加速器的存储格式、分块策略与算法分析

矩阵加速器的架构设计与采用的存储格式、分块策略和算法密不可分，本章在 2.1 节和 2.2 节的基础上分析本研究对存储格式、分块策略和算法的选择。

3.1 稀疏矩阵存储格式分析

相较于 2.2 节中介绍的其它存储格式，COO 与 CSR 格式在面向非零元素不规则分布的稀疏矩阵时，具有更好的通用性与更高的存储效率，同时避免了零值元素造成的无效计算[35]。稀疏矩阵中非零元素是否满足特定的分布特征，直接影响 ELL、DIA 格式存储效率的高低。HYB 格式需要分析稀疏矩阵中非零元素的分布，选取合适的阈值后，才能达到较高的存储效率。使用 Bitmap 格式存储大规模稀疏时，大量固定的存储空间被浪费。Fibertree 格式用于存储低阶张量时，面临较高的构建和维护代价。考虑到处理大规模稀疏矩阵时，分析稀疏矩阵本身就是一大开销，本研究从通用性好、效率高的 COO、CSR 和 BCSR/BCSC 格式中进行选择。

BCSR/BCSC 格式的存储效率因子矩阵大小与非零元素分布波动，由于其是 CSR 格式的变形，其存储效率的上限与 CSR 相差不大。对于 COO、CSR 格式，考虑如下场景，矩阵的规模是 128×128，每个矩阵元素为 32 bit 的有符号数。比较矩阵内部的非零元素个数（N）为变量的情况下，COO、CSR 和稠密矩阵 3 种存储方案的存储空间（R）开销变化。

对于 COO 存储格式，存储非零元素的坐标需要 14 bit，7 bit 存储非零元素所在行，7 bit 存储非零元素所在列。存储空间与非零元素个数之间的关系为：

$$R_{COO} = (7+7+32) \times N \div 8 = \frac{23}{4}N \tag{3-1}$$

对于 CSR 存储格式，存储非零元素的一个指针（ptr）需要 15 bit，共 129 个；需要 7 bit 存储非零元素所在列索引（idx）。存储空间与非零元素个数之间的关系为：

$$R_{CSR} = [(32+7)N + 129 \times 15]/8 = \frac{39}{8}N + \frac{1935}{8} \tag{3-2}$$

采用稠密矩阵存储稀疏矩阵，需要固定的存储空间 $128 \times 128 \times 4$ 等于 65536 B。图3-1为它们的函数关系在坐标轴上的表示。在上述情景中，根据图中 3 个函数关系的交点，计算分析可知：（1）当矩阵的稀疏度达到 98.32% 以上时，COO 相较于 CSR 存储格式更具优势；反之，采用 CSR 存储格式更具优势;（2）稀疏度低于 30.4% 时，稠密矩阵的存储效率高于 COO 格式；（3）稀疏度低于 18.26%

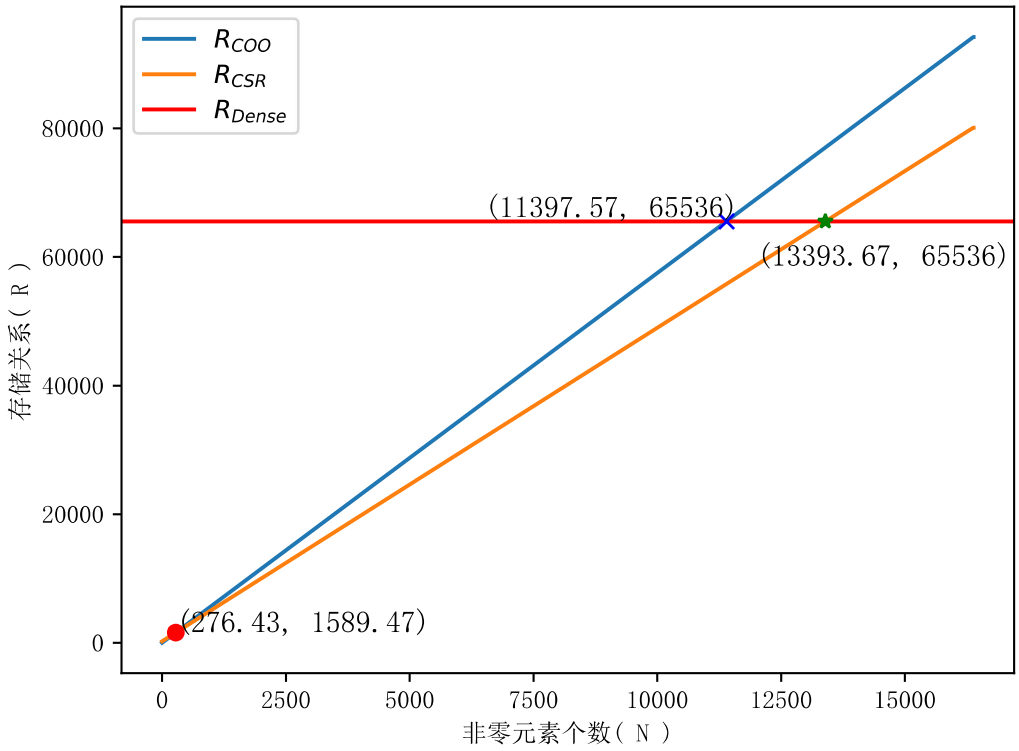


图 3-1 128×128 矩阵下 COO、CSR 和非压缩格式的存储空间与非零元素个数关系

时，稠密矩阵格式的存储效率高于 CSR 格式。因此，在上述场景中，对于稀疏度在 18.26%-98.32% 之间的稀疏矩阵，CSR 格式相较于另外两种存储格式能够更好地节省存储空间，其适用范围更大。

BCSR 格式存储稀疏矩阵分块后的矩阵，虽然牺牲了 CSR 格式对稀疏矩阵的极致压缩效果，但是换取了单次计算下更高的并行度。在现代科学与工程应用中，稀疏矩阵中的非零元素大多呈局部密集、块状分布[72]。因此本研究采用 CSR、CSC 格式的变形 BCSR 和 BCSC 格式来存储稀疏矩阵，保证矩阵加速器在处理稀疏度高、中等的低阶张量计算时能够获得较好的计算性能。

3.2 矩阵分块策略分析

将矩阵转换成分块矩阵，能够将高维矩阵转换成低维矩阵能够简化复杂计算。算法层面，分块矩阵允许动态调整块大小，适应不同计算需求。调度层面，分块矩阵可以将子矩阵分配到多个处理器或计算节点，提升并行计算效率。

矩阵分块策略的选择直接影响 BCSR 格式的存储效率与矩阵加速器的架构设计。处理稀疏矩阵相关计算时，采用分块策略导致的零值填充不可避免地使得矩阵加速器进行无效计算。佛罗里达大学截止至 2024 年 12 月公开的稀疏矩阵数据集[73]中，行、列维度在区间 [100, 500] 上的稀疏矩阵共 305 个。其中矩阵的

稀疏度最小为 21.51%，稀疏度最大为 99.79%，平均稀疏度为 96.37%，稀疏度在 98.32% 以上的稀疏矩阵共 122 个，占比 40%。统计它们在不同分块策略（2×2、4×4、8×8 和 16×16）下平均有效块占比、平均块内非零值占比，结果如表 3-1所示。

表 3-1　不同分块策略下，平均有效块占比和平均零值填充率占比统计

分块策略	平均有效块占比（%）	平均块内非零值占比（%）
2×2	9.27	43.07
4×4	19.03	20.68
8×8	32.70	11.44
16×16	48.51	7.35

平均有效块占比、平均块内非零值占比的高低反映分块策略的合理性。平均有效块占比越低，分块策略的存储开销也越小；平均块内非零值占比越低，表示该分块策略下子矩阵进行运算时的无效计算开销越大。显然分块的粒度越细，分块策略的无效计算次数越低。对于 8×8、16×16 分块策略，平均块内非零值占比太低使得无效计算次数过高；理论上，2×2 的分块策略最大化降低无效计算次数，但单次计算并行度过小；4×4 的分块策略虽然平均块内非零值占比相较于 2×2 下降了一半，但是单次计算性能在 2×2 分块策略的基础上提升 4 倍。所以，本研究采用 4×4 的分块策略，保证分块矩阵采用 BCSR 格式的存储效率，并且基于 4×4 规模的计算阵列能够获得可观的最大计算性能。

3.3　基于 BCSR 格式的矩阵、向量存储方案

从二进制角度进行观察，内存中的数据是一串 01 序列。在特定解码方式下，这串 01 序列被赋予意义。硬件电路对存储中数据的处理粒度是比特，而上位机对存储中的数据处理粒度是机器字。未被压缩的矩阵、向量在内存中，每个元素依次排列存储，对矩阵加速器与上位机的兼容性都很好。为采用压缩格式存储的矩阵、向量设计合适的存储方案，也要兼顾加速器电路设计和上位机软件设计，不仅能够减少软硬件开销，而且能够提高基于矩阵加速器的软件开发套件（Software Development Kit，SDK）的构建效率。因此，本研究采用将数据的最小存储单位对齐至字节的策略，在 BCSR 格式的基础上设计压缩矩阵、向量在存储中的存储方案。

基于 3.1 和 3.2 节的对存储格式、分块策略的分析结果，结合高级编程语言的特性，针对维度为 128×128 的矩阵、长度为 128 的向量，本文提出基于 BCSR

的矩阵、向量存储方案，统一了多模式矩阵加速器处理的矩阵、向量格式。

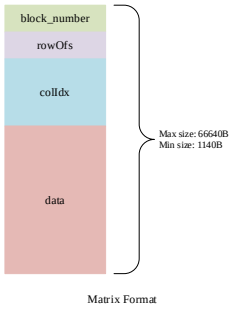
维度为 128×128 的矩阵按照大小为 4×4 的子矩阵进行分块后是 32×32 的分块矩阵，最多 1024 个子矩阵。按照 BCSR 格式进行存储时一个指针至少需要 11 bit 存储，一个索引至少需要 5 bit 存储。依据字节对齐策略，存储方案中使用 2 B 存储一个指针，1B 存储一个指针。长度为 128 的向量矩阵均匀切分成 32 个长度为 4 的子向量，使用 1 B 存储索引。为了兼容矩阵加速器对 BCSR 格式下矩阵的支持，矩阵、向量的压缩格式的第一行存储压缩后的矩阵、向量的有效块的个数。此时向量被当作行为 1 的矩阵存储，有利于多模式矩阵加速器的设计的同时，有利于矩阵加速器 SDK 的开发。

为了满足基于 4×4 分块策略的加速器的带宽需求，本研究选用行宽为 128 bit 的片内存储，每次能够取出 4 个位宽为 32 bit 的数据类型。以 32 bit 的数据类型为例，矩阵的存储方案如图3-2（a）所示，由有效块个数（block number）、指针区（ptr）、索引区（idx）以及数据区（data）组成。该存储方案中 16 个指针数据占 4 行的片内存储共 64 B；在结果矩阵的压缩格式生成过程中，无法计算得到有效块的个数，因此无法在计算完成前获得数据区的起始地址，需要存储最大有效块个数的索引，共 64 行 1024 B。向量的存储方案如图3-2（b）所示，索引区同样需要 32 B 来存储最大有效块个数的索引。无论是矩阵还是向量，对于有效块个数和数据区的策略均是一样。有效块的个数用 128 bit 存储，避免矩阵加速器加速器访存时的地址不对齐问题。对于上位机而言，浪费的 14 B 的片内存储后续可以用来记录矩阵的其它信息，比如数据类型，这取决于程序员的想象力。数据区大小和有效块个数正相关，大小为单个有效块的大小乘上有效块的个数，是该存储方案有效节约存储空间的核心。

相较于非压缩格式，存储全零矩阵最好情况下仅存储压缩格式能够节约 98.32%；相较于非压缩格式，存储稠密矩阵最坏情况下需要 66660 B，仅多浪费 1.68% 的存储空间。相较于非压缩格式，存储全零向量最好情况下仅存储压缩格式能够节约 90.63%；相较于非压缩格式，存储稠密矩阵最坏情况下需要 560 B，多花费 9.37% 的存储空间。分析位宽为 32 bit 的数据类型采用上述存储方案存储的最好、最坏情况可知，该存储方案的存储效率与存储对象的规模相关，规模越大存储效率越高，所以存储 128×128 这样的大规模矩阵代价远小于收益。虽然存储稠密向量时，该存储方案存在接近 10% 的存储空间浪费，但是对矩阵、向量存储方案的统一降低了后续架构设计过程中计算模块复杂度。

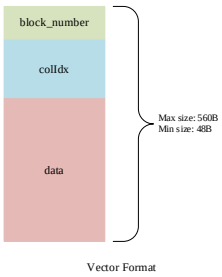
在先进芯片制程已经步入 3*nm*，且片上系统规模指数级增长的今天，处理器的片内存储通常大于 10 MB，因此上述存储方案并不激进。在后续的架构设计过程中，矩阵加速器内部每块存储区的大小定为 66 KB 能够存储一个压缩后的稠

名称	数量/数量区间	大小区间	位宽（bit）	行数	总大小（B）
block_number	1	[0, 1024]	128	1	16
rowOfs	32	[0, 1024]	16	4	64
colIdx	1024	[0, 31]	8	64	1024
data	[0, 16384]	[0, 2^{32}-1]	32	[0, 4096]	Max：65536
					Min：0
Total size					Max：66640
					Min：1104



（a）

名称	数量/数量区间	大小区间	位宽（bit）	行数	总大小（B）
block_number	1	[0, 32]	128	1	16
colIdx	32	[0, 31]	8	2	32
data	[0, 128]	[0, 2^{32}-1]	32	[0, 32]	Max：512
					Min：0
Total size					Max：560
					Min：48



（b）

图 3-2　基于 BCSR/BCSC 格式的矩阵、向量存储方案

密矩阵，原型验证过程采用三片这样的片内存储用于存储 2 个输入矩阵和 1 个结果矩阵。

3.4　矩阵乘法算法分析

计算低阶张量所采用的算法是矩阵加速器架构的一大重要因素，直接影响算子数目的考量、结果矩阵的生成、回写策略等。同时矩阵加速器的制造成本、应用场景等影响设计的面积、功耗等考量，进而算法的选择。本文的研究目标是设计面积小、可扩展性好和能效高的矩阵加速器。本节讨论矩阵 × 矩阵算法的选择，最后给出本研究采用矩阵乘法算法的伪代码。关于分块矩阵乘法算法的分析，本节分为两部分分析：一是将分块矩阵视为整体的分块矩阵乘法的考虑，二是子矩阵乘法的考虑。

3.4.1　分块矩阵乘法分析

从分块矩阵的视角看，内积算法只需 1 个子矩阵规模的计算阵列，就能正常工作，每次计算一个结果矩阵中的一个子矩阵。面向稀疏矩阵乘法，该算法下的性能提升可以通过设计复杂的匹配机制，评估制造成本部署更多的计算阵列来提升性能。唯一缺点是分块矩阵的输入复用性差。

外积算法的性能已经在工作[13]中体现。对于外积算法计算分块矩阵乘法需要考虑的两种极端情况：一是假如部署足够的算子，毫无疑问，能够获得极强的

计算性能，同时加速器将面临巨大的面积开销；二是假如部署的算子规模不够，矩阵加速器需要频繁地合并中间结果和回写操作，结果矩阵的压缩操作同样是不可避免的时间开销。

基于分而治之思想的 Strassen 算法能够减少矩阵乘法总次数，在架构设计层面应用能够有效降低大规模稠密矩阵计算的功耗[74]，但是将直接面对这样的窘境：一方面，如果将规模为 32×32 的分块矩阵按照 Strassen 算法展开，那么设计实现过程所需的乘、加算子将指数级增加导致芯片面积过大，这显然是不现实的。如果像递归实现的 Strassen 算法程序一样，循环调用某个模块，一次计算部分中间结果，直接的结果是矩阵乘法的计算时间增加。工作[17]中的计算通道采用了这种策略，增加了单个矩阵计算时间开销。

Gustavson 算法可以看作内积算法与外积算法的折衷，一次计算结果矩阵中一行。Gustavson 算法的一大优势是要求输入矩阵的压缩格式一致，不同于内积、外积算法，为了方便取数通常将其中一个输入矩阵转置。同时，Gustavson 算法相较于外积算法减少了大量的中间结果的数目，但同样需要对结果矩阵进行压缩。Gustavson 算法的输入复用性与内积算法一样差，但内积算法具有良好的输出复用性，能够逐个生成结果矩阵的压缩格式。

应用 Cannon 算法进行分块矩阵乘法需要对分块矩阵做更多的处理，包括设计新的稀疏矩阵、向量存储方案，分块矩阵的解析匹配机制等。Cannon 算法需要充足的算子来搭建脉动阵列，同时要求足够的带宽传输用于计算的子矩阵保证脉动阵列有源源不断地输入。

综上所述，外积算法、Strassen 算法和 Cannon 算法需要足够多的算子，才有最佳的计算性能，但是由于芯片面积的限制，并不适合本研究。考虑到 Gustavson 算法仍然存在较多中间结果需要进行压缩，由于输入左矩阵中的一个子矩阵需要乘上输入右矩阵的一行，无法充分利用片内存储带宽。内积算法因其良好的输入复用性，矩阵加速器内易于设计流水线架构，以及结果矩阵压缩格式的快速生成。未来可通过增加矩阵加速器内的计算阵列，快速提升该矩阵加速器的计算性能。因此选择内积算法作为分块矩阵乘法算法。

3.4.2 子矩阵乘法分析

在现代处理器架构中，例如 AMD Zen4 架构和英特尔第 10 代 Cannon Lake 架构，一条乘法指令执行时间平均是 3 个时钟周期，一条加法指令执行时间平均是 1 个时钟周期[75]。假设如下情况，分块矩阵的子矩阵的规模为 4×4，片内存储一次可以取回一个矩阵中的 4 个元素，记加法所需时间为 t，乘法所需的时间为 $3t$。在给定足够的算子的情况下，统计 5 种的算法完成子矩阵乘法所需的时

间，如表 3-2 。

表 3-2　5 种矩阵乘算法下，4×4 规模矩阵乘法统计

矩阵乘算法	加法器（个）	乘法器（个）	计算时间
内积	3	4	80*t*
外积	16	16	16*t*
Strassen	32	7	52*t*
Gustavson	4	4	64*t*
Cannon	16	16	28*t*

分析统计表可得，外积、Cannon 算法需要的乘法器最多 16 个，所需面积最大，计算效率最高。内积、Gustavson 算法在算子相同的情况下，Gustavson 算法所需的计算时间小于内积，究其原因，Gustavson 算法每次计算结果矩阵中的一行，在实际硬件实现时需要比内积算法的硬件实现需要更多的存储资源（如触发器）。限定的每次 4 个数据访存速度，实际上无法满足 2.1.3 小节中 2-1 至 2-5 的 Strassen 计算步骤的数据吞吐量。假设带宽足够的前提下，统计得到 Strassen 算法所需的时间为 52t。对于子矩阵乘法，选用计算性能最佳的外积算法，同时 16 个的乘累加器并不占用过多的面积。

3.4.3　基于 BCSR 格式的内积-外积的矩阵乘法

在 3.4.1 节和 3.4.2 节对分块矩阵乘法、子矩阵乘法的分析结果上，本节提出基于 BCSR 格式的内-外积矩阵乘法，作为矩阵加速器架构设计的基石。该算法流程3-4中调用了 4 个函数，分别是读索引函数（readIdx）、生成标志位数组函数（genBitArray）、匹配最近距离有效块函数（matchBlock）和外积算法函数（outerProduct）。算法流程中压缩后的输入矩阵 *A*，输入矩阵 *B* 和输入矩阵 *C* 均由指针数组 *ptr*、索引数组 *idx* 和子矩阵数组 *data* 组成，其中子矩阵数组中的每个有效元素代表 4×4 的子矩阵。其中指针数组的长度为 32，索引数组的长度为 1024，为了方便说明算法中将数据区的长度设置为 1024，实际的 SDK 实现时可以是变长数组。

算法3-1为读索引函数，其输入包含计数器（*cnt*）、当前指针 (*currentPtr*) 和索引数组（*idx*）。*cnt* 记录当前的步骤中匹配到第几个索引，*currentPtr* 是当前步骤中索引访问的上界。当 *cnt* 的值小于 *currentPtr* 的值时，从 *idx* 取出 *currentIdx*，随后 *cnt* 加 1；否则置 *currentIdx* 的值为-1，*cnt* 的值不变。输出 *currentIdx* 和 *cnt*。

算法 3-1 读索引函数

```
1: function READIDX(currentPtr, cnt, idx)
2:     if cnt ≤ currentPtr then
3:         currentIdx = idx[cnt]
4:         cnt = cnt + 1
5:     else
6:         currentIdx = -1
7:     end if
8:     return currentIdx, cnt
9: end function
```

算法3-2为生成标志位数组函数，其输入包含当前索引（*currentIdx*）和标志位数组（*bitArray*）。*bitArray* 的长度固定 32，记录一行或一列非零子矩阵的位置信息。当 *currentIdx* 不等于-1 时，将 *bitArray* 中的对应值设置成 1；否则不进行任何操作。该函数返回标记完成的 *bitArray*。

算法 3-2 生成标志位数组函数

```
1: function GENBITARRAY(currentIdx, bitArray)
2:     if currentIdx ≠ -1 then
3:         bitArray[currentIdx] = 1
4:     end if
5:     return bitArray
6: end function
```

算法3-3是匹配最近距离有效块函数，其输入包含输入矩阵 *B* 当前索引（*currentIdxB*）、匹配计数器（*cmpCnt*）、标志位数组 *A*（*bitArrayA*）、标志位数组 *B*（*bitArrayB*）、过往指针 *A*（*lastPtrA*）、过往指针 *B*（*lastPtrB*）、偏移 *A*（$ofsA$）和偏移 *B*（$ofsB$）。其中 *cmpCnt* 是 2 个标志位数组的匹配位置，*lastPtrA* 和 *lastPtrB* 表示输入矩阵 *A* 和输入矩阵 *B* 当前匹配行、列中有效块的起始位置，$ofsA$ 和 $ofsB$ 记录当前行、列中有效块的个数。

当 *cmpCnt* 的值小于 *currentIdxB* 时，将逐次匹配 2 个标志位数组中 *cmpCnt* 指向的标志位。在匹配的过程中 $ofsA$ 和 $ofsB$ 根据 2 个标志位数组中标志位的存在情况各自进行累加。函数的输出分为匹配成功与匹配不成功两种情况，匹配成功即两个标志位都存在的情况，返回 2 个有效块在各自的数据区中的位置（*blkAID* 和 *blkBID*）结束函数，由过往指针加上偏移得到；如果匹配失败，则将 *blkAID* 和 *blkBID* 赋值-1。*cmpCnt* 大于 *currentIdxB* 时，函数结束返回值

为-1 的 $blkAID$ 和 $blkBID$。该函数还返回 $ofsA$、$ofsB$ 和 $cmpCnt$ 的值，更新对应的值。

算法 3-3 匹配最近距离有效块函数

```
1: function MATCHBLOCK(currentIdxB, cmpCnt, bitArrayA, bitArrayB, lastPtrA,
   lastPtrB, ofsA, ofsB)
2:     while cmpCnt ≤ currentIdxB do
3:         if bitArrayA[cmpCnt] == 1 then
4:             ofsA = ofsA + 1;
5:         end if
6:         if bitArrayB[cmpCnt] == 1 then
7:             ofsB = ofsB + 1;
8:         end if
9:         if bitArrayA[cmpCnt] == 1 && bitArrayB[cmpCnt] == 1 then
10:            blkAID = lastPtrA + ofsA; blkBID = lastPtrB + ofsB;
11:            break;
12:        else
13:            blkAID = -1; blkBID = -1;
14:            cmpCnt = cmpCnt + 1;
15:        end if
16:    end while
17:    return blkAID, blkBID, ofsA, ofsB, cmpCnt
18: end function
```

2.1.2 节已经给出外积的数学公式，这里省略外积算法函数（outerProduct）的伪代码。其函数输入是 2 个 4×4 规模的子矩阵，输出为 2 个子矩阵乘法的结果。

基于 BCSR 格式的内积-外积的矩阵乘法的算法流程的主体框架由两层的循环逻辑构成，其本质是内积算法。该算法对于 BCSR 格式的解析，是将输入矩阵一行的有效块的位置信息映射成 Bitmap 格式，再进行内积算法下的规则匹配。

算法 3-4 基于 BCSR 格式的内积-外积的矩阵乘法

输入： ptrA[32], idxA[1024], dataA[1024], idxB[32], idxB[1024], dataB[1024]

输出： ptrC[32], idxC[1024], dataC[1024]

```
1: lastPtrA, currentPtrA, lastPtrB, currentPtrB= 0, 0, 0, 0;
2: currentIdxA, currentIdxB = 0, 0;
3: cntA, cntC = 0, 0;
4: for i = 0 to 31 do
5:     bitArrayA[32] = 0; cntB = 0;
6:     currentPtrA = ptrA[i]; lastPtrA = currentPtrA;
7:     repeat
8:         currentIdxA, cntA = READIDX(currentPtrA, cntA, idxA);
9:         bitArrayA = GENBITARRAY(currentIdxA, bitArrayA);
10:    until (currentIdxA ≠ -1)
11:    for j = 0 to 31 do
12:        bitArrayB[32] = 0; cmpCnt = 0
13:        currentPtrB= ptrB[j]; lastPtrB = currentPtrB;
14:        ofsA, ofsB, cmpCnt = 0, 0, 0
15:        repeat
16:            currentIdxB, cntB = READIDX(currentPtrB, cntB, idxB);
17:            bitArrayB = GENBITARRAY(currentIdxB, bitArrayB);
18:            blkAID, blkBID, ofsA, ofsB, cmpCnt = MATCHBLOCK(currentIdxB,
    cmpCnt, bitArrayA, bitArrayB, lastPtrA, lastPtrB, ofsA, ofsB)
19:            if blkAID ≠ -1 && blkAID ≠ -1 then
20:                dataC[cntC] = dataC[cntC] + OUTERPRODUCT(dataA[blkAID],
    dataB[blkBID]);
21:            end if
22:        until (currentIdxB ≠ -1)
23:        if dataC[cntC] is not all 0 then
24:            idxC[cntC] = j; cntC = cntC + 1;
25:        end if
26:    end for
27:    ptrA[i] = cntC;
28: end for
```

算法3-4中声明的内部变量计数器 *C*（*cntC*）用于记录结果矩阵中的非零子

矩阵的个数。*cntC* 指向的 *dataC* 数据区，存储当前正在计算的子矩阵。输入矩阵 *A* 的一行和输入矩阵 *B* 的一列相乘完成时，如果 *dataC*[*cntC*] 不是全零，则记录该子矩阵的索引 *j*。伪代码中保存结果矩阵中的子矩阵的方式十分简单，*cntC* 加一。在矩阵加速器中，需要将改操作回写至存储中。

基于内积-外积的矩阵乘法算法在后续架构设计时，通过添加部分额外的判断条件进而支持了列向量 × 行向量、矩阵 × 列向量和行向量 × 矩阵 3 种计算模式。通过部署额外的计算资源，达到对行向量 × 列向量、矩阵 + 矩阵、向量 + 向量计算模式的支持。

3.5 本章小结

通过本章的分析，本研究选择 BCSR 格式存储规模为 128×128 的方阵，按照子矩阵大小为 4×4 的分块策略，将其转换成规模为 32×32 的分块矩阵。为了兼容矩阵、向量混合运算流程，将长度为 128 的向量划分成 32 个长度为 4 的子向量，并提出矩阵、向量的实际存储方案降低加速器的架构设计复杂度。算法层面上，分块后的矩阵、向量之间的运算采用内积算法，降低运算结果的压缩格式的生成难度；子矩阵、子向量间的运算采用外积算法，提升局部的计算性能。

第四章　异步多模式矩阵加速器架构设计

在第三章的存储格式、分块策略分析结果和提出的算法基础上，本章采用粗粒度异步微流水线设计多模式矩阵加速器，支持 11 种计算模式，满足多种应用场景的实际需求。同时本章详细介绍多模式矩阵加速器中各个模块的微架构以及它们的工作流程。

4.1　系统架构

如图4-1所示，矩阵加速器包含 7 个核心模块，分别是控制单元（Control Unit, CU_M）、矩阵计算单元（Matrix Unit, MXU）、数乘单元（Scalar Multiplication Unit, SMU）、压缩单元（Compress Unit, COMPR）、解压单元（Decompress Unit, DECOMPR）、访存接口（Access Interface）和存储模块（Memory）。

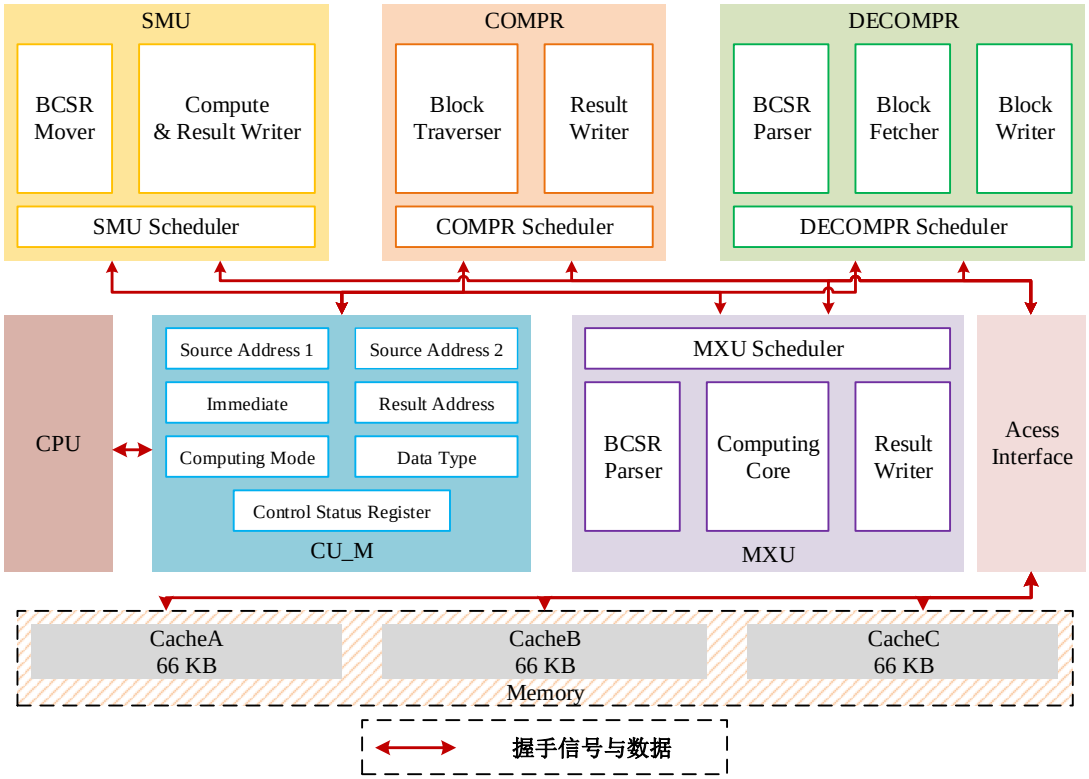


图 4-1　多模式矩阵加速器整体架构

下面简要描述各个核心模块的功能：

（1）控制单元：CPU 和矩阵加速器之间的“桥梁”，负责通信、交互。

（2）矩阵计算单元：最大的核心计算单元，通过灵活的匹配、调度机制支持 7 种计算操作。

（3）数乘单元：负责立即数乘向量和立即数乘矩阵两种模式的计算。

（4）压缩、解压单元：压缩、解压单元按照 3.3 节提出的基于 BCSR 格式的压缩方案，支持矩阵、向量在内的 4 种压缩、解压方式。压缩后的矩阵、向量能够作为矩阵计算单元的输入。解压单元为上位机提供高效的矩阵、向量解压操作。

（5）访存接口：矩阵计算单元、数乘单元、压缩单元和解压单元的访存通道，功能模块通过访存接口读写片内存储。

（6）存储模块：由 3 块 66 KB 的片内存储构成，用于存放输入矩阵和结果矩阵。

4.2 控制单元（CU_M）与矩阵计算微码

上位机通过读写控制单元，调度多模式矩阵加速器进行计算操作。控制单元模块内的核心是 7 个可配置寄存器：源地址寄存器 1（Source Address Register 1）、源地址寄存器 2（Source Address Register 2）、立即数寄存器（Immediate Register）、结果地址寄存器（Result Address Register）、计算模式寄存器（Computing Mode Register）、数据类型寄存器（Data Type Register）和控制状态寄存器（Control Status Register）。

4.2.1 矩阵计算微码

多模式矩阵加速器支持多种数据类型下常见的低价张量运算操作。由于多模式矩阵加速器在原型验证的过程中，仅部署整型计算单元（如乘法器、加法器），所以目前矩阵加速器支持 6 种位宽小于 32 bit 的整型数据的计算，其数据编码如表 4-1所示。矩阵加速器支持 11 种计算模式，计算模式对应的微码如表 4-2所示。

表 4-1 矩阵加速器支持的数据类型编码

数据类型	数据编码（4bit）
uint32	0001
int32	0010
uint16	0011
int16	0100
uint8	0101
int8	0110

表 4-2 矩阵加速器计算微码

计算模式	计算微码（4bit）
矩阵 × 矩阵 = 矩阵	0001
矩阵 + 矩阵 = 矩阵	0010
列向量 × 行向量 = 矩阵	0011
矩阵 × 列向量 = 列向量	0100
行向量 × 矩阵 = 行向量	0101
行向量 × 列向量 = 数	0110
向量 + 向量 = 向量	0111
数 × 向量 = 向量	1000
数 × 矩阵 = 矩阵	1001
矩阵/向量压缩	1010
矩阵/向量解压	1011

4.2.2 控制单元

控制单元的功能包含：写入配置寄存器；启动矩阵加速器的功能模块；通知上位机计算结果。

矩阵加速器工作流程为：上位机根据要进行的矩阵计算模式配置对应寄存器，配置的最后一步是将控制状态寄存器的最低位置 1，这是启动标志位。每次将控制状态寄存器的启动标志位置 1 后，控制单元均会判断配置是否正确。配置正确后，控制单元根据计算模式寄存器中的计算微码，启动对应的功能模块。以 INT32 型数乘矩阵计算为例，先后向立即数寄存器、源地址寄存器 1、计算模式寄存器、数据类型寄存器中分别写入立即数、矩阵起始地址、数乘矩阵计算微码（1001_2）、INT32 数据编码（0010_2），最后将控制状态寄存器中的启动标志位置 1。控制单元自检后，将相关寄存器与事件捆绑发送到数乘单元。矩阵加速器功能模块完成计算后，将相关的信息写入控制状态寄存器。最后，由控制单元通知上位机计算结果。

4.3 存储（Memeroy）与访存接口（Access Interface）

4.3.1 存储（Memeroy）

多模式矩阵加速器内部的存储中包含 CacheA、CacheB 和 CacheC 在内的 3 片 66 KB 的片内存储，能够满足数据位宽 32 bit 的低阶张量计算操作。存储模块支持多模式矩阵加速器和 CPU 的随机访问。每块片内存储均是单端静态随机存

储器（Static Random Access Memory，SRAM）。为了满足上述访存需求，对 Cache 进行如下封装，在每个 Cache 上添加“Merge2-Fifo1-Split2”控制链路，封装后的 3 个 Cache 组成了矩阵加速器的内部存储模块，如图4-2所示。同时，为了提高写操作效率，存储模块对多模式矩阵加速器与 CPU 的写操作不会返回请求事件。

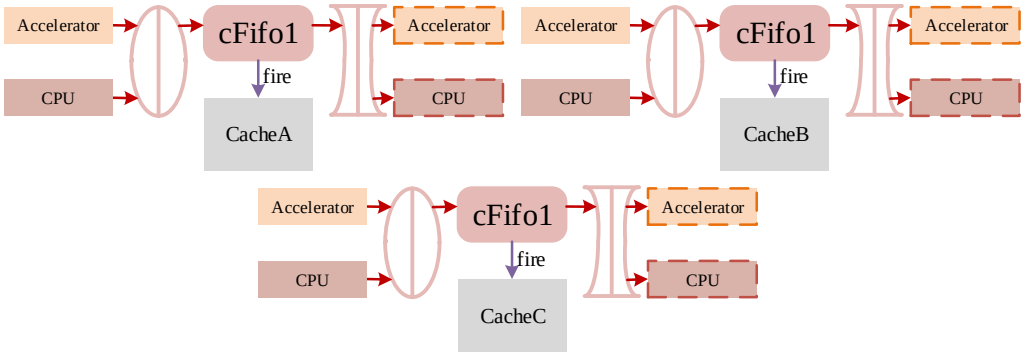


图 4-2　存储微架构

受限于 Cache 的片数，本研究中要求 CPU 只在矩阵加速器空闲的时候向矩阵加速器写入数据、或者读取计算结果。因此，多模式矩阵加速器与 CPU 不会同时访问存储中的 3 片 Cache，这里使用 2 路互斥结构（ConfMerge2）的隔离多模式矩阵加速器和 CPU 的访存通路。CPU 读取 CacheA 中的数据时，CacheA 根据地址读出数据后通过 2 路择路结构（SelSplit2）将数据和事件送到与 CPU 直连的总线上。当 CPU 往 CacheC 中写入数据时，SelSplit2 上的 $valid_0$ 和 $valid_1$ 均为 0，不会继续传递事件。CPU 需要在收到应答后，自驱进行下一操作。矩阵加速器向片内存储写入数据过程与 CPU 写操作过程相同。

4.3.2　访存接口（Access Interface）

矩阵加速器内的 COMPR、MXU 等 4 个计算模块，均存在访存操作，读写存储中的矩阵、向量。访存接口的功能是使得 4 个计算模块能够根据访存地址准确地访问存储模块中的 3 块 Cache。分析 4 个计算模块的读写需求后，为 COMPR、DECOMPR 和 SMU 三个模块各分配了 1 读 1 写 2 个通道，共计 6 个通道；为 MXU 分配了 2 读 1 写 3 个通道。访存接口包含直连 3 个 Cache 的 3 个通道。5 个读通道分别存在对应的用于接收读回数据的端口。存储模块将读出的数通过访存接口送回对应的模块。

访存接口内部的拓扑结构如图4-3所示，图中着重体现 SMU 读操作的访存链路。以 SMU 的读操作为例，SMU 将读地址（$addrSMU_R$）、对应的访存标志位（$flagSMU_R$）与事件捆绑向访存接口发送。3 路择路分支（SelSplit3）根据读地址决定访问的 Cache，这里假设访问 CacheA。于是，择路分支将事件与数据发

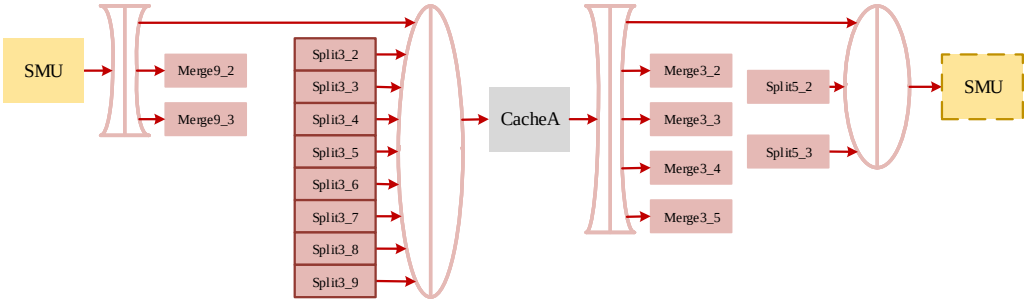


图 4-3　访存接口拓扑结构

送到和 CacheA 相连的 9 路互斥融合模块（ConfMerge9）上，进而访问 CacheA。从 CacheA 中读出数据后，SelSplit5 根据访存标志位决定事件与数据的流向，能够将事件、数据传输到 SMU 模块的 ConfMerge3 上。如果是 SMU 的写操作，往 CacheA 中写入数据后，存储模块根据访存标志位终止此次写操作。

4.4　三级流水矩阵计算模块（MXU）

三级流水矩阵计算模块是矩阵加速器中的核心计算模块，承担了矩阵加速器包含矩阵 × 矩阵、矩阵 × 列向量等在内的 7 种的计算模式。顾名思义，三级流水矩阵计算模块拥有基于粗粒度异步微流水线设计的三级流水微架构，其中，一级流水为 BCSR 解析器（BCSR Parser），二级流水为计算核心（Computing Core），三级流水为结果回写模块（Result Writer），上述 3 个模块通过 MXU 内部的调度模块（MXU Scheduler）访问矩阵加速器中访存接口访问存储中的数据，其架构如图4-4所示。

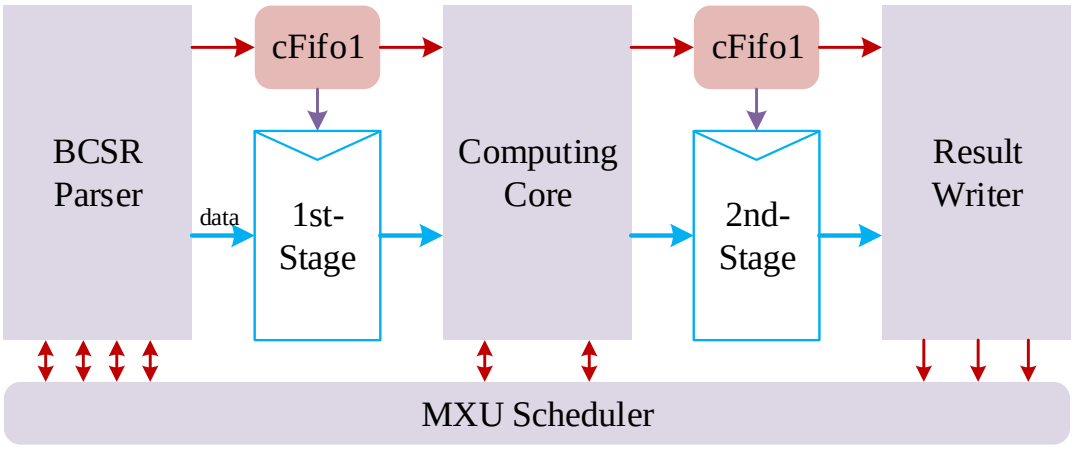


图 4-4　三级流水矩阵计算单元架构

4.4.1 第一级流水：BCSR 解析器（BCSR Parser）

工作[17]中的解码器在产生矩阵/向量一行的标志数组后，开始匹配需要计算的块。这种匹配机制在计算资源有限的情况下，不能够快速完成匹配块的计算，因此无法充分利用计算资源，计算效率较低。本文基于 3.3 节中设计的矩阵/向量的存储方案，设计了 BCSR 解析器。BCSR 解析器向计算核心发送事件和数据的同时，如果当前模式下的压缩格式解析未完成，那么 BCSR 解析器会进行下一轮解析操作。这样的工作机制，相较于工作[17]，可以有效提升 MXU 的吞吐率。BCSR 解析器的核心功能是将（1）匹配上的、需要计算的子块/子向量的索引发送到计算核心进行计算；（2）产生第二、三级需要的标志位信息。

BCSR 解析器向计算核心输出 8 个重要信息，分别是：

（1）子块 A 索引（*blkAID*），子块 B 索引（*blkBID*）：匹配的子块或子向量在存储空间中数据区的索引，计算核心根据索引计算子块或子向量的起始位置；

（2）子块 A 标志位（*blkAIDExist*），子块 B 标志位（*blkBIDExist*）：子块或子向量是否有效，是计算核心是否需要取数的判断条件之一；

（3）结果块列索引（*resultIdx*）：结果矩阵的子块的列索引，或是结果向量的子向量的行/列索引；

（4）子块完成标志位（*aBlockFinish*）：当前的子块/子向量计算是否完成的标志；

（5）行完成标志位（*rowFinish*）：当计算结果是矩阵且行完成标志为 1 时，结果矩阵的一行已经匹配完成；

（6）模式结束标志位（*modFinih*）：当前计算模式下所有的匹配完成。

指针取数器（Pointer Fetcher, PtrFetcher）、索引取数器（Index Fetcher, IdxFetcher）、标志位数组生成器（BitArray Generator）、块匹配器（Block Matcher, BlkMatcher）4 个功能模块组成压缩格式解析器。BCSR 解析器如同三级流水矩阵计算模块的“大脑”，控制、协调矩阵计算模块的工作。而在 BCSR 解析器中，块匹配器是控制中心，其它 3 个模块的功能相对简单，块匹配器控制着 BCSR 解析器的整体工作。该模块间的事件流如图4-5所示，CU_M 驱动 MXU 的本质是驱动指针取数器，索引取数器、指针取数器、标志位数组成生成器、块匹配器被顺序触发，块匹配器决定 BCSR 解析器如何进行后续计算流程的循环。

4.4.1.1 指针取数器（PtrFetcher）

指针取数器的 3 个主要功能是：（1）模块内部的偏移缓冲区为空时，从存储中取出下一组的指针；（2）根据计算信息给出正确的指针，当前行的起始指针（*startPtr*）和结束指针（*endPtr*）；（3）输出 BCSR 解析器正在解析的两个输入

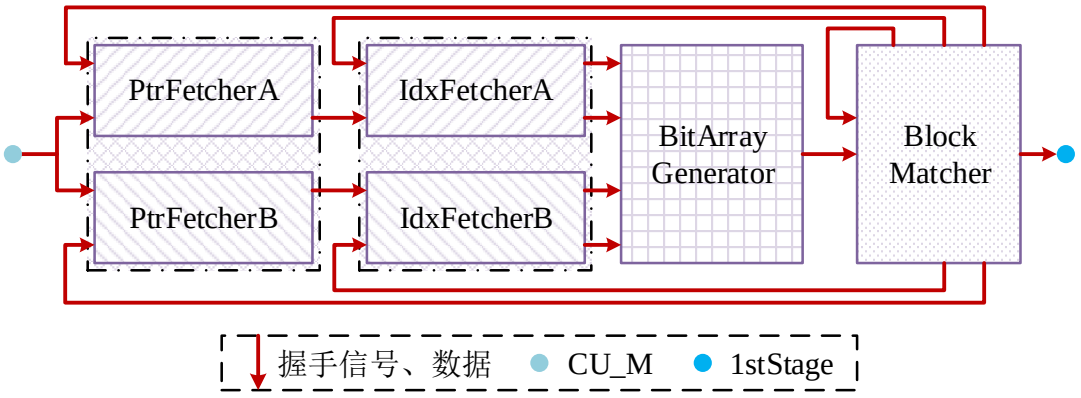


图 4-5 BCSR Parser 内部事件流图

的匹配行（*ptrCntA* 和 *ptrCntB*）。

指针取数器可分为 2 部分对称的指针取数逻辑（PtrFetcherA 和 PtrFetcherB），分别访问 2 个输入的压缩格式中的指针数据。指针取数逻辑主要由 3 个核心逻辑电路构成，分别是指针地址生成（GenPtrAddr）、指针缓冲区（PtrBuffer）和指针选择（PtrChooser），图4-6是 PtrFetcherA 的微架构。当指针缓冲区为空时，GenPtrAddr 根据计算模式产生下一指针的所在地址。在 3.3 节中的存储方案中，存储中的一行规则地存放了 8 个指针，从存储中取回后存放在指针缓冲区。PtrChooser 根据块匹配器提供的相关标志位决定是否更新 *startPtr* 和 *endPtr*，同时输出缓冲区是否为空的标志位（*ptrBufferEmpty*）。当输入为向量时，PtrChooser 只会更新一次，然后保持不变。

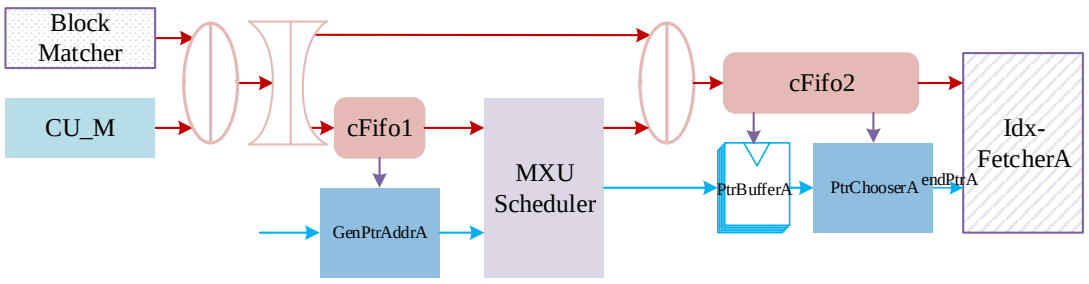


图 4-6 PtrFetcherA 微架构

4.4.1.2 索引取数器（IdxFetcher）

索引取数器的 2 个主要功能是：（1）模块内部的索引缓冲区为空时，从存储中取出下一组的索引；（2）根据计算信息给出当前索引（*currentIdx*）。

索引取数器分为 2 部分对称的索引取数逻辑（IdxFetcherA 和 IdxFetcherB），分别访问 2 个输入的压缩格式中的索引数据。索引取数逻辑主要由 2 个核心逻辑电路索引地址生成（GenIdxAddr）和索引选择（IdxChooser）构成，内部包含索引

缓冲区（IdxBuffer）、块计数器（blkCounter），微架构如图4-7所示。当索引缓冲区为空时，GenIdxAddr 根据计算模式产生下一索引的所在地址。在 3.3 节中的存储方案中，存储中的一行最多能够存放 16 个索引，从存储中取回后存放在索引缓冲区。以 IdxFetcherB 为例，当 $blkCounterB$ 小于 $endPtrB$ 时，IdxChooserB 从索引缓冲区输出当前子块或子向量对应的索引，累加块计数器的同时输出 $IdxBufferB$ 是否为空的标志位（$idxBufferEmpty$）；反之表示当前的列的子块已经遍历结束，IdxFetcherB 只会向标志位数组生成器发送驱动事件。

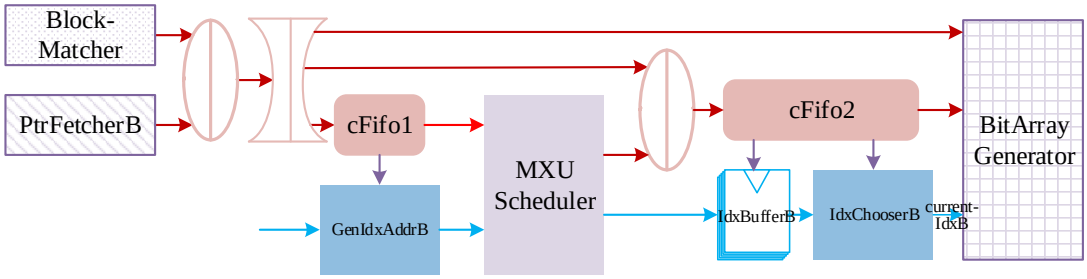


图 4-7　IdxFetcherB 微架构

4.4.1.3 标志位数组生成器（BitArray Generator）

标志位数组生成器的 2 个主要功能是：（1）根据索引取数器的 $currentIdx$，生成分块后矩阵或向量的标志位数组，用于还原子块或子向量在原矩阵中的位置。（2）计算当前最小匹配索引（$minIdx$）。

标志位数组生成器分为 2 部分标志位数据生成逻辑（BitArrayA Generator 和 BitMapB Generator），根据 IdxFetcher 给出的子块或子向量的索引生成标志位数组，其微架构见图4-8。因为 MXU 采用内积算法计算分块后的矩阵、向量，以矩阵 × 矩阵为例，运算符右侧的矩阵会被重复解析，所以在 BitArray Generator 内放置了一个规模为 32×32 的二维标志位数组（$bitMapB$）。故对于运算符号右侧的分块矩阵和分块向量，IdxFetcher 遍历、解析一次即可。一个长度为 32 的标志位数组（$bitArrayA$），就足够记录运算符左侧的输入中有效块的位置信息。BitArray Generator 每次生成标志位后，输出 $bitArrayA$ 和 $bitMapB$ 中由 $ptrCntB$ 决定的某行。

模块内通过最小索引判断逻辑（SetMinIdx），比较得到块匹配器能够向前匹配的最远位置。以矩阵 A× 矩阵 B 为例，计算开始时，$bitArrayA$ 和 $bitMapB$ 内全是 0。第一次若是从 IdxFetcher 得到 $currentIdxA$ 和 $currentIdxB$ 分别为 4 和 2，说明在块匹配器中可以匹配到的最远位置是 2，因为无法确认矩阵 B 的当前列是否存在索引为 3 或 4 的有效块。当矩阵 A 中的当前行的 $bitArrayA$ 已经产

生，正在解析矩阵 B 的某列，会将 $minIdx$ 设置成 $currentIdxB$。

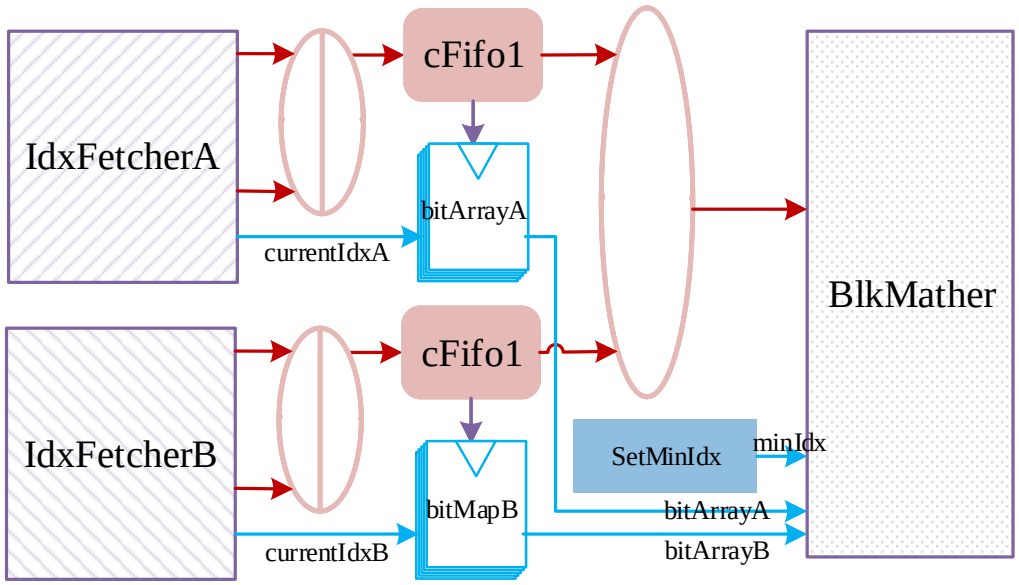


图 4-8　BitArray Generator 微架构图

4.4.1.4 块匹配器（BlkMatcher）

块匹配器的主要功能是：（1）计算产生 BCSR 解析器向第二、三级流水 8 个重要信息；（2）根据当前计算信息，驱动 BCSR 解析器中相应的模块进行下一次的匹配。块匹配器原理图见4-9，内部的块匹配逻辑（Block Match）、子块索引计算逻辑（GetBlkID）、解析器控制逻辑（Parser Control），协同完成块匹配器的复杂功能。

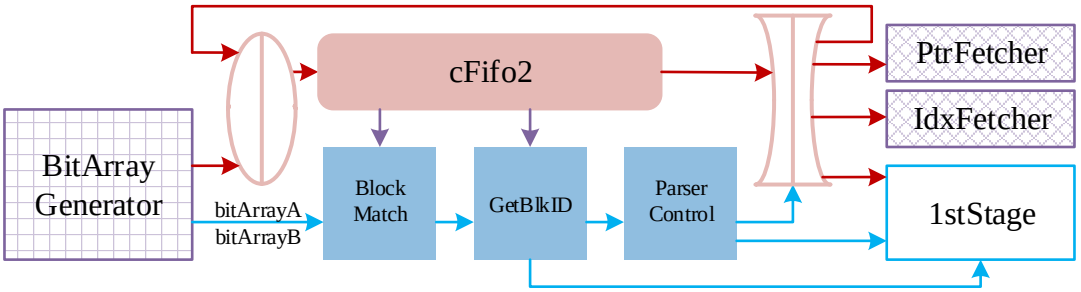


图 4-9　BlkMatcher 微架构图

块匹配逻辑内部依靠当前的匹配位置———匹配计数器（$matchCounter$）首先产生 2 个子块标志位，然后根据模式特征判断是否匹配成功。加法模式下，两个子块至少存在 1 个表示成功匹配；乘法模式下，两个子块都存在表示成功匹配。模式列向量 × 行向量属于特殊情况，因为列向量中的每个子向量需要逐个和

行向量中的所有子向量进行外积乘法，所以需要使用 *ptrCntA* 确定当前匹配位置，而不是 *matchCounter*。

在 *bitArrayA* 已经被计算得到的情况下，子块索引值无法直接得到，因此子块索引计算逻辑被设计来计算 2 个子块索引的实际值。其中的计算思路是指针取数器的 *startPtr* 加上非零子块在当前行中的偏移 $blkOfs$，$blkOfs$ 表示当前子块是当前行中的第 $blkOfs$ 个非零子块，$blkOfs$ 在块匹配逻辑工作后累加。

解析器控制逻辑通过当前计算模式、*matchCounter*、*ptrCntA* 和 *ptrCntB* 的值，产生换行标志位（*RowBreakFlag*）、换列标志位（*ColBreakFlag*）、*aBlockFinish*、*rowFinish* 和 *modFinih* 等值。其中 *RowBreakFlag* 影响 PtrFetcherA 是否更新 *startPtrA* 和 *endPtrA*，*ColBreakFlag* 影响 PtrFetcherB 是否更新 *startPtrA* 和 *endPtrA*。*matchCounter* 等于 32 时，依据计算模式赋值相关标志位信息。

4.4.2 第二级流水：计算核心（Computing Core）

计算核心的主要功能是子块与子向量的计算。BCSR 解析器按计算流程输出正确的子块索引后，计算核心根据模式微码选择对应算法流程进行取数，然后把数据分配到 PE 阵列（PE Array）中部分或全部的计算单元进行对应的计算，最后在结果融合模块（Result Merger）中将最终结果送到第三级流水。计算核心由数据取数器（Data Fetcher）、数据分配器（Data Allocator）、PE 阵列和的结果融合模块构成，几个模块在计算核心中顺序触发。

4.4.2.1 数据取数器（Data Fetcher）

数据取数器的内主要包含两个取数逻辑（DataFetcherA 和 DataFetcherB）和两个数据缓存区（*dataBufferA* 和 *dataBufferB*），功能是根据子块 A 索引和子块 B 索引计算得到子块/子向量的地址，通过访存接口取出子块或者子向量的数据，分别暂存在数据缓存区中。同时计算出下次的取数地址，接着驱动访存接口与数据分配器。图4-10展示了 DataFetcher 的微架构，主要展示了 DataFetcherA 的异步微流水线逻辑，DataFetcherB 仅用虚线框进行表示。

（1）矩阵 × 矩阵：对于匹配的子块 A_i, B_j，两个取数逻辑每次取回子块 A_i 中的一列和子块 B_i 中的一行，共需进行 4 次的取数。

（2）矩阵 + 矩阵：若两个子块标志位均为 1，说明两个子块需要进行加法。若是只有一个子块标志位为 1，说明子块标志位为 1 的子块即为结果矩阵的一个子块，将其存入结果矩阵即可。

（3）列向量 × 行向量、行向量 × 列向量：对于匹配子向量 a_i, b_j，两个取数

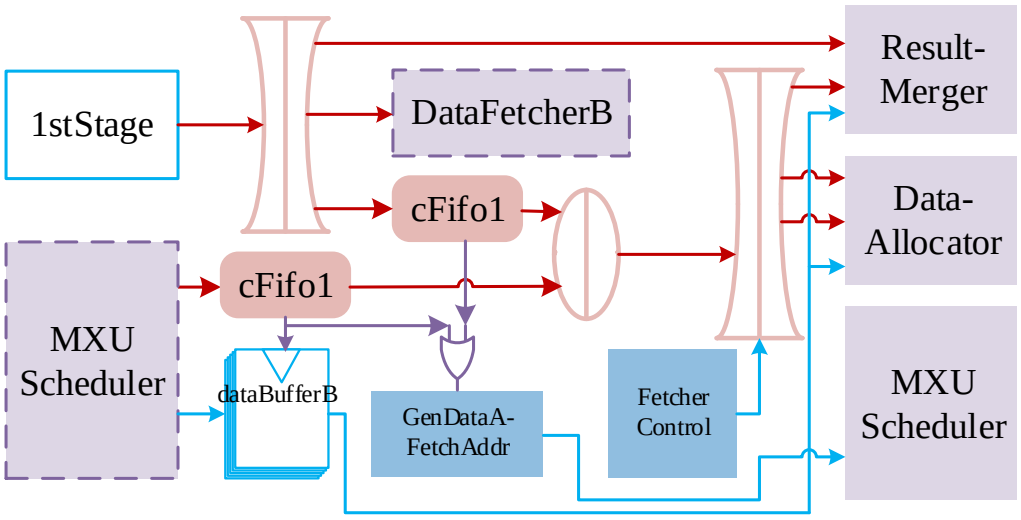


图 4-10 DataFetcherA 微架构图

模块只需进行 1 次的访存。

（4）矩阵 × 列向量、行向量 × 矩阵：两个模式下，当前负责取子向量的取数逻辑只工作 1 次，负责取子块的取数逻辑一次工作 4 次。

当两个子块标志都是 0 时，数据预取器的取数逻辑不工作，直接通过旁路将事件传送到结果融合模块进行下一步处理。

4.4.2.2 数据分配器（Data Allocator）与 PE 阵列（PE Array）

数据分配器的功能与 PE 阵列中计算单元的划分、数据取数器的输出密切相关。

16 个乘累加单元（MAC）和 4 个加法器（Adder）组成的 PE 阵列，承担 7 种计算模式下几乎所有的计算。由于没有一种计算需要同时调度所有的计算单元，具体到每种模式的每次计算，矩阵 × 矩阵、列向量 × 行向量调用 16 个乘累加单元，矩阵 × 列向量、行向量 × 矩阵和行向量 × 列向量调用 4 个乘累加单元，矩阵 + 矩阵、向量 + 向量调用 4 个加法单元，共 3 种调度情况。因此，20 个 PE 按功能类型、每组四个划分 5 个计算道（Lane1, ... ,Lane5），如图4-11(c) 所示，其中 16 个乘累加单元被划分为 Lane1-Lane4，Lane1-Lane4 共同完成子块 × 子块的计算任务，Lane1 还负责完成行向量 × 列向量的计算任务，Lane2 还负责完成矩阵 × 列向量、和行向量 × 矩阵的计算任务，Lane5 负责矩阵 + 矩阵、向量＋向量的计算任务。因为乘累加单元的延时大于加法器延时，所以 Lane1-Lane4、Lane5 由两个延时不同的 Fifo 控制，这样使得加法、乘法模式均可获得最佳计算性能，同时保证二者不会并发降低了功耗。在不同的乘法模式下，会产生对应的标志位使能相应的计算道，例如矩阵 × 列向量模式下只有 Lane2 会工作，矩阵 × 矩阵

模式下 Lane1-Lane4 同时并发计算。

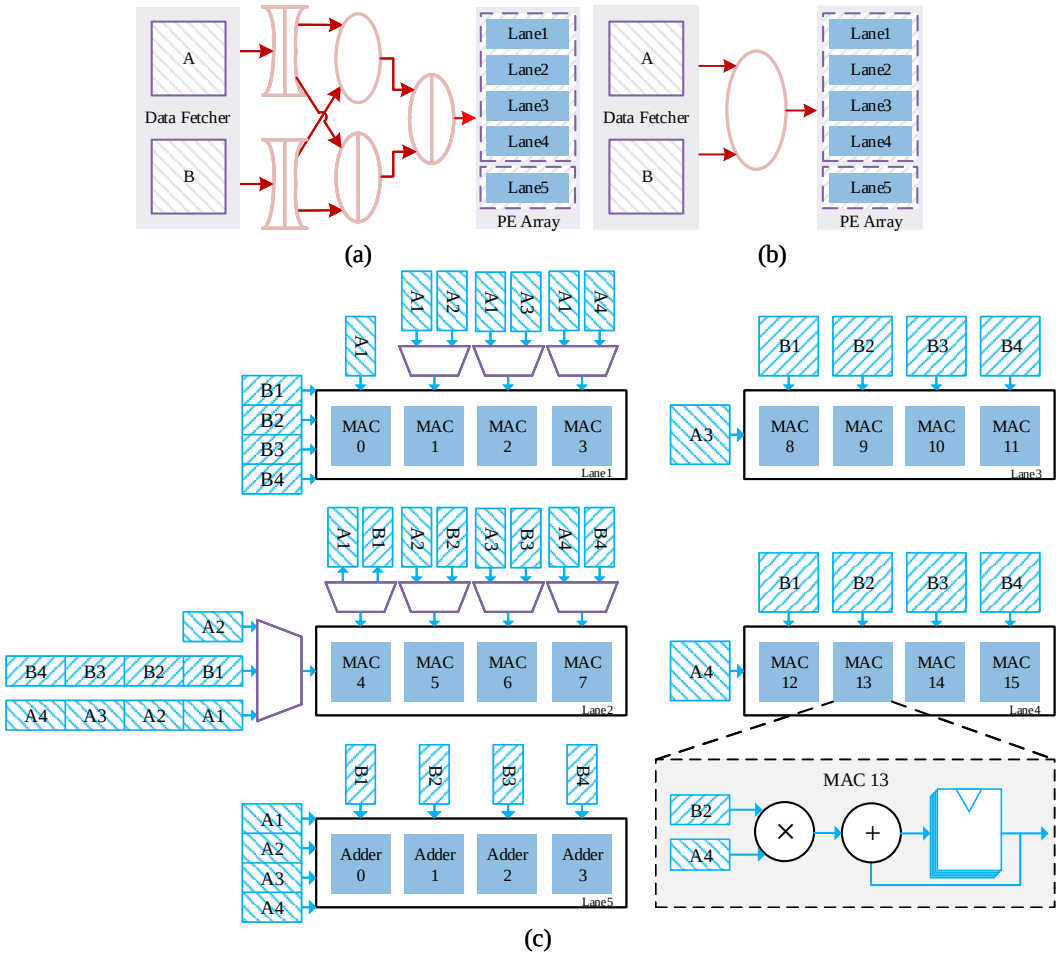


图 4-11　Data Allocator、PE Array 微架构

数据预取器中的两个取数逻辑根据矩阵计算的模式类型产生给 Lane1-Lane4 或 Lane5 的两个事件之一，共有 4 个可能的事件（*driveToMulLaneA*，*driveToMulLaneB*，*driveToAddLaneA*，*driveToAddLaneB*）需要数据分配器管理，图4-11(a) 为 *driveToMulLaneA*、*driveToMulLaneB* 的事件逻辑，图4-11(b) 为 *driveToAddLaneA*、*driveToAddLaneB* 的事件逻辑。数据分配器的工作是（1）将数据取数器中缓存区的数分配到 PE 阵列上进行计算，其本质是选择电路逻辑；（2）管理数据预取器发来的事件，正确地分离、融合事件以驱动 PE 阵列。

假设数据取数器中的缓存下的数据为子向量（A1，A2，A3，A4）、子向量（B1，B2，B3，B4）。下面介绍不同模式下，数据分配器的事件流、数据分发策略以及 PE 阵列的计算流程：

（1）矩阵 × 矩阵：数据分配器等待 *driveToMulLaneA* 和 *driveToMulLaneB* 都到来后，融合成一个事件驱动 PE 阵列中的 Lane1-Lane4。对于子块 × 子块，数据分配器将 A1 作为 3 个 2 输入选择器（Mux2）的输出接入 Lane1，同时 A2 通

过 3 输入选择器（Mux3）、B1-B4 通过 4 个 Mux2 的输出到 Lane2 上，使得 PE 阵列每次计算一对列乘行得到一个中间结果，四次的计算过后得到其结果。当 *aBlockFinish* 为 0 时，PE 阵列会不断累加结果并缓存；当 *aBlockFinish* 为 1 时，PE 阵列会将当前的计算结果发送到结果融合模块。第二级级间寄存器缓存下该结果后，将 PE 阵列中的寄存器复位。

（2）矩阵 + 矩阵、向量 + 向量：数据分配器等待 *driveToAddLaneA* 和 *driveToAddLaneB* 都到来后，融合成一个事件驱动 PE 阵列中的 Lane5。对于子块 + 子块，数据分配器将数据发送到 Lane5 中进行计算，每次计算得到子矩阵中的一个行向量，并将结果发送到结果融合模块。

（3）列向量 × 行向量：对于两个子向量的外积乘，数据分配器中的事件流、数据的分发方式和 PE 阵列中的复位时机与矩阵 × 矩阵模式相同。唯一的区别是 PE 阵列一次的计算就能产生结果矩阵中的一个子块。

（4）矩阵 × 列向量：对于子块 × 子向量，Lane2 每次接收数据分配器通过 4 个 Mux2 输出的子块中的一列 (A1,A2,A3,A4) 以及通过 Mux3 输出子向量中的一个数，四次计算后完成。第一次计算时，数据分配器等待 *driveToMulLaneA* 和 *driveToMulLaneB* 都到来后，融合成一个事件驱动 PE 阵列中的 Lane2；后三次计算时，数据分配器等待 *driveToMulLaneA* 到来后，即刻驱动 Lane2。当 *aBlockFinish* 为 0 时，Lane2 会不断累加中间结果；当 *aBlockFinish* 为 1 时，Lane2 将当前的计算结果发送到结果融合模块。第二级级间寄存器缓存下该结果后，将 PE 阵列中的寄存器复位。

（5）行向量 × 矩阵：对于子向量 × 子块，Lane2 每次接收数据分配器通过 4 个 Mux2 输出的子块中的一行 (B1,B2,B3,B4) 以及通过 Mux3 输出子向量中的一个数，四次计算后完成。第一次计算时，数据分配器等待 *driveToMulLaneA* 和 *driveToMulLaneB* 都到来后，融合成一个事件驱动 PE 阵列中的 Lane2；后三次计算时，数据分配器等待 *driveToMulLaneB* 到来后，即刻驱动 Lane2。结果向量中子向量计算完成的条件、寄存器的复位时刻与模式矩阵 × 列向量相同。

（6）行向量 × 列向量：数据分配器等待 *driveToMulLaneA* 和 *driveToMulLaneB* 都到来后，融合成一个事件驱动 PE 阵列中的 Lane1。两个子向量的内积乘法，数据分配器将（A2，A3，A4）作为 3 个 2 输入选择器（Mux2）的输出接入 Lane1 进行计算，每次 Lane1 会计算、累加得到一个长度为 4 的子向量。当 *modFinih* 为 1 时，PE 阵列将该子向量发送到结果融合模块进行最后的累加合并。

4.4.2.3 结果融合模块（Result Merger）

结果融合模块的微架构如图4-12所示。结果融合模块进一步处理数据取数器和 PE 阵列给出的事件与数据，然后将事件与结果传输到第二级间寄存器组。结果融合模块的组合逻辑包括加法模式下的暂存结果逻辑（Hold Result）、4 路加法树（Adder Tree）。如果 BCSR 解析器传来的两个块标志位均无效，事件和数据将从结果融合模块中直接传递到第二级间寄存器。

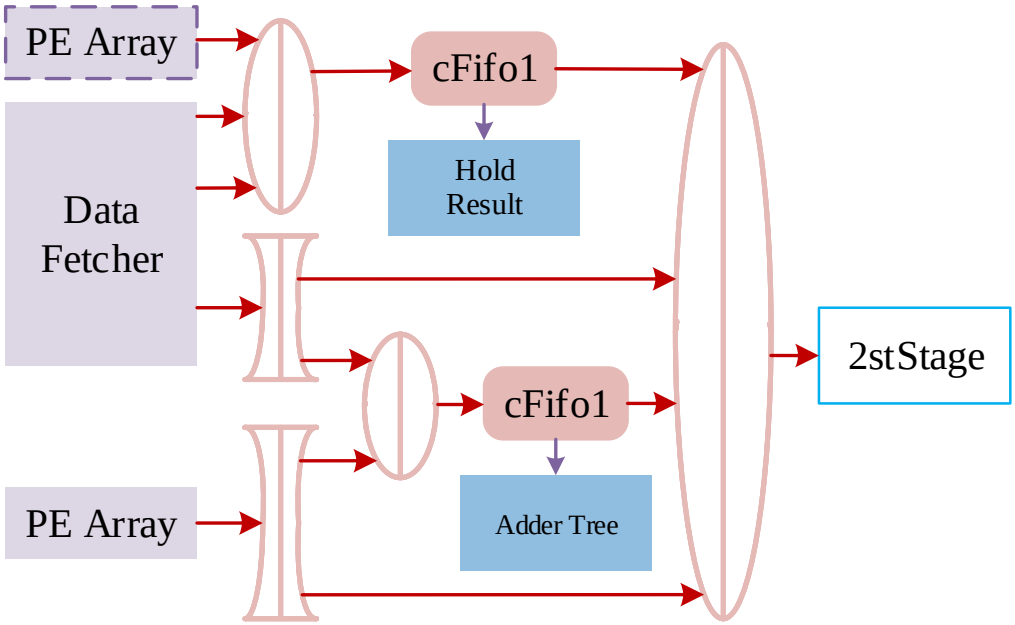


图 4-12　Result Merger 微架构

加法模式下，结果矩阵的子块可能直接是数据取数器的访问的子块，可能是 PE 阵列计算的结果。通过标志位 *flagAddResult* 记录事件从哪个模块传来，*flagAddResult* 作为 3 路选择器选择信号，正确输出加法的结果。每次结果融合模块会收到 4 个数据，矩阵 + 矩阵模式下结果融合模块接连四次将结果写入暂存结果逻辑，向量 + 向量模式下只要一次即可。

乘法模式下，结果矩阵的子矩阵、结果向量的子向量一定来自 PE 阵列，但是事件可能来自 PE 阵列或者数据取数器中。造成这种可能性的原因很简单，是因为 BCSR 解析器匹配得到当前模式下的最后一对索引无需计算。此外，如果当前模式是行向量 × 列向量，计算结果为标量，需要将 Lane1 中的 4 个数通过 4 路加法树进行累加合并。

4.4.3　第三级流水：结果回写模块（Result Writer）

如图4-13所示，零值检测器（Zero Detector）、缓冲区管理者（Buffer Manager）和回写管理者（Writer Manager）三大模块构成了结果回写模块。结果回写模块

收到计算核心的事件后，先检测计算核心存放在第二级间寄存器的结果是否有效，随后根据计算结果的类型在缓冲区管理者中生成对应的压缩格式，最后回写管理者回写结果的数据、压缩格式。

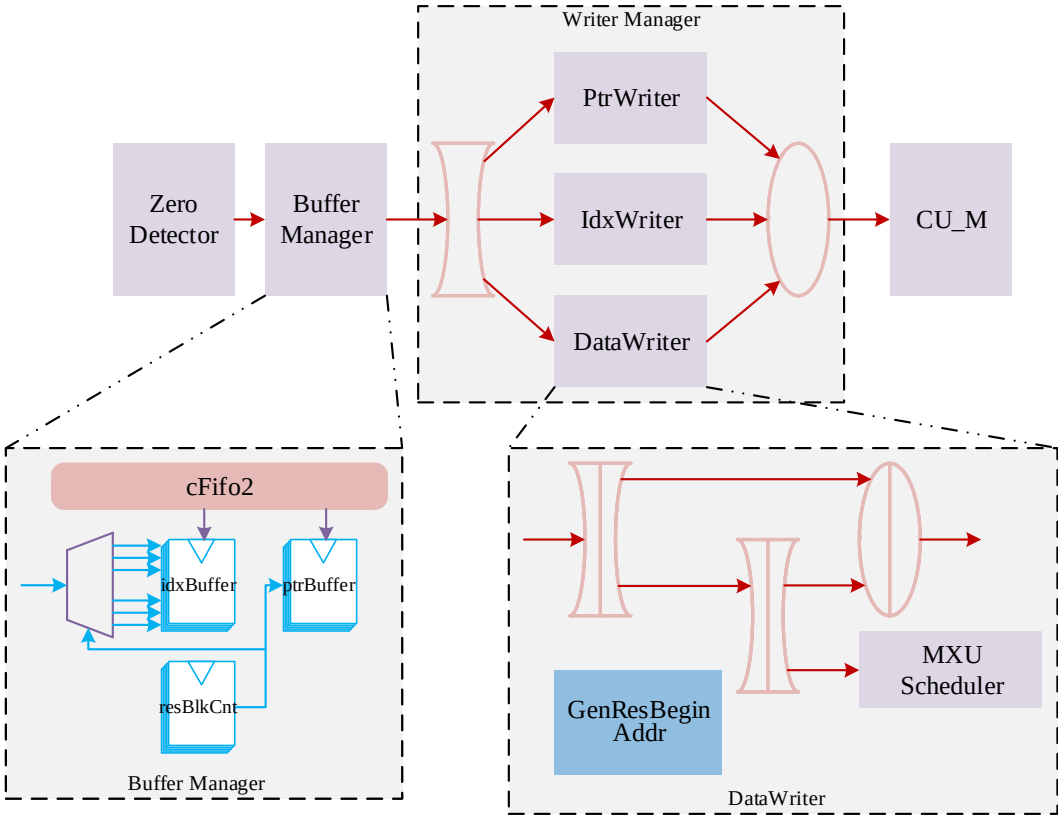


图 4-13　结果回写模块微架构图

4.4.3.1 零值检测器（Zero Detector）

零值检测器内部逻辑简单，由判断数据是否为零的逻辑构成，能够同时判断 16 个数据是否全为零。如果从计算核心传来的当前模式下的计算结果不全为零，那么标志位 *isResultValid* 将置为 1；否则，置标志位 *isResultValid* 为 0。

4.4.3.2 缓冲区管理者（Buffer Manager）

缓冲区管理者内部包含索引缓冲区（$idxBuffer$）和指针缓冲区（$ptrBuffer$）。根据 3.3 节中设计的存储方案，索引缓冲区和指针缓冲区的大小均为 128 bit，分别可以缓存 8 个指针和 16 个索引。

索引缓冲区内部有一个结果块计数器（$resBlkCnt$），用于统计计算结果中非零子块的个数，内部的标志位 *idxFull* 在索引缓冲区满时置 1。当计算结果为矩阵或向量，且满足标志位 *isResultValid* 和标志位 *aBlockFinish* 同时为真的条件时，将 BCSR Parser 匹配得到 *resultIdx* 写入索引缓冲区，同时累加结果块

计数器。对不同的计算模式，当结果是矩阵时，索引缓冲区的写入次数范围为 0 ~ 1024；结果是向量时，写入次数的范围为 0 ~ 32。

当计算结果为矩阵或向量，且满足条件 *rowFinish* 为真时，将当前 *resBlkCnt* 的值写入指针缓冲区。对于矩阵，指针缓冲区的写入次数为 16 次；对于向量写入次数为 1 次。当计算结果为数时，索引缓冲区与指针缓冲区均不会被触发。指针缓冲区内部的标志位 *ptrFull* 在指针缓冲区满时置 1。

4.4.3.3 回写管理者（Writer Manager）

回写管理者由回写起始地址生成（GenResBeginAddr）、指针回写模块（PtrWriter）、索引回写模块（IdxWriter）和数据回写模块（DataWriter）构成。MXU 支持的所有计算模式下，回写管理者会同时触发三种回写模块。当三个回写模块据完成各自的回写操作后，将会告知回写管理者。如果某个回写模块无需进行回写操作，那么该回写模块只会传递事件。当回写管理者收到 3 个回写模块的回写完成的事件后，通知第二级级间寄存器。

回写起始生成逻辑通过控制单元模块中的结果地址寄存器的值加上当前结果类型中的偏移计算得到压缩格式中指针起始地址、索引起始地址和数据起始地址。

（1）计算结果为矩阵时，指针回写模块每次在 *ptrFull* 为 1 时，回写指针缓冲区的数据。回写 4 次后，意味结果矩阵计算完成，此时将回写压缩格式中的非零块个数。索引回写模块在 *idxFull* 为 1 时，回写一次索引缓冲区的数据。回写次数与结果矩阵中子块的个数成正相关关系。若是计算完成时，索引缓冲区中非空，则需要回写当前的索引缓冲区。数据回写模块在当前子块存在非零元素时工作，子块将通过若干次的回写操作回写到存储中去，位宽为 32 bit 的数据类型需要回写 4 次。

（2）计算结果为向量时，指针回写模块只在计算完成的同时回写指针缓冲区，此时索引缓冲区只有 1 个值，该值记录了非零子向量的个数。索引回写模块在 *idxFull* 为 1 时，回写一次索引缓冲区的数据。回写次数与结果向量中子向量的个数成正比。与结算结果为矩阵的情况相同，标志位 *modFinish* 为 1 时，只要索引缓冲区非空，则需要回写当前的索引缓冲区。数据回写模块在当前子向量存在非零元素时工作，子向量将通过若干次的回写操作回写到存储中去，位宽为 32 bit 的数据类型需要回写 1 次。

（3）计算结果为向量时，指针回写模块和索引回写模块不工作。数据回写模块将最终结果写到结果地址寄存器指向的地址。

MXU 完成计算任务后，回写管理者将会把相关信息回写到控制单元中控制

状态寄存器中。

4.5 数乘模块（SMU）

数乘单元由 BCSR 搬移模块（BCSR Mover）、计算与结果回写模块 (Compute & Result Writer) 和其内部的调度模块（SMU Scheduler）组成，负责立即数 × 向量和立即数 × 矩阵两种模式的计算。BCSR 搬移模块、计算与回写模块通过调度模块接入访存接口访问片内存储中的数据，其架构与内部主要逻辑如图4-14所示。

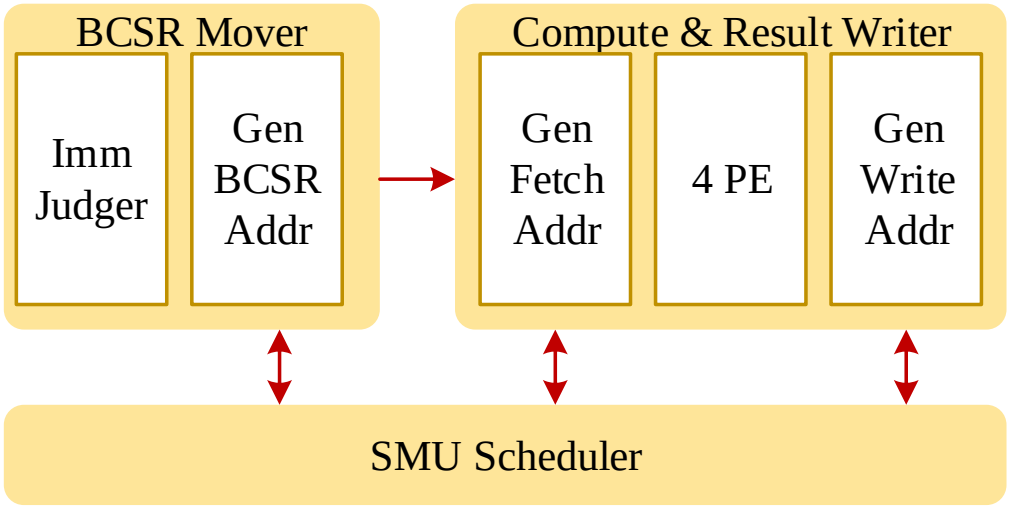


图 4-14 数乘模块架构

4.5.1 BCSR 搬移模块（BCSR Mover）

当向量和矩阵的稀疏度高时，两个计算模式只要完成立即数乘以向量、矩阵中的非零值即可。因此，当立即数不等于零时，对于基于 BCSR 格式的向量、矩阵，数乘计算后结果矩阵的压缩格式不发生改变。BCSR 搬移模块负责搬运本次计算结果的索引信息并记录有效块个数，图4-15是其微架构。

BSCR 搬移模块首先判断立即数是否为等于零，如果立即数等于零，则数乘计算的结果为零向量或全零矩阵，直接向矩阵控制单元中的 CSR 寄存器写入完成标志位；如果立即数不等于零，则驱动索引格式地址生成（GenBCSRAddr）逻辑。BCSR 搬移模块在进行第一次搬移时记录有效块个数（*blkNumber*），作为计算和结果回写模块的输入。BCSR 搬移模块根据计算模式，进行固定次数的搬移操作，将结果索引写入至指定地址。内部的寄存器组 *indexA* 和 *indexC* 用于暂存取回的索引和回写的索引。索引搬移的过程没有任何计算，BCSR 搬移模块将从调度模块读回的数据与计算得到的地址捆绑后，立即通过调度模块进行回写操

作。

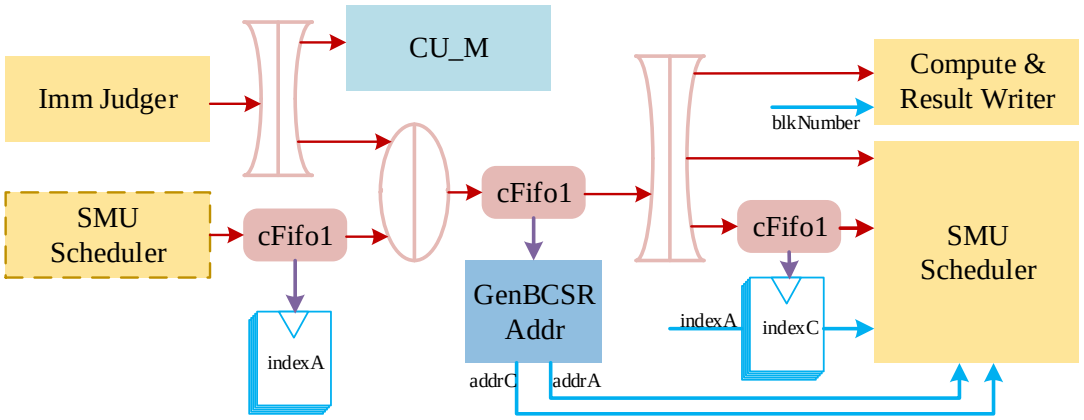


图 4-15　索引生成模块架构图

4.5.2　计算与结果回写模块（Compute & Result Writer）

图4-16是计算与结果回写模块的微架构，其功能是取出有效块进行计算，然后将结果回写对应的存储区。计算与结果回写模块收到 BCSR 搬移模块发来的事件后，取数地址生成（GenFetchAddr）逻辑生成取数地址，然后事件和取数地址（$fetchAddr$）给调度模块，取回要计算的四个数据。本模块设计由两个寄存器组（$buff0$ 和 $buff1$）构成的 PINGPONG-Buffer 用于缓存需要计算的数据，来提高 4 个处理单元（Process Element，PE）的利用率。每个缓冲区最多可以缓存 4 个 32 bit 数据。存储数据从 $buff1$ 开始，数据的每次写入是根据取数地址生成逻辑内部的计数的最低进行选择，最低位是 0 写入 $buff1$，反之写入 $buff0$。缓存有效数据的 buffer 通过 2 路选择器将数据送入 4 个 PE 进行计算，同时回写地址生成（GenWriteAddr）逻辑产生计算结果地址（$resAddr$）。计算完成后将计算结果与相应地址绑定通过调度模块写入片内存储。本轮计算完成后使用另一个 buffer 里存储的数据进行下次计算，使得 PE 可以不断工作，提高计算效率。取数地址生成逻辑内部的计数器值大于有效块的个数时，产生结束标志位，计算与结果回写模块回写最后一个结果时，同时发送事件至矩阵控制单元。

4.6　矩阵、向量压缩单元（COMPR）与解压单元（DECOMPR）

工作[14,17]提出的矩阵计算架构计算压缩后的稀疏矩阵，并用压缩格式存储其计算结果。两个工作对稀疏矩阵的压缩、解压依赖于上位机算力。无论是压缩矩阵、向量还是解压矩阵、向量，需要根据压缩格式编写压缩、解压程序。对于大规模矩阵，上位机执行压缩、解压程序时，大量指令的译码操作是一笔不菲的

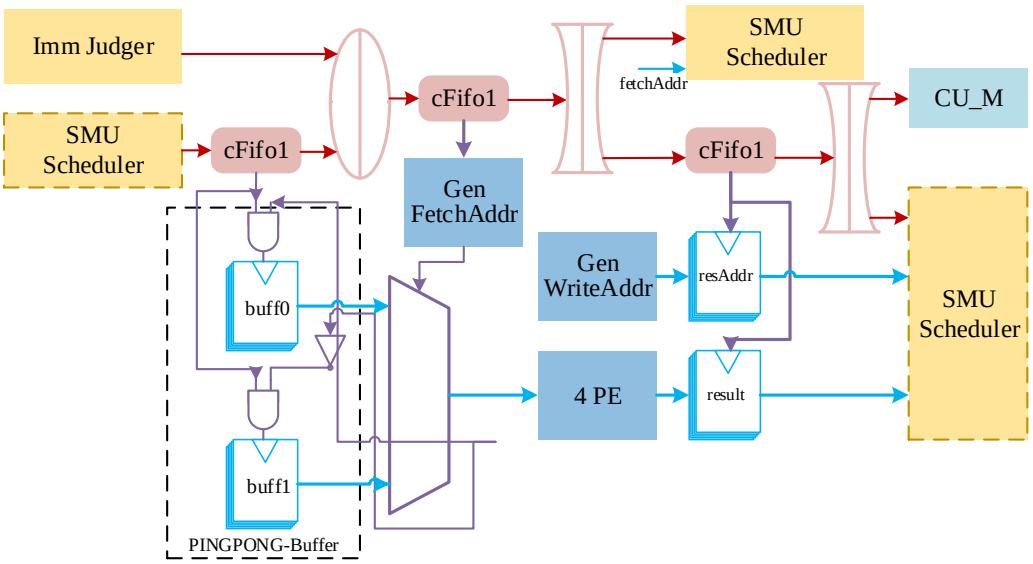


图 4-16 计算与结果回写模块架构图

开销。多模式矩阵加速器的输入和输出要求 3 种矩阵、1 种向量的压缩、解压方案，设计定制化的硬件压缩、解压单元，能够有效提升压缩、解压性能，完善多模式矩阵加速器功能。

关于涉及的 4 种压缩、解压方式，区分的方式是扩展压缩、解压计算微码至 6 位，对高两位编码。对于行向量、列向量，它们的压缩方式是一致的，其高位编码为 00_2。矩阵的压缩解压方案，以矩阵乘法为例 $C = AB$ 进行介绍。一是 BCSC 格式存储矩阵 A 的转置 A^T，其编码为 01_2；二是使用编码 10_2 标识 BCSC 存储矩阵 B；三是 BCSR 格式存储矩阵 C，编码是 11_2。

4.6.1 矩阵、向量压缩单元（COMPR）

压缩单元的功能是矩阵、向量的压缩，其架构如图4-17所示，内部可划分为块遍历模块（Block Traverser）、结果回写模块（Result Writer）和压缩调度模块（COMPR Scheduler）3 个模块，其中结果回写模块复用三级流水计算单元中的结果回写模块，进一步节约设计开销，此处不再赘述。

4.6.1.1 块遍历模块（Block Traverser）

块遍历模块微架构如图4-18所示，除粗粒度异步微流水线外，包含地址生成（GenDataAddr）逻辑、双缓冲区和转置（Transpose）逻辑 3 个部分。地址生成逻辑支持矩阵 BCSR、BCSC 格式和向量压缩下 3 种格式下的地址逐次生成。块遍历模块内拥有两个缓冲区：输入缓冲区和输出缓冲区（$inBuffer$ 和 $outBuffer$）。每个缓冲区的大小均可容纳一个子矩阵与子向量，其中输入缓冲区用于暂存取回

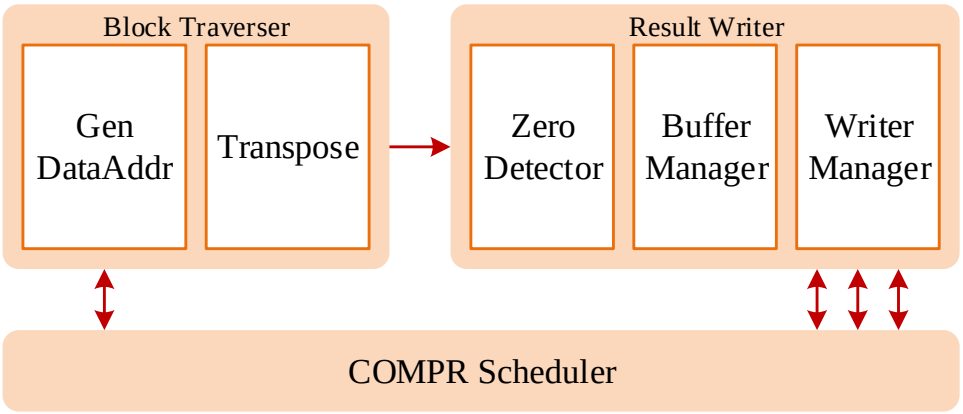


图 4-17 矩阵、向量压缩模块架构图

的数据，输出缓冲区存储输出到结果回写模块的数据。转置逻辑能够将子矩阵中的元素进行转置。其工作与否，取决于压缩格式编码是否为 10_2。

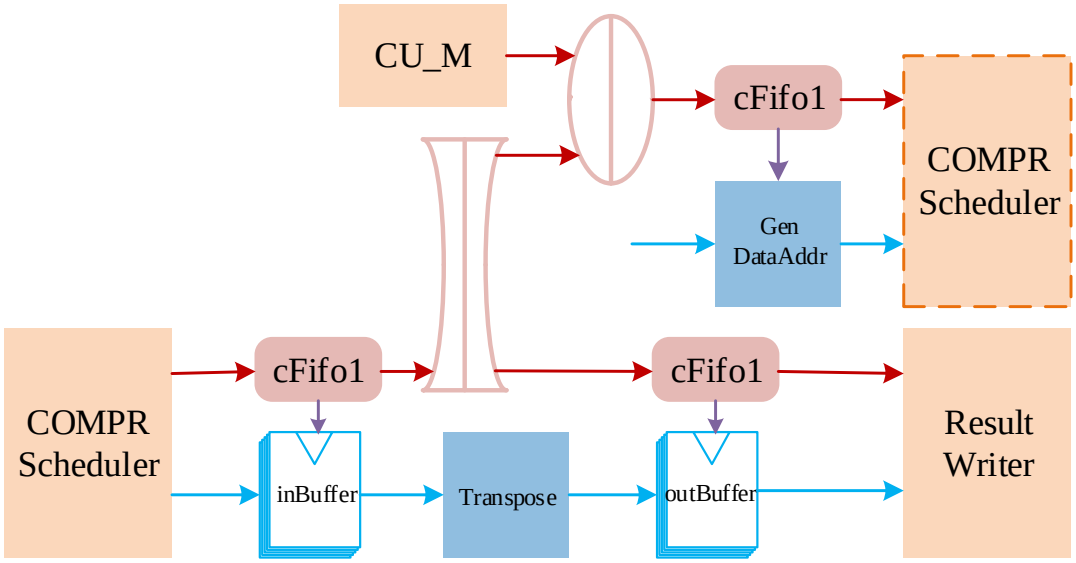


图 4-18 块遍历模块架构图

块遍历模块接收矩阵控制单元的事件后开始第一轮的工作，将地址生成逻辑计算的取数地址发送到压缩单元的调度模块，通过访存接口进行取数。压缩单元的调度模块读数返回的事件驱动块遍历模块继续工作，根据压缩模式编码决定驱动的次数，完成对矩阵或向量的遍历。对于编码为 01_2 的压缩方案，子矩阵从 $inBuffer$ 输出至 $outBuffer$ 的过程中需对子矩阵进行转置操作。

4.6.2 矩阵、向量解压单元

多模式矩阵加速器的解压单元能够完成上述 4 种解压模式，图4-19展示其微架构，解压流程共分为三个部分，包括解析 BCSR 格式、取回有效块以及回写子

矩阵。和矩阵计算单元相同，解压单元需要解析矩阵、向量的 BCSR 格式。矩阵计算单元同时解析两个输入的 BCSR 格式并匹配，但是解压单元只要解析一个输入，因此解压单元根据功能需求将矩阵计算单元中的 BCSR 解析器中指针取数器、索引取数器、标志位数组生成器进行裁剪、复用，进而形成解压单元中的 BCSR 解析模块（DECOMPR BCSR Parser）。回写子矩阵包含有效块的回写和零值块的回写、矩阵转置。

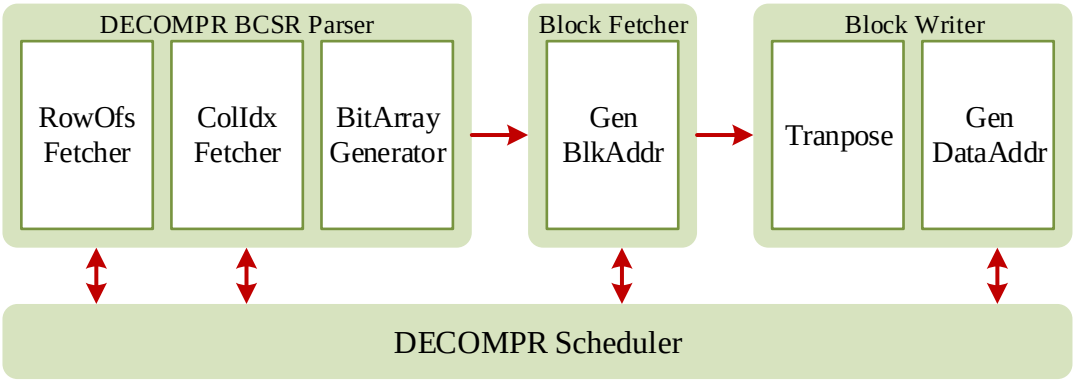


图 4-19　矩阵、向量解压单元架构图

4.6.2.1 指针取数器、索引取数器与标志位数组生成器

4.3.1 节 BCSR 解析器已详细介绍 3 个模块，本小节侧重讲述三个模块的工作流程。指针取数器接收到控制单元的启动事件后，若指针缓冲区非空，根据计算信息给出当前行的结束指针（*endPtr*），同时驱动索引取数器。否则，从片内存储中取回新的一行的指针写入指针缓冲区。索引取数器每次被驱动，判断内部的块计数器（*blkCounter*）是否小于当前的结束指针，判断条件为真则从非空的索引缓冲区中取出索引给到标志位数组生成器，若索引缓冲区为空，则从存储中区中取回新的一行，判断条件为假则生成行完成标志位驱动标志位数组生成器。标志位数组生成器如果接收到索引，则根据索引生成标志位数组后，驱动索引取数模块。标志位数组生成器接收到行完成标志位，表示本行有效的列索引号都遍历完，则绑定标志位数组和驱动事件发送给有效块取数器（Block Fetcher）。当块回写模块（Block Writer）写完当前行，如果未解压完成则驱动指针取数器进行下一行的解析，否则向矩阵控制单元写入完成标志位。

4.6.2.2 有效块取数器（Block Fetcher）

有效块取数器的微架构见图4-20，其功能是逐位遍历标志位数组，根据标志位数组信息选择驱动调度器或块回写模块又或自身。内部的地址生成逻辑用于

生成有效块的地址，由于在 BCSR 格式内部有效块是紧密排布，因此实际是简单的累加逻辑。工作逻辑如下：

（1）标志是 1 时，地址生成逻辑产生有效块地址用于取数。取回的有效块缓存在模块内部能存储 16 个元素的 $inBuffer$ 中。如果时矩阵类型的解压，则访存 4 次；向量访存 1 次。有效块被完整取回后，驱动块回写模块的同时，驱动自身进行下一标志位解析。

（2）标志位为 0 时，地址生成逻辑不工作，直接同时驱动块回写模块与自身，进行下一标志位的解析。

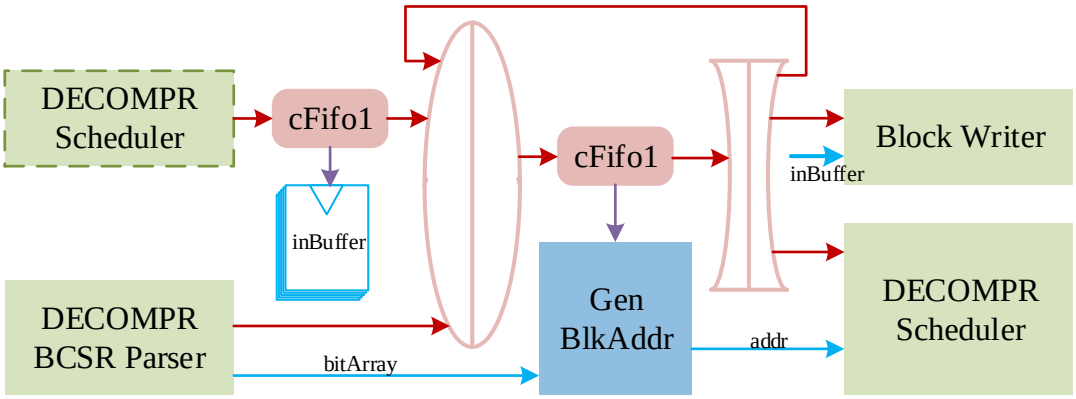


图 4-20　解压单元有效块取数器微架构

4.6.2.3 块回写模块（Block Writer）

图4-21展示了块回写模块的微架构，内部包含转置逻辑、输出缓冲区（*outBuffer*）和回写地址生成逻辑，其内部功能是回写有效块或者全零子矩阵。工作逻辑如下：

（1）当标志位为 1 时，3 路选择器根据解压码，选择有效块或有效块转置后的结果写入 $outBuffer$。地址生成逻辑计算产生有效块的起始地址，然后将绑定的地址与数据通过调度模块进行回写。

（2）当标志位是 0 时，$outBuffer$ 中将被写入全零，用于被写回。

如果当前回写的子矩阵或子向量不是最后一块，调度模块返回事件后会通过走 sink 这个无效路径。如果是最后一块，则发送事件至矩阵计算单元。

4.7　本章小结

本章结合存储格式、分块策略的分析结果和内-外积融合算法，基于粗粒度异步微流水线技术提出异步多模式矩阵加速器架构。本章首先在整体层面简要

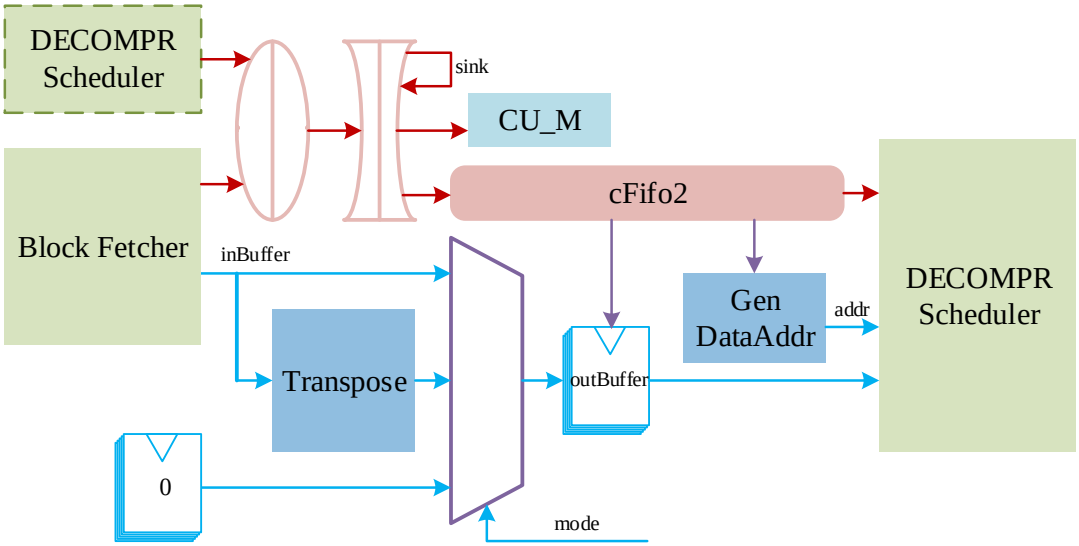


图 4-21 解压单元数据回写模块微架构

地介绍了多模式矩阵加速器的整体架构，接着介绍该架构支持的矩阵乘法、数乘向量、向量的压缩和解压等在内的 11 种计算操作与几种数据类型，以及控制单元的工作流程。然后详细介绍存储、访存接口、矩阵计算单元、数乘单元、解压单元和压缩单元在内的 7 个核心单元的工作机制。

第五章　异步多模式矩阵加速器的实现与验证

在第五章提出的多模式矩阵加速器架构的基础上，本章首先设计了能够用于验证矩阵加速器的功能是否符合设计目标，和分析矩阵加速器性能、功耗指标的测试方案。然后，按照 ASIC 前端设计流程采用 110*nm* 工艺实现、验证多模式矩阵加速器，从性能、面积和功耗三个方面比较分析。

5.1　异步多模式矩阵加速器测试方案设计

5.1.1　异步片上系统整体架构

本文提出的多模式矩阵加速器通过异步握手协议与上位机交互，上位机通过读写多模式矩阵加速器控制单元内的寄存器配置加速器。单个多模式矩阵加速器的矩阵计算单元内部仅部署 16 个乘累加器，难以像 GPU 那样提供大规模算力，然而在端侧 MCU 执行低阶张量计算任务时，其能够有效填补算力短板，显著提升计算效率。为验证多模式矩阵加速器作为协处理器的可行性，参考工作[17]中的验证方案，将多模式矩阵加速器作为单核 CPU 的协处理器，集成 2 种常见的外设接口组成片上系统，并在 ASIC 前端设计流程下实现。

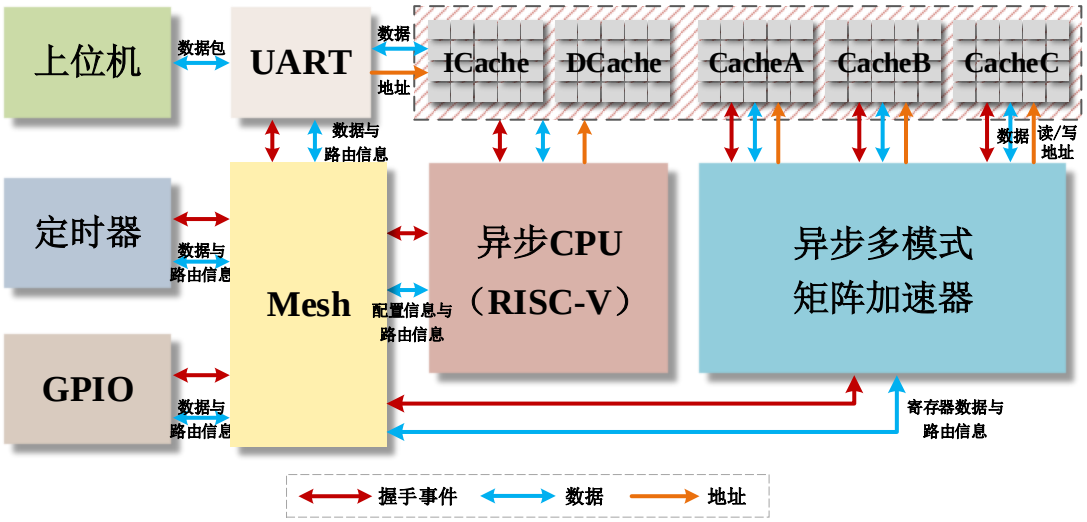


图 5-1　异步片上系统架构图

片上系统的整体框架如图5-1所示，内部包含一个哈佛架构的异步 RISC-V CPU 核，支持 RV32IMC 指令子集，该 CPU 核通过 mesh 型的 NoC 与 UART、GPIO、定时器和多模式矩阵加速器互联。片上系统中除片内存储以外的模块，均采用本实验室团队自主研发的异步电路设计。片上系统的具体工作流程如下：上

位机首先通过串口初始化多模式矩阵加速器的内部存储、CPU 核内的指令存储器和数据存储器。紧接着 CPU 开始执行程序，通过访存指令配置多模式矩阵加速器后，矩阵加速器执行相应的计算任务，完成后产生中断告知 CPU。最后 CPU 将结果再通过串口传输到上位机，校验结果。

5.1.2 测试数据集

SciPy 是一个用于数学、科学和工程计算的 Python 开源库，它建立在 NumPy 之上，提供了许多高效的数值算法和工具，能够帮助研究人员更方便地处理各种科学计算任务。SciPy 由多个子模块组成，每个子模块专注于特定的科学计算领域。scipy.sparse 及其子模块提供了用于处理稀疏矩阵的工具。

由于目前多模式矩阵加速器中的处理单元仅支持整型数据类型，本研究使用 scipy.sparse 模块随机生成 128×128 的矩阵、以及长度为 128 的向量作为多模式矩阵加速器的整型测试数据集，用于验证多模式矩阵加速器在不同模式下的计算操作是否正确。

图5-2为矩阵、向量的生成代码，通过修改 random 函数中的函数变量调整生成矩阵和向量的特征。scipy.sparse 模块并不支持生成 BCSR/BCSC 格式压缩的稀疏矩阵，所以 random 函数中的 $format$ 参数指定为 dense 生成非压缩格式下的矩阵。通过修改 random 函数中的 $density$ 变量来改变生成矩阵、向量的稀疏度。然后调用定制的矩阵、向量格式转换函数 CompressVector 和 CompressMatrix，对源矩阵、源向量和计算结果中的矩阵、向量进行压缩，并写入文件。random 函数中的参数 m, n 用于调整生成矩阵的维度，参数 $dtype$ 用于调整矩阵中元素的数据类型，由于要符合当前版本矩阵加速器的计算特征，在生成随机矩阵、向量的步骤中不做更改。

关于生成矩阵、向量中稀疏度的考量如下，对于随机生成的矩阵、向量中的非零元素，大概率呈现非零元素均匀分布在矩阵、向量中的各个位置，导致矩阵、向量的压缩效率大幅降低。在稀疏度低于 70% 时，大部分的稀疏矩阵压缩过后查看压缩格式发现并不存在非零块，其计算时间无法得到改善。所以，当稀疏度低于 90% 时，随机生成一组稀疏度为 0%、70%、80%、90% 进行统计多模式矩阵加速器的性能。当稀疏度高于 90% 时低于 99% 时，每 0.5% 稀疏度跨度随机生成 50 组数据对矩阵加速器的性能进行统计，记录平均计算时间。当稀疏度高于 99% 时，每 0.1% 稀疏度的变化，生成 100 组的数据对矩阵加速器的性能进行统计。

```
# generate vecA and vecB, and change them to np.ndarray type.
vecA = random(m=1, n=128, density=(1-0.0), format='dense', dtype=np.int32) # 行向量
vecB = random(m=128, n=1, density=(1-0.0), format='dense', dtype=np.int32) # 列向量
CompressVector(vecA.A1, file="./data_folder_MXU/vectorA.txt")
CompressVector(vecB.A1, file="./data_folder_MXU/vectorB.txt")
# generate matrixA and matrixB
matrixA = random(m=128, n=128, density=(1-0.0), format='dense', dtype=np.int32)
matrixB = random(m=128, n=128, density=(1-0.0), format='dense', dtype=np.int32)
# compress matrixA and matrixB
# write them to files
# A^T : BCSC
PtrA_mul, IdxA_mul, dataA_mul = Dense2BCSR(matrixA, compress=2)
writeBCSR(PtrA_mul, IdxA_mul, dataA_mul, file='./data_folder_MXU/matrixA_mul.txt')
# B   : BCSC
PtrB_mul, IdxB_mul, dataB_mul = Dense2BCSR(matrixB, compress=3)
writeBCSR(PtrB_mul, IdxB_mul, dataB_mul, file='./data_folder_MXU/matrixB_mul.txt')
# A : BCSR
PtrA, IdxA, dataA = Dense2BCSR(matrixA)
writeBCSR(PtrA, IdxA, dataA, file='./data_folder_MXU/matrixA.txt')
# B : BCSR
PtrB, IdxB, dataB = Dense2BCSR(matrixB)
writeBCSR(PtrB, IdxB, dataB, file='./data_folder_MXU/matrixB.txt')
```

图 5-2 测试数据集生成代码

5.1.3 基于 SAIF 文件的功耗分析

在芯片设计中，功耗分析是实现低功耗设计的关键环节。通过系统的功耗分析，设计者能够准确识别高功耗模块，并基于此采用诸如低功耗工艺技术、电源门控以及动态电压与频率调节（DVFS）等策略，以实现功耗的精细化控制。这不仅有助于显著延长芯片寿命、降低散热开销及系统成本，还能确保芯片满足日益严格的能效标准与环保要求。Design Compiler（DC）、Vivado 等电子设计自动化（Electronic Design Automation，EDA）软件可根据参数信息计算电路设计的动态功耗与静态功耗。本研究对多模式矩阵加速器采用基于 SAIF 文件的功耗分析方法，可以精确统计不同模式下多模式矩阵加速器的平均功耗。

SAIF 文件即开关活动内部交换格式文件，被用于分析、优化设计的动态功耗。SAIF 文件记录了系统动态变化率，通常被用于动态功耗分析。开关活动（switching activity）包含静态概率（static probability）和转换率（toggle rate），静态概率是指一个信号处于某个特定的逻辑状态的概率，转换率指信号在单位内像线网、引脚等设计对象从逻辑 1 到逻辑 0 和逻辑 0 到逻辑 1 的转换次数[76]。

图5-3为 ASIC 实现下的功耗分析流程，其中 Tech Library 为代工厂提供的记录标准单元功耗信息的工艺库，Netlist 为电路设计的门级网表，Switching Activity 为网表中每个标准单元的开关行为情况，Parasitic 为设计中连线等寄生参数，比如寄生电容、寄生电阻。电路设计流程中，SAIF 文件在仿真过程中由仿真软件（如，VCS）生成。图5-4为本文在 ASIC 前端设计流程下使用的功耗分析脚本，其中命令分为开启软件的功耗分析功能、读取 ASIC 实现过程中使用的工艺库、读

取时序文件、读取 SAIF 文件和报告功耗 6 个部分。

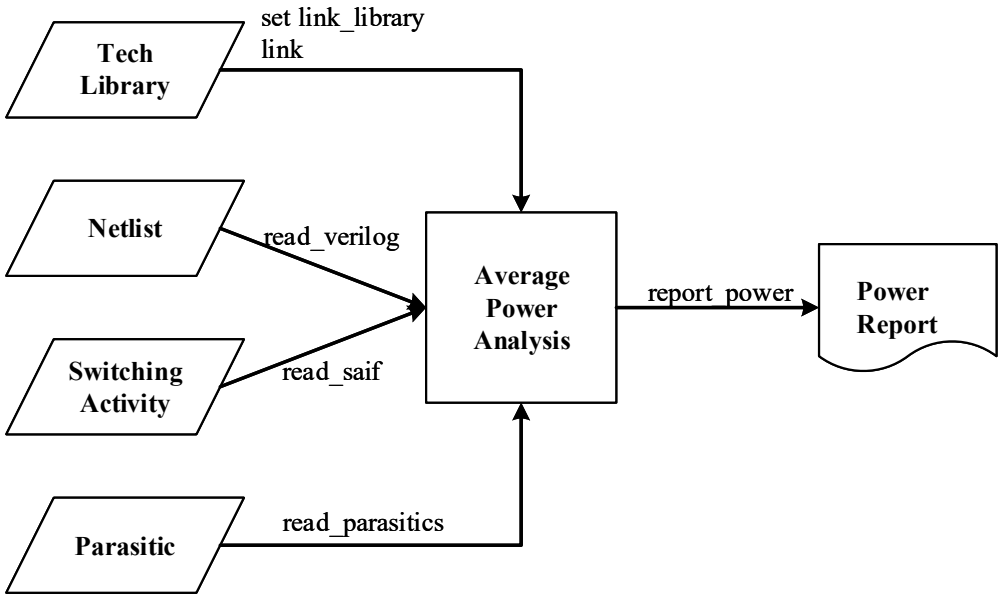


图 5-3　ASIC 下基于 SAIF 文件的功耗分析流程

```
set power_enable_analysis true

# set UMC110 std_cell, memory lib
set stdcel_libs "/backup/pdk/UMC/UMC110/project/sc/synopsys/ua11lscll12hdre_108c125_wc.db"
set memory_libs "/public/SRAM_UMC110/SHKD110_4224X8X16BM1/SHKD110_4224X8X16BM1_ss1p08v125c.db"
set target_library "$stdcel_libs $memory_libs"
set link_library   "* $target_library"

# read netlist
read_verilog "/team/asc/zhang.lingzhuang/2024Matrix/DC_zhang/output/MXU.mapped.v"
link_design MXU
set top MXU
current_design $top

# read sdf
read_sdf -analysis_type on_chip_variation "/team/asc/zhang.lingzhuang/2024Matrix/DC_zhang/output/MXU.sdf"

# read saif
read_saif "/team/asc/zhang.lingzhuang/2024Matrix/VCS_frontend/Matrix/Matrix.saif" -strip_path tb_Matrix/Matrix

# report power based on saif file
check_power
report_power > MXU_power.rpt
```

图 5-4　基于 SAIF 文件的功耗分析脚本

5.2　异步多模式矩阵加速器的 ASIC 实现与分析

本研究基于 ASIC 前端设计流程在 110*nm* 工艺下实现多模式矩阵加速器，最后进行性能、面积和功耗分析。

5.2.1　异步电路系统 ASIC 前端设计流程

ASIC 设计的前端流程结果为后端流程提供网表时序、时序文件等核心文件。在前端流程中对电路设计进行充分的验证、分析，才能保证经过布局布线、时钟树综合、寄生参数提取等一系列后端流程后数字电路的正确性。基于粗粒度异步

微流水线的电路系统模块间依赖时序来保证正常工作，因此同步 ASIC 设计流程中的功能仿真步骤的意义不大。异步电路系统 ASIC 实现下的前端流程如图5-5所示，包含 4 个步骤，虚线表示 RTL 设计的迭代。

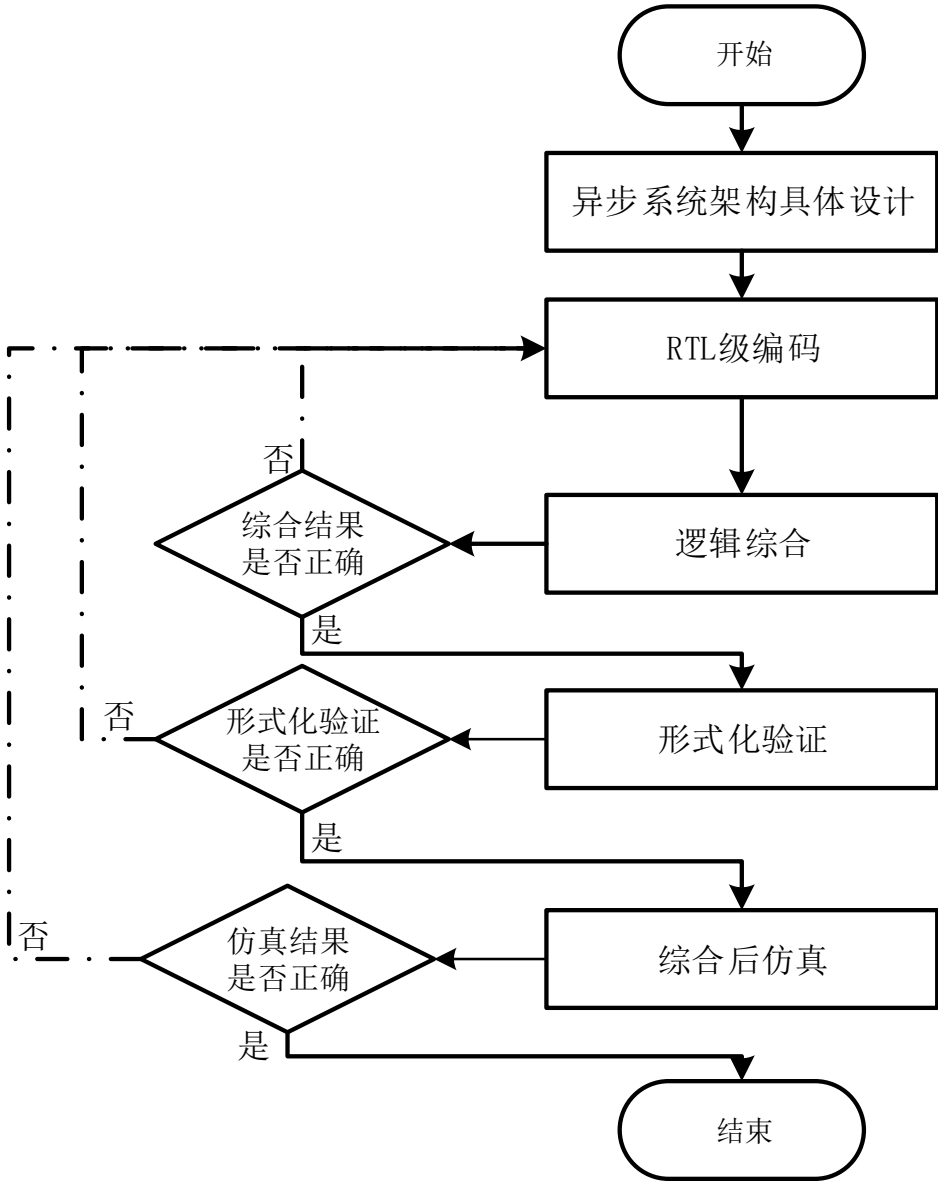


图 5-5　异步电路系统 ASIC 前端流程

5.2.1.1 RTL 级编码

根据整体系统的功能需求完成对多模式矩阵加速器的架构设计和子模块的微架构设计后，使用 Verilog 编写相应的子模块、系统的代码，用于下一步的逻辑综合步骤。异步多模式矩阵加速器 RTL 设计编码的第一步是使用工艺库标准单元构建的 Click 型“发送-中继-接收”结构，进而构建粗粒度异步微流水线中的组件，如图5-6所示。接着依据子模块的微架构图实现数据通路上的核心组合

逻辑，以及采用粗粒度异步微流水线搭建控制通路。图5-7展示多模式矩阵加速器的核心 RTL 代码框架，这些模块最终构成异步多模式矩阵加速器。

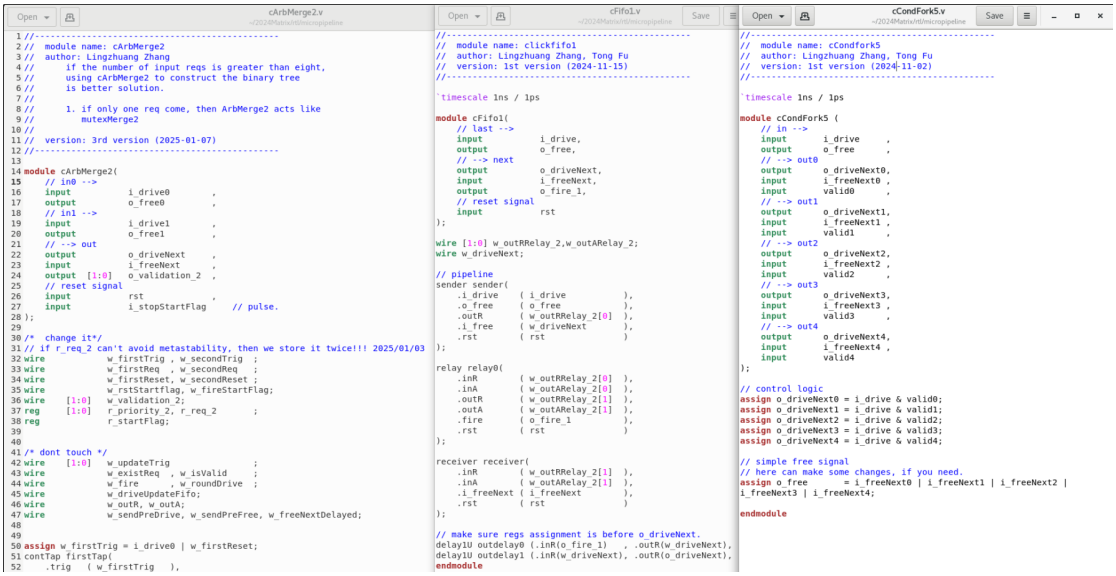


图 5-6　粗粒度异步微流水线搭建

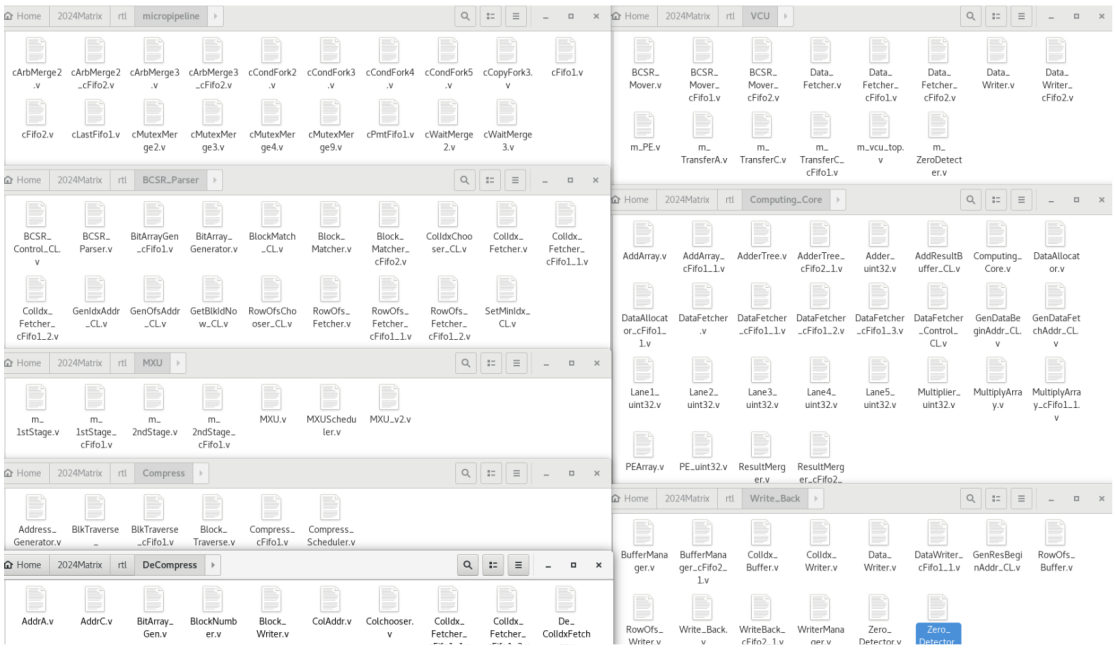


图 5-7　多模式矩阵加速器 RTL 设计

5.2.1.2 逻辑综合

逻辑综合是根据芯片代工厂提供的标准单元库，将硬件描述语言描述的 RTL 级电路转换成门级网表的过程。在这一步骤中，综合工具会根据工艺库文件和时序约束文件的要求，在符合最大信号跳变时间、速度和面积等条件下，将行为级

描述转化为由标准单元构成的拓扑网络连接，并保证电路功能与 RTL 代码一致。逻辑综合产生门级网表和时序文件，作为形式化验证和综合后仿真测试的输入，用于后续验证流程。

研发人员可以根据综合工具的运行日志和报告文件，初步判断代码设计是否正确，然后根据实际需要调整代码、综合策略等。本文采用新思科技的 DC 工具对多模式矩阵加速器设计进行逻辑综合，将工艺库的设置、综合的若干指令序列、设计的约束写入 tcl 脚本方便快速执行。在综合过程中，设计关键计算单元的最长路径约束，能够有效提升多模式矩阵加速器的整体性能，图5-8是约束最大延迟的脚本语言，将矩阵计算单元中的乘累加器的最长路径约束至 10 *ns*。

```
#PE_uint32
## Lane4
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE12/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE12/* -filter {@port_direction ==out}]
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE13/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE13/* -filter {@port_direction ==out}]
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE14/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE14/* -filter {@port_direction ==out}]
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE15/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane4_uint32/u_PE15/* -filter {@port_direction ==out}]

## Lane3
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE8/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE8/* -filter {@port_direction ==out}]
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE9/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE9/* -filter {@port_direction ==out}]
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE10/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE10/* -filter {@port_direction ==out}]
set_max_delay 10 -from [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE11/* -filter {@port_direction ==in}] \
                 -to   [get_ports u_Computing_Core/u_PEArray/u_MultiplyArray/u_Lane3_uint32/u_PE11/* -filter {@port_direction ==out}]
```

图 5-8　乘累加器最长路径约束语句

5.2.1.3 形式化验证

形式化验证是通过数学方法（如逻辑推理、模型检查等）来验证数字电路设计与逻辑综合的门级网表在逻辑功能上是否一致。本文使用新思科技的 Formality 工具对多模式矩阵加速器进行形式化验证。Formality 工具借助等价性检测方法检验代码设计与网表文件是否等价，确保综合过程中没有改变 RTL 设计的逻辑功能。图5-9中的“Verification Succeeded”表示多模式矩阵加速器的综合后的门级网表和 RTL 设计一致。

5.2.2　综合后仿真验证

完成形式化验证后，使用综合产生的门级网表和时序文件进行综合后仿真验证。本文在 Testbench 文件中模拟矩阵加速器与上位机的交互行为，验证其时序和功能。本研究借助新思科技的 VCS 仿真工具和 DVE 波形查看工具对多模式矩阵加速器进行综合后仿真，并将每种模式的计算结果写入文本。最后将计算结果与软件端的计算结果进行比较，判断此次计算的正确性。

验证矩阵加速器所使用的 Testbench 的核心逻辑如图5-10所示，多模式矩阵加速器配置信息读取、片内存储的初始化、计算结果的读写。将计算模式、矩阵

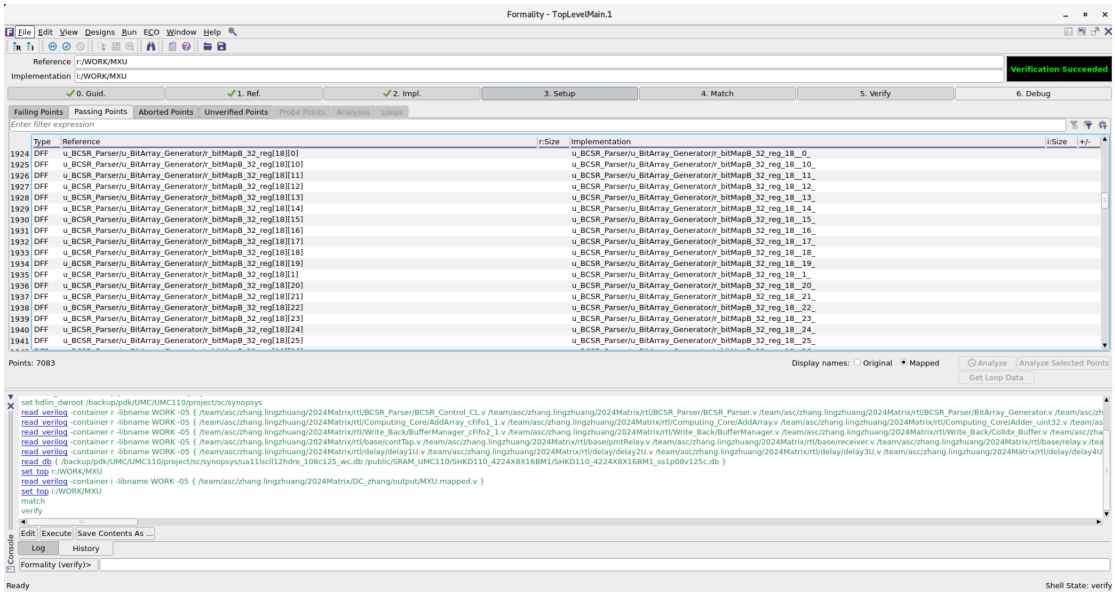


图 5-9　多模式矩阵加速器形式化验证结果

或向量的起始地址写在文本文件中，通过读取该文本文件配置多模式矩阵加速器的控制单元然后启动矩阵加速器。Testbench 读入记录矩阵、向量数据的文本文件，利用索引层级实现片内存储的快速初始化。每次矩阵加速器向片内存储中写入数据的同时将其存入 Testbench 中设置的寄存器组，多模式矩阵加速器计算完成后，将计算结果写入文件中。

```
initial begin
    fdB = $fopen(`INDEX, "w");
    fdC = $fopen(`RESULT, "w");
    fdD = $fopen(`STEP, "w");
    for(i=0; i<8192; i=i+1) begin
        sramA[i] = 128'b0;
        sramB[i] = 128'b0;
        sramC[i] = 128'b0;
    end
    for(i=0; i<7; i=i+1) begin
        r_config_32[i] = 32'b0;
    end
    $readmemh(`CONFIG, r_config_32);
    i_beginAddrA_32     = r_config_32[0];   // addrA
    i_beginAddrB_32     = r_config_32[1];   // addrB
    i_beginAddrC_32     = r_config_32[3];   // addrC
    i_computingMode_4   = r_config_32[4];   // mode
    i_dataType_4        = r_config_32[5];   // data type
    case(i_computingMode_4[3:0])
        4'b0001: begin
            $readmemh(`MXAMUL, sramA);
            $readmemh(`MXBMUL, sramB);
        end
        4'b0010: begin
            $readmemh(`MXA, sramA);
            $readmemh(`MXB, sramB);
        end
        4'b0011, 4'b0110, 4'b0111:begin
            $readmemh(`VCA, sramA);
            $readmemh(`VCB, sramB);
        end
        4'b0100:begin
            $readmemh(`MXAMUL, sramA);
            $readmemh(`VCB   , sramB);
        end
        4'b0101:begin
            $readmemh(`VCA   , sramA);
            $readmemh(`MXBMUL, sramB);
        end
    endcase
end

initial
begin
    #100 rst = 1;
    #100 rst = 0;
    #100 rst = 1;
    //$toggle_start;
    #10 i_driveFrmCU = ~i_driveFrmCU;
    #1  i_driveFrmCU = ~i_driveFrmCU;

    #1000 $stop;
end
```

```
// initial memory
initial begin
    for(i=0; i<4224; i=i+1) begin
        {u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte15[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte14[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte13[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte12[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte11[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte10[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte9[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte8[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte7[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte6[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte5[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte4[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte3[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte2[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte1[i],
         u_MXU.u_Memory.u_CacheA.u_SHKD110_4224X8X16BM1.Memory_byte0[i] } = sramA[i];
        {u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte15[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte14[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte13[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte12[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte11[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte10[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte9[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte8[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte7[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte6[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte5[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte4[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte3[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte2[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte1[i],
         u_MXU.u_Memory.u_CacheC.u_SHKD110_4224X8X16BM1.Memory_byte0[i] } = sramB[i];
        {u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte15[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte14[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte13[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte12[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte11[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte10[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte9[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte8[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte7[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte6[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte5[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte4[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte3[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte2[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte1[i],
         u_MXU.u_Memory.u_CacheB.u_SHKD110_4224X8X16BM1.Memory_byte0[i] } = sramC[i];
    end
```

```
// write data to sramC.
always@(posedge w_fireCacheB) begin
    sramC[r_addr_13] = r_result_128;
end

// write sramC to files.
always@(posedge o_driveToCU) begin
    #10 i_freeFrmCU   = ~i_freeFrmCU;
    #1  i_freeFrmCU   = ~i_freeFrmCU;
    #10 i_rstArbMerge = ~i_rstArbMerge;
    #1  i_rstArbMerge = ~i_rstArbMerge;
    //$toggle_stop;
    //$toggle_report("MXUv2.saif", 1.0e-9, u_MXU);
    case(i_computingMode_4[3:0])
        4'b0001, 4'b0010, 4'b0011 : begin
            for(i=0; i<4165; i=i+1) begin:block1
                if(i < (4*sramC[0][10:0]+11'd69)) begin
                    $fdisplay(fdC, "%h", sramC[i]);
                end
                else begin
                    disable block1;
                end
            end
        end
        4'b0110                   :begin
            $fdisplay(fdC, "%032h", sramC[0]);
        end
        4'b0100,4'b0101,4'b0111 : begin
            for(i=0; i<4165; i=i+1) begin:block2
                if(i < (sramC[0][10:0]+12'd3)) begin
                    $fdisplay(fdC, "%h", sramC[i]);
                end
                else begin
                    disable block2;
                end
            end
        end
        default                   : begin
            $fdisplay(fdC, "xxxxxxxxxxxxxxxx\n");
        end
    endcase

    $fclose(fdC);
end
```

图 5-10　多模式矩阵加速器 Testbench 的主要逻辑

多模式矩阵加速器支持 11 种计算模式并且模块众多，因此本小节展示部分模块的综合后仿真波形图。图5-11是稀疏矩阵乘法计算模式下 BCSR 解析器正确工作的仿真波形图，图5-12是使用文本比较工具对计算结果与软件计算结果的匹配结果图，两个文件中的数据一致说明多模式矩阵加速器计算正确。压缩单元中

的回写模块的仿真波形图如5-13所示，说明该模块能够判断从块遍历模块传来的子矩阵是否存在非零值，同时将结果写入片内存储中。图5-14是数乘单元进行数乘矩阵计算，并将结果回写存储的仿真波形，说明数乘矩阵计算正确。

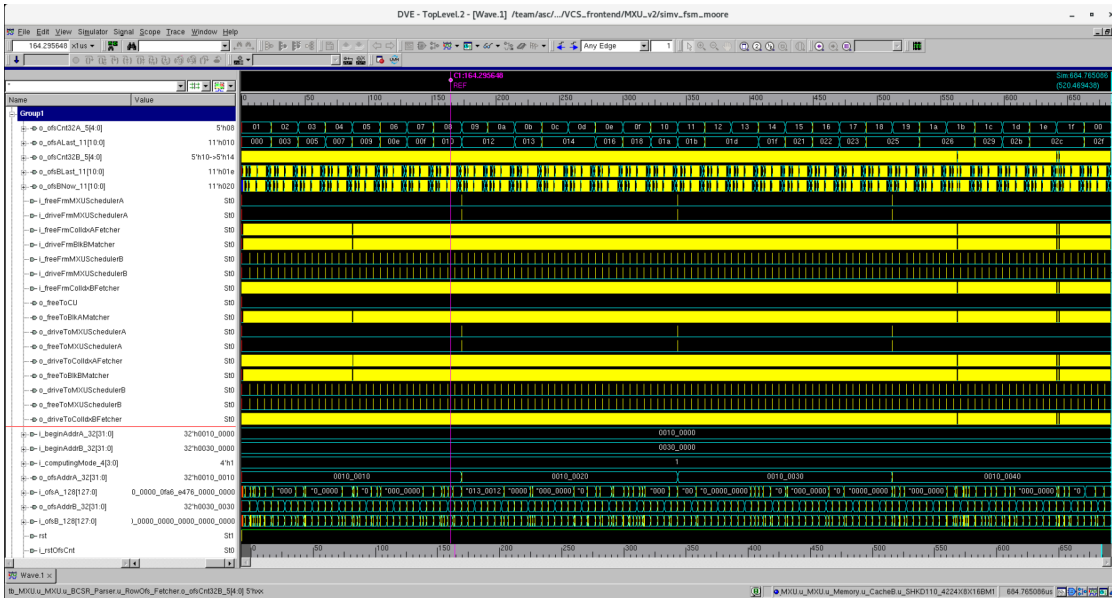


图 5-11　稀疏矩阵乘法综合后仿真波形

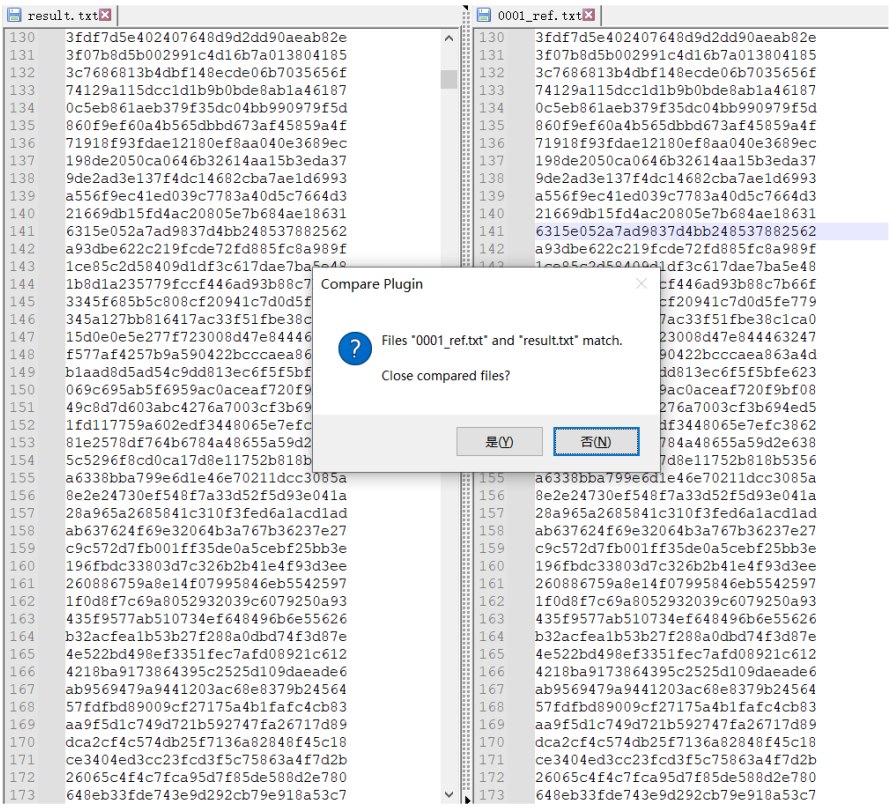


图 5-12　稀疏矩阵乘法计算结果匹配

通过大量不同稀疏度的测试数据集的仿真验证，本研究逐步完善多模式矩阵

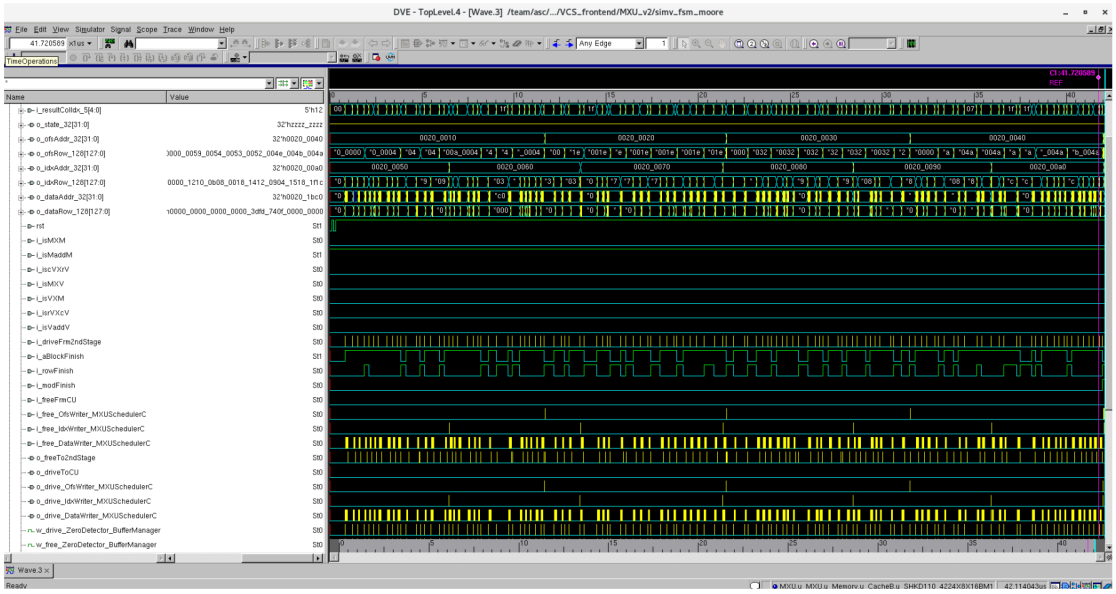


图 5-13　压缩单元回写模块仿真波形



图 5-14　数乘单元模块仿真波形

加速器的 RTL 设计，成功验证其支持的 11 种计算模式，最终在 ASIC 设计前端流程下实现多模式矩阵加速器。

5.2.3　性能分析对比

本文基于综合后仿真的时间统计多模式矩阵加速器的计算时间，使用 5.1.2 节中介绍的随机数据集，测试多模式矩阵加速器在 11 种模式下的平均计算时间。如表 5-1和表 5-2所示，表的每行记录某个稀疏度下不同计算模式各自的平均计算时间（单位：*us*）。表 5-1记录了矩阵相乘（M×M）、矩阵相加（M+M）、列向量乘行向量（cV×rV）、矩阵乘列向量（M×V）、行向量乘矩阵（V×M）。表 5-2记

录了行向量乘列向量（rV×cV）、向量相加（V+V）、数乘矩阵（imm×M）、数乘向量（imm×V）、矩阵压缩（comprM）、矩阵解压（decomprM）。其中由于向量的压缩、解压数据量较少，不纳入本文的性能统计。

稀疏度等于 99.9% 时，矩阵相乘、矩阵相加、向量外积、矩阵乘列向量、行向量乘矩阵、向量内积、向量相加、立即数乘矩阵、立即数乘向量的平均计算时间分别为 670.26 *us*、28.42 *us*、21.37 *us*、21.46 *us*、21.45 *us*、1.09 *us*、1.08 *us*、6.95 *us*、0.42 *us*。对于在矩阵计算单元中进行计算的 7 种模式，当矩阵的稀疏度接近 100% 时，解析、匹配矩阵、向量的存储格式是不可避免的时间开销，因此平均计算时间会逐渐趋近于某个值。然而，数乘单元进行的两种计算操作不需要解析矩阵、向量的压缩格式，所需要的计算时间与有效块的个数挂钩，当有效块的个数是 1 时，只要完成一次有效块的计算操作即可结束，因此数乘向量，数乘矩阵的平均计算时间会距趋近于 0 *us*。稀疏度为 0% 的矩阵压缩、解压操作所需要的时间是相同情况下矩阵乘向量模式的两倍，这是由于在本次设计中压缩单元和解压单元未采用流水线设计技术。当稀疏为 99.9% 时，矩阵的解压、压缩操作所需的时间并未大幅减少，这是因为不论是压缩矩阵还是解压矩阵需要完整地进行一次矩阵访存，压缩矩阵时输入矩阵是稠密矩阵要完整遍历，解压矩阵时结果矩阵的稠密矩阵要全部回写至片内存储。综上，除压缩、解压外的所有计算模式的平均时间随矩阵、向量的稀疏度的提高而快速收敛，说明本文提出的多模式矩阵加速器能够有效地加速稀疏度高的低阶张量计算。

下面从多模式矩阵加速器和矩阵计算单元（MXU）内的计算阵列的两个维度分析多模式多模式矩阵加速器的性能。首先根据多模式矩阵加速器的稠密矩阵乘法计算时间 7482.52 *us*，即花费 7482.52 *us* 时间完成 128×128 规模的矩阵乘操作，共 4177920 次计算操作。根据性能计算公式 5-1，公式中的 N_{op} 表示完成的计算操作数，t_{op} 表示所需要的时间，计算得到多模式矩阵加速器的 INT32 整型算力约为 558.36 *MOPS*。然后分析矩阵计算单元中的计算阵列的 INT32 算力，采用公式 5-2 中的统计方法，其中 N_{MAC} 表示乘累加器的数量，f 表示计算频率。在逻辑综合步骤中，已将乘累加器的最长路径约束至 10 *ns*，因此成累加器的等效主频是 100 *MHz*。最终计算得到计算阵列能够提供的最大算力是 3.2 *GOPS*。

$$P_{ops} = \frac{N_{op}}{t_{op}} \tag{5-1}$$

$$P_{ops} = 2 \cdot N_{MAC} \cdot f \tag{5-2}$$

分析两种算力评估方式得出的结果不同的原因，第一，多模式矩阵加速器是面向稀疏的低阶张量开发，存在解析低价张量的时间开销。第二，矩阵计算单元

表 5-1 多模式矩阵加速器的仿真性能统计 _1

稀疏度	M×M	M+M	cV×rV	M×V	V×M
0%	7482.52	255.10	174.82	176.52	237.70
70.0%	7418.90	255.27	109.72	197.81	179.10
80.0%	7143.51	253.91	66.38	149.96	137.44
90.0%	5239.84	240.41	37.17	87.78	88.47
90.5%	5079.71	239.22	35.67	86.11	81.89
91.0%	4857.53	236.80	35.67	84.33	79.99
91.5%	4617.82	233.84	32.59	77.44	72.30
92.0%	4425.87	230.75	31.37	75.91	65.96
92.5%	4160.38	227.46	31.14	70.53	70.1
93.0%	3820.36	224.17	32.53	67.36	66.89
93.5%	3751.17	220.11	25.37	54.25	55.18
94.0%	3309.67	212.40	28.32	57.25	57.49
94.5%	3201.91	210.23	25.72	50.43	49.64
95.0%	2850.44	200.96	24.83	46.52	48.18
95.5%	2571.53	193.12	24.85	44.77	46.20
96.0%	2270.02	182.99	23.75	40.87	41.85
96.5%	1992.33	172.32	22.87	36.63	36.85
97.0%	1714.10	159.44	22.87	35.14	35.19
97.5%	1442.21	144.16	22.31	32.06	31.68
98.0%	1197.38	126.42	22.31	29.93	29.64
98.5%	986.96	106.06	21.81	26.49	26.21
99.0%	825.12	83.60	21.44	24.12	23.93
99.1%	796.06	77.11	21.45	23.86	23.69
99.2%	770.23	72.49	21.44	23.55	23.37
99.3%	749.59	67.00	21.45	23.09	23.36
99.4%	728.82	60.95	21.45	22.94	23.22
99.5%	711.22	55.16	21.45	22.77	22.96
99.6%	696.25	48.69	21.45	22.67	22.49
99.7%	684.31	42.12	21.37	21.81	21.78
99.8%	676.43	35.57	21.37	21.65	21.63
99.9%	670.36	28.42	21.37	21.46	21.45

表 5-2 多模式矩阵加速器的仿真性能统计 _2

稀疏度	rV×cV	V+V	imm×M	imm×V	comprM	decomprM
0%	3.86	4.31	176.91	2.21	343.68	327.51
70.0%	2.68	4.05	176.06	2.17	342.97	326.97
80.0%	2.25	3.32	172.16	1.95	337.69	323.69
90.0%	1.32	2.88	144.50	1.32	323.31	302.30
90.5%	1.32	2.80	141.45	1.39	310.29	297.64
91.0%	1.32	2.80	136.87	1.34	302.28	295.04
91.5%	1.31	2.64	131.95	1.18	298.49	293.65
92.0%	1.31	2.57	132.29	1.04	296.51	289.00
92.5%	1.92	2.03	126.18	1.18	295.16	287.07
93.0%	1.93	2.03	120.58	1.18	292.73	281.44
93.5%	1.68	1.74	117.70	0.97	289.62	282.01
94.0%	1.75	1.87	109.38	1.04	288.70	274.45
94.5%	1.69	1.74	107.52	0.97	282.13	271.85
95.0%	1.63	1.75	99.37	0.90	276.52	264.59
95.5%	1.66	1.80	94.62	0.90	271.09	260.48
96.0%	1.60	1.73	86.65	0.90	256.18	252.78
96.5%	1.52	1.73	79.35	0.83	252.82	248.26
97.0%	1.51	1.70	69.68	0.76	246.74	240.73
97.5%	1.43	1.61	60.52	0.76	239.23	233.74
98.0%	1.25	1.58	51.70	0.69	230.01	226.05
98.5%	1.14	1.46	39.99	0.65	224.63	218.12
99.0%	1.13	1.27	28.28	0.62	211.24	207.69
99.1%	1.13	1.27	27.43	0.54	209.33	206.44
99.2%	1.13	1.27	24.72	0.53	201.15	203.57
99.3%	1.14	1.32	22.17	0.50	200.19	202.62
99.4%	1.13	1.27	18.78	0.49	198.74	199.86
99.5%	1.13	1.28	16.57	0.46	197.02	197.98
99.6%	1.13	1.28	13.86	0.45	196.38	196.04
99.7%	1.09	1.08	11.15	0.43	194.25	193.55
99.8%	1.09	1.08	8.77	0.42	193.47	191.66
99.9%	1.09	1.08	5.89	0.42	190.60	189.30

用较少的硬件资源支持 7 种计算模式，矩阵计算单元内部不可避免的增加了额外的控制逻辑。第三，公式 5-2 的评估方式是一种理想状态，没有考虑矩阵元素的调度和回写操作等时间开销。方式一计算得到的算力与方式二计算的比值约为 17.45%，说明多模式矩阵加速器存在优化空间，比如通过进一步细分流水线提升多模式矩阵加速器的算力。

为进一步分析基于 110*nm* 制程实现的本架构在稀疏矩阵乘法计算上的加速效果，本研究在主频为 4.1GHz 的 Intel i5-10600KF 上利用由 Intel MKL[77]库构建 python 下的 scipy 库进行一系列计算操作，并记录运行时间。本文只测量低阶张量计算的执行时间，不包含数据初始化时间，保证二者的一致性。两边使用相同的数据集进行测试，且保证代码读入的数据类型为整型，来保证性能测试公平性。并且通过多次测量求平均值的方式，使得获取 CPU 的执行时间更加精确。

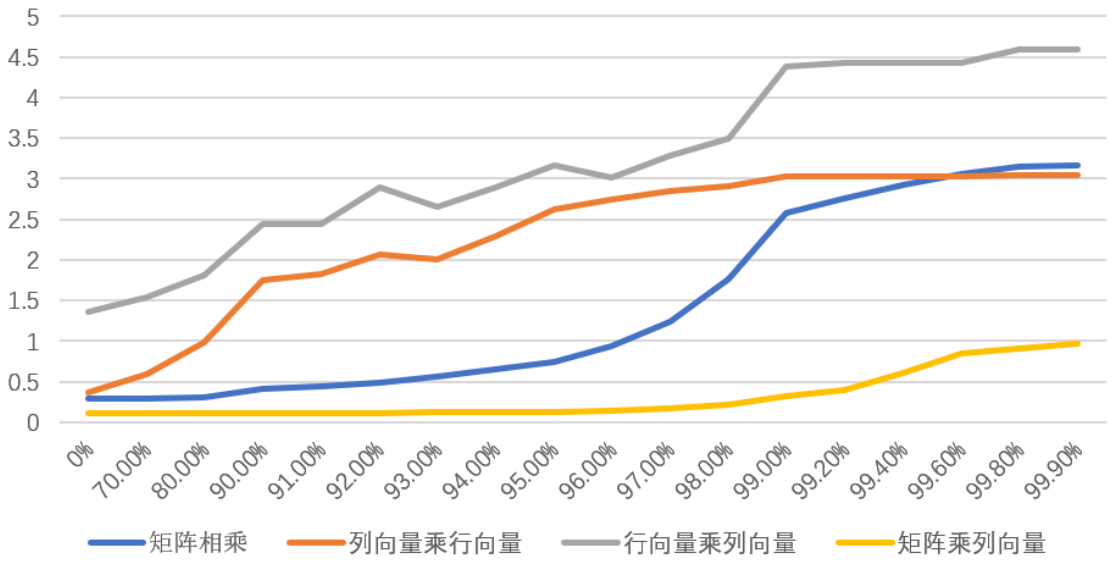


图 5-15　对于不同稀疏度本架构与 Intel MKL 库在 4 种模式下的加速比折线图

与 Intel MKL 库相比，本架构在矩阵相乘、矩阵乘列向量、列向量乘行向量、行向量乘列向量 4 种计算模式下的加速比曲线如图5-15所示。除解压、压缩模式外的其余计算模式，在稀疏度大于 99.5% 时计算性能接近 Intel MKL 库，不做过多展示。矩阵的稀疏度大于 99.00% 时，相较于 Intel MKL，本架构在矩阵乘法模式下实现了平均 2.5× 的平均加速。当矩阵的稀疏度大于 95.00% 时，相较于 Intel MKL，本架构在列向量乘行向量与行向量乘列向量 2 个模式下分别实现了至少平均 2.5×、3× 的平均加速。

5.2.4 面积报告与功耗分析

5.2.4.1 面积报告

多模式矩阵加速器验证完成后，使用 DC 工具查看面积报告，见图5-16。最终，多模式矩阵加速器的综合后面积是 5.4615 mm^2，其中控制单元面积是 0.0130 mm^2，约占 0.238%；矩阵计算单元（MXU）面积为 0.5871 mm^2，约占比 10.748%；数乘单元面积是 0.1227 mm^2，约占 2.2466%；压缩单元面积是 0.0823 mm^2，约占 1.507%；解压单元面积为 0.0456 mm^2，约占比是 0.835%；访存接口面积是 0.0251 mm^2，约占 0.460%；片内存储的面积是 4.5855 mm^2，约占 83.955%。矩阵计算单元内拥有最多的计算单元和复杂的控制逻辑，是 4 个计算单元中最大的。三片 66 KB 的 Cache 构成的片内存储占据大量的硬件资源，成为多模式矩阵加速器中最大的开销。

```
Number of ports:                        62324
Number of nets:                        141467
Number of cells:                        79743
Number of combinational cells:          67164
Number of sequential cells:             10660
Number of macros/black boxes:               3
Number of buf/inv:                      13133
Number of references:                      10

Combinational area:             589623.215883
Buf/Inv area:                    62952.287985
Noncombinational area:          299869.587317
Macro/Black Box area:          4572046.500000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:               5461539.303200
Total area:                 undefined

Hierarchical area distribution
------------------------------

                                 Global cell area                Local cell area
                                 ----------------------  ---------------------------------------
Hierarchical cell                Absolute      Percent   Combi-       Noncombi-    Black-
                                 Total         Total     national     national     boxes          Design
-------------------------------  ------------  -------   -----------  -----------  -------------  ---------------
Matrix_top                       5461539.3032    100.0     264.9600       0.0000        0.0000  Matrix_top
u_CU_M                             12954.3360      0.2    4225.0080    7424.4000        0.0000  CU_M
u_MXU                             587075.1860     10.7      24.2880       0.0000        0.0000  MXU
u_SMU                             122743.8242      2.2      72.8640       0.0000        0.0000  SMU
u_COMPR                            82316.4484      1.5       7.7280       0.0000        0.0000  COMPR
u_DECOMPR                          45597.4081      0.8       7.7280       0.0000        0.0000  DECOMPR
u_AccessInterface                  25127.0403      0.5   21990.5762       0.0000        0.0000  AccessInterface
u_Memory                         4585460.1002     84.0     407.3760   11012.4002        0.0000  Memory
```

图 5-16 多模式矩阵加速器面积报告

5.2.4.2 功耗分析对比

DC 工具的功耗报告如图5-17所示，总功耗是由内部功耗（internal power）、开关功耗（switching power）和漏电功耗（leakage power）组成，多模式矩阵加速器的总功耗是 1.6507 mW，内部功耗为 0.5677 mW，开关功耗是 1.0283 mW，漏电功耗是 54.7408 uW。由于异步多模式矩阵加速器在不同的工作模式下只有特定的计算单元工作，DC 工具的功耗报告并不能准确反映多模式矩阵在计算时的平均功耗。因此，本研究采用 5.1.3 节基于 SAIF 文件的功耗分析方法，提高功

耗分析的准确性。功耗分析的软件使用新思科技的 Prime Time（PT）工具。使用 VCS 工具生成多模式矩阵加速器运行时的 SAIF 文件。

```
Design        Wire Load Model            Library
------------------------------------------------
Matrix_top             wl0               ua11lscll12hdre_108c125_wc


Global Operating Voltage = 1.08
Power-specific unit information :
    Voltage Units = 1V
    Capacitance Units = 1.000000pf
    Time Units = 1ns
    Dynamic Power Units = 1mW    (derived from V,C,T units)
    Leakage Power Units = 1pW


Attributes
----------
i - Including register clock pin internal power


  Cell Internal Power  = 567.7034 uW   (36%)
  Net Switching Power  =   1.0283 mW   (64%)
                         ---------
Total Dynamic Power    =   1.5960 mW  (100%)

Cell Leakage Power     =  54.7408 uW

Information: report_power power group summary does not include estimated clock tree power. (PWR-789)

                 Internal         Switching           Leakage            Total
Power Group      Power            Power               Power              Power   (   %    )  Attrs
--------------------------------------------------------------------------------------------------
io_pad             0.0000           0.0000       |     0.0000           0.0000  (   0.00%)
memory             0.0000        4.1902e-02        5.0806e+07        9.2709e-02  (   5.62%)
black_box          0.0000           0.0000            0.0000           0.0000  (   0.00%)
clock_network      0.0000           0.0000            0.0000           0.0000  (   0.00%)  i
register           0.0000           0.0000            0.0000           0.0000  (   0.00%)
sequential         0.2611        5.6252e-04        1.1855e+06           0.2629  (  15.92%)
combinational      0.3066           0.9858        2.7489e+06           1.2952  (  78.46%)
--------------------------------------------------------------------------------------------------
Total              0.5677 mW        1.0283 mW     5.4741e+07 pW        1.6507 mW
```

图 5-17　多模式矩阵加速器 DC 功耗报告

VCS 工具生成 SAIF 文件需要在 Testbench 文件中插入如下 5 条命令。命令“set_gate_level_monitoring”用于开启或关闭门级单元的仿真行为监控，本研究是基于门级网表的功耗分析，所以开启门级单元的仿真行为监控。命令“set_toggle_region”可用于设置一个特定模块进行信号翻转的监测，本研究设置整个多模式矩阵加速器作为监控对象。VCS 工具通过命令“toggle_start”和“toggle_stop”开启或停止对模块在工作过程中的仿真行为的监测。命令“toggle_report”能够将监测模块的门级行为按照某个时间精度写入指定 SAIF 文件，这里设置的时间精度是纳秒级（$1.0e-9$）。

```
1    $set_gate_level_monitoring("on");
2    $set_toggle_region(u_Matrix);
3    $toggle_start;
4    $toggle_stop;
5    $toggle_report("Matrix.saif", 1.0e-9, u_Matrix);
```

基于多模式矩阵加速器仿真生成的 SAIF 文件，使用 PT 工具进行功耗分析。稠密矩阵乘法结果如图5-18所示，PT 报告中的功耗单元是瓦（W）。其总功耗为 10.5000 mW，内部功耗是 8.5940 mW，开关功耗是 1.8580 mW，漏电功耗是 53.9400 uW。因为稀疏矩阵中零值分布的随机性，所以本文统计矩阵、向量的稀疏度为 0% 和 99.9% 时多模式矩阵加速器在不同工作模式下的运行功耗，如表5-3所示，表中的功耗单位是（mW）。

```
                          Internal  Switching  Leakage    Total
Power Group               Power     Power      Power      Power   (     %)  Attrs
--------------------------------------------------------------------------------
clock_network                0.0000    0.0000    0.0000    0.0000 ( 0.00%)  i
register                     0.0000    0.0000    0.0000    0.0000 ( 0.00%)
combinational             2.472e-03 1.794e-03 2.200e-06 4.268e-03 (40.62%)
sequential                8.363e-04 6.086e-05 9.365e-07 8.981e-04 ( 8.55%)
memory                    5.286e-03 3.731e-06 5.081e-05 5.341e-03 (50.83%)
io_pad                       0.0000    0.0000    0.0000    0.0000 ( 0.00%)
black_box                    0.0000    0.0000    0.0000    0.0000 ( 0.00%)

  Net Switching Power  = 1.858e-03   (17.69%)
  Cell Internal Power  = 8.594e-03   (81.80%)
  Cell Leakage Power   = 5.394e-05   ( 0.51%)
                         ---------
Total Power            =    0.0105  (100.00%)
```

图 5-18　多模式矩阵加速器基于 SAIF 文件的稠密乘法平均功耗

表 5-3　在 0% 与 99.9% 稀疏度下，多模式矩阵加速器不同计算模式的平均功耗统计

稀疏度 / 计算模式	0%	99.9%
MXM	10.5000	1.6600
M+M	12.4000	1.9710
cVXrV	6.9160	0.4643
MXV	7.1700	0.6323
VXM	6.6970	0.5829
rVXcV	5.9620	1.8220
V+V	6.8580	1.8800
immXM	7.7290	7.7140
immXV	6.7770	6.3900
compM	4.3912	3.6275
decomprM	4.0660	3.5620

在异步无时钟电路中，总功率等于翻转速率 × 动态功率 + 静态功率。基于握手协议控制的异步电路，只有在工作条件满足的时候，电路才会工作产生动态功率[14]。分析表 5-3 中的数据可知，多模式矩阵加速器进行非压缩、解压操作工作

时的平均功耗随矩阵、向量的稀疏度增大而逐渐减小。举矩阵向量乘的例子，矩阵、向量的稀疏度很高时代表这有效块的数量越少，此时计算过程中的大部分操作是 BCSR 格式的解析、匹配操作，涉及的计算操作很少，所以实际的平均功耗很低。然而稠密矩阵加的平均功耗是 12.40 mW 略高于矩阵乘法的平均功耗，这是为何呢。从二者的运算步骤差异上进行分析，矩阵相加的过程中子矩阵的每次相加得到的结果需要立即回写至片内存储中去，而矩阵相乘需要分块矩阵一行乘以一列计算完成后才能产生一个需要回写至片内存储的子矩阵，因此矩阵相加的平均功耗大于矩阵相乘。分析数乘单元、压缩单元与解压单元的平均功耗受矩阵稀疏度影响不大的原因，首先对于数乘单元来说，不存在矩阵、向量的解析操作，每次的计算操作固定只有压缩格式的搬移和数据计算；其次对于矩阵的压缩和解压操作内部需要完整的一次矩阵操作。综上所述，面向稀疏高的低阶张量的计算场景中，本文实现的异步多模式矩阵加速器架构降低平均计算功耗的同时提升了计算效率。

5.3 异步片上系统实现、分析

多模式矩阵加速的验证完毕后，本文继续在 AISC 前端设计流程下实现 5.1.1 节中设计的异步 SoC，验证多模式矩阵加速器作为协处理器的可行性，并简单分析性能、功耗。

5.3.1 异步片上系统功能测试

首先，为了能够成功使用 RISC-V 编译器编译出对应异步 SoC 的具体架构的程序，本研究采用 C 语言编写适合本加速器架构的简易版矩阵函数库。如图5-19所示，四个计算单元的调用函数，包含多模式矩阵加速器支持的 11 种模式。图5-20的 C 代码通过调用矩阵计算单元完成行向量乘列向量计算操作。计算完成后，通过串口打印向量计算结果与字符串"rVXcV done!" 表示向量乘完成。

最后，成功在综合后仿真步骤中验证异步 SoC 中的 RISC-V CPU 核、串口等功能模块后，测试整个系统是否能够正常工作。本研究首先将编译出的包含多模式矩阵加速器需要的数据的程序写入文本文件，在 Testbench 中声明文件所在的路径并通过读文件函数将数据读入声明的寄存器数组中，最终在 Testbench 模拟串口的通信过程初始化异步 SoC 中的片内存储。图5-21中的 Verilog 代码是 Testbench 环境初始化异步 SoC 中的指令存储器的过程，分打包数据、发送数据包两个步骤。

初始完异步 SoC 中的存储后，异步 SoC 内部的初始化完成的标志位拉高，CPU 开始取指并执行指令。图5-22是异步 SoC 的执行结果，说明异步 RISC-V

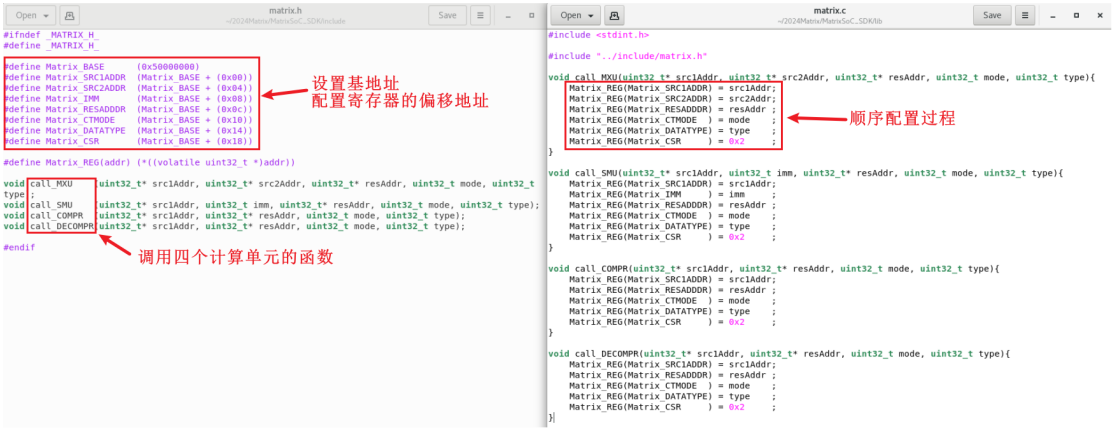


图 5-19　矩阵库函数代码

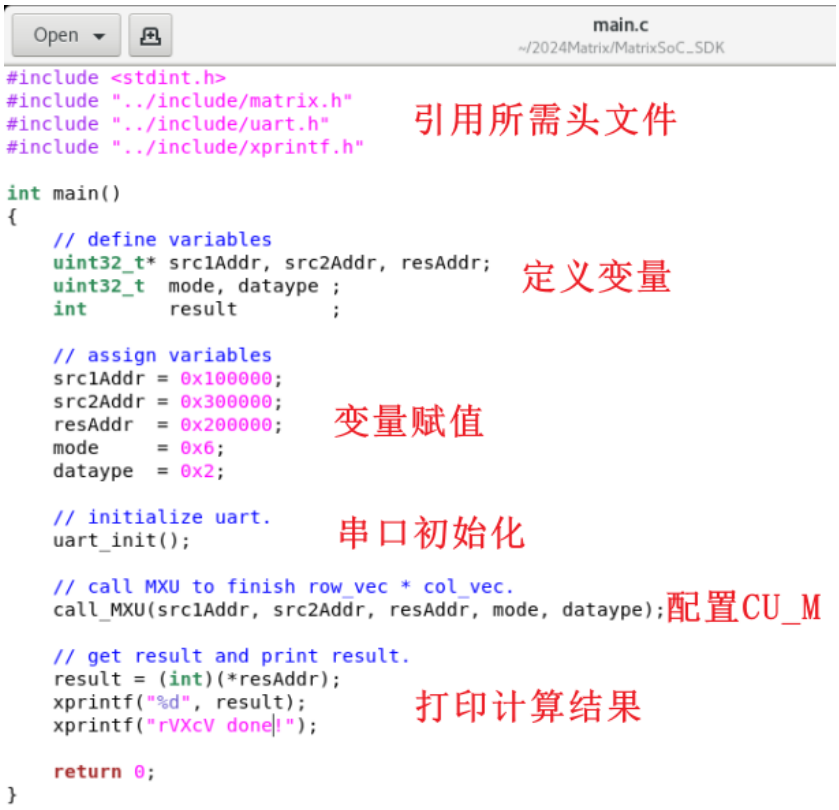


图 5-20　行向量乘列向量模式测试程序

CPU 核能够通过访存指令正确配置多模式矩阵加速器使其正常工作。本文研究最终对多模式矩阵加速器支持的 11 种计算模式逐个按照上述流程验证，同时按照 5.2.2 节中的结果对比方式验证其计算结果的正确性，证明了多模式矩阵加速器作为协处理器的可行性，能够弥补单核性能不足的 CPU 的算力。

5.3.2　异步 SoC 面积、功耗报告对比

异步片上系统的功能测试结束后，将 DC 综合后生成的面积、功耗报告记录至表 5-4。该表统计了 6 个与本研究相关的工作的制程、功耗、面积和片内存储



图 5-21　Testbench 中模拟指令、数据初始化过程代码段



图 5-22　异步 SoC 执行行向量乘列向量计算程序综合后仿真结果

大小等参数信息。根据该表中的数据，并结合 5.2.4 节中的功耗分析结果推断虽然异步片上系统目前在性能上无法与表中前 5 个工作仍存在差距，但是基于异步电路设计方法的工作在功耗密度（W/mm^2）方面具有一定优势，且这种优势随着低阶张量的稀疏度的提升而提升。

相较于异步设计[14]中的矩阵加速器，本研究在 DC 报告的平均功耗的近似的情况下，对矩阵乘法模式有着平均 5 倍的性能提升。本芯片性能加速原因可归纳为，首先本架构设计 BCSR 解析器在机制上避免全零子矩阵导致的无效计算。其次本架构的的基于粗粒度异步微流水设计，矩阵计算单元内部通过划分三级不定长流水线来提升其性能。最后本架构中的矩阵计算单元内部署 16 个乘累加器，相较于工作[14]中的 3 个算子，能够提供更高的峰值性能。

本研究中的异步 SoC 虽然在性能和表中提及的前 5 个工作存在差距，但是在处理高度稀疏的低阶张量或者在不需要长时间连续运行的场景下，本文实现的异步 SoC 展现出较好的功耗优势。简要分析造成性能上差距的原因，一是因为本研究的访存通道数过少导致的访存带宽过低，虽然统计的同步加速器的片内

表 5-4 矩阵加速器芯片基本参数

研究工作	制程（nm）	面积（mm^2）	功耗（W）	片内存储（KiB）
Flexagon[78]	28	5.28	2.998	1331.2
MatRaptor[27]	28	2.257	1.34	320
SpArch[25]	40	28.49	9.26	940
OuterSPACE[13]	32	86.74	23.99	528
SIGMA[26]	28	65.10	22.33	\
异步设计[14]	110	17.30	0.00468	232
本芯片	110	7.30	0.00303	198

存储的规模相差倍数不大，但是访存通道数是 10^4 量级[12]，而本研究的的访存通道仅 3 个。二是本研究采用的工艺制程是 110 nm，与几个工作所采用制程之间存在鸿沟。三是由于本研究采用的分块策略影响，并且算子规模有限仅 16 个，计算性能上限较低。

5.4 本章小结

本章基于多模式矩阵加速器架构，设计了多模式矩阵加速器的验证方案。接着简要介绍延迟限定型异步系统的 ASIC 前端实现流程，在介绍完多模式矩阵加速器的综合仿真验证后，详细分析了多模式矩阵加速器在 11 种计算模式下性能、功耗分析。最后，在 ASIC 流程下实现异步片上系统，并于相似工作进行比较，验证本章提出的异步片上系统在功耗方面具有一定优势，证明多模式矩阵加速器作为协处理器的可行性，同时间接证明基于粗粒度异步微流水线设计方法异步系统一定发展前景。

第六章 结论

6.1 总结

低阶张量计算是张量代数中的重要部分，如稀疏矩阵乘法、稀疏矩阵向量相乘等计算在图神经网络、线性系统求解等多种科学计算领域扮演重要角色。数据稀疏、数据分布零散等稀疏矩阵特征使得 CPU、GPU 等计算硬件无法充分发挥其性能优势。专用硬件加速器一般专注于某种计算的优化，无法完整实现实际的算法流程。本文聚焦高能效的多模式矩阵加速器，基于 BCSR 格式采用粗粒度异步微流水线，提出一种支持 11 种计算模式的矩阵加速器。最终，本文采用 110*nm* 制程完成多模式矩阵加速器的 ASIC 前端设计流程并进行详细分析。本文的主要工作如下：

（1）粗粒度异步微流水线设计方法的应用，实现无全局时钟。第二章详细介绍粗粒度异步微流水线包含 Fifo、分支、融合三种核心结构，和具体的 8 种模块。本研究借助粗粒度异步微流水线取消时钟数降低整体功耗，同时大大降低异步电路设计难度，将复杂逻辑成功映射成硬件电路。多模式矩阵加速器划分为异步微流水流水线构成的控制通路和大量组合逻辑、寄存器组成的数据通路，脉冲是模块间的通信介质，数据与脉冲的绑定确保整个加速器的正常工作。

（2）提出基于 BCSR 格式的存储方案，并设计内-外积矩阵乘算法。第三章首先分析 2.2 节介绍的存储策略，本研究选取 BCSR 格式存储稀疏矩阵，提高单次计算的并发性。接着统计分析 305 个稀疏矩阵在常见的分块策略下的平均有效块占比和平均零值填充率，决定采用 4×4 的分块策略，保证 BCSR 格式存储效率的同时，和 2×2 分块策略相比有着 4 倍的计算性能提升。基于上述的分析结果，针对规模 128×128 的稀疏矩阵，采用 4×4 分块策略，在 BCSR 格式基础上，结合片内存储每行大小 128 比特的实际情况，创新设计有利于代码库实现的矩阵和向量的存储方案。最后在 BCSR 格式的基础上，凭借内积算法和外积算法各自的优势，提出一种基于“内积-外积”融合的矩阵乘算法。借助内积算法输出复用性优势，降低生成结果矩阵或向量压缩格式的难度。凭借外积算法所具备的输入复用特性，减少局部取数的频次，增强局部计算效能，以此推动整体计算性能的提升。

（3）提出支持十一种计算操作的多模式矩阵加速器架构。在第四章本研究首先设计 11 条计算微码，这些微码能够支持包括矩阵乘法、矩阵加法以及稀疏矩阵的压缩与解压操作在内的 11 种计算模式。随后，提出一种矩阵计算架构，该架构由矩阵控制模块、三级流水矩阵计算单元、数乘单元、压缩单元以及解压单

元构成。矩阵控制模块的作用是对微码进行解析，进而驱动功能模块，以完成特定的计算任务。与传统 CPU 通过批量执行指令来控制和完成矩阵计算操作不同，此架构避免指令译码过程中的时间损耗，有效提升矩阵计算性能。同时，设计支持 7 种计算的 BCSR 解析器，借助该解析器，与矩阵和向量相关的多种计算能够以较低的成本部署在同一矩阵计算单元上。压缩单元和解压单元则用于实现原始矩阵与压缩矩阵之间的相互转换，从而完善多模式矩阵加速器的功能。最后，多模式矩阵加速器成功在 110*nm* 制程下完成 ASIC 前端设计流程，仿真结果表明矩阵的稀疏度大于 99.0% 时，相较于 Intel MKL，本架构在矩阵乘法、列向量 × 行向量和行向量 × 列向量 3 种模式下实现了平均 2.5×、2.5× 和 3× 的平均加速。同时，在稀疏度 0% 和 99.9% 下，基于 SAIF 文件的功耗分析结果表明异步多模式稀疏矩阵加速器面向高度稀疏的低阶张量计算具有较好的功耗优势。

6.2 展望

本文选用粗粒度异步微流水线设计方法研究、设计多模式矩阵加速器，提出内-外积混合算法、稀疏矩阵存储方案，最后基于 110*nm* 制程在 ASIC 前端设计流程下实现支持多种低阶张量计算的多模式矩阵加速器，完成仿真、验证。本文研究结果证明异步多模式矩阵加速器架构在低功耗应用领域存在优势具备可行性，但仍存在一些需要改进的不足：

（1）在架构层面上，首先是 4×4 的分块策略虽然确保局部单次计算性能，但仍存在无效计算。未来通过调研不同分块策略在计算过程中的性能得失，选取适宜于应用场景的分块策略。考虑支持多种分块策略的计算，对输入矩阵进行更多的数据预处理，保证稀疏矩阵加速器的性能，规避极端输入矩阵导致性能下降。其次，3 个片内存储导致结果输出、新矩阵输入过程中，矩阵加速器无法继续复杂模式计算。将来可以增加片内存储，扩展访存接口的拓扑结构，提高多模式矩阵加速器的整体性能。最后，单个多模式矩阵加速器的算力有限——551MOPS 的 INT32 算力。理论上通过片上网络挂载多个多模式矩阵加速器，设计合理的调度策略以及高校存储层次，能够实现片上系统的算力翻倍。

（2）在性能方面，首先是行向量乘矩阵和矩阵乘列向量 2 个计算模式能够进一步优化，在计算核心内分别针对 row-wise 算法和 column-wise 算法进行优化，当子向量中的元素数等于 0 时，不对子矩阵进行取数，直接跳到下一次计算，进一步提升两个模式的性能。其次，设计更优性能的乘累加器，大幅降低子矩阵外积乘法的时间开销，提升整体性能。最后，使用先进制程进行实现，完成性能飞跃。根据成本、应用场景，选择合适先进的工艺制程进行实现，达到高能效的目标策略。

（3）在应用层面上，多模式矩阵加速器当前仅支持 8 种整型数据类型的低阶张量计算，不支持高阶张量运算，存在应用局限性。未来通过部署浮点型的乘法器、加法器，扩大多模式矩阵加速器的应用领域。后续调研高阶张量计算并进行归纳整理，合理划分软硬件边界，软件端实现高阶张量分解，增加硬件的计算模式，构建软硬件协同的张量加速器。

综上所述，基于粗粒度异步微流水线设计完成的多模式矩阵计算架构仍有较大的提升空间，持续深挖上述问题，或能促使专用矩阵加速器的性能得以增强，通用性得以拓展。这无疑将为各类科学计算领域的计算工作，打造出更强大的计算后盾，全力推动相关任务高效完成。

参考文献

[1] Odemuyiwa T O, Asghari-Moghaddam H, Pellauer M, et al. Accelerating sparse data orchestration via dynamic reflexive tiling[C]//ASPLOS 2023: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. New York, NY, USA: Association for Computing Machinery, 2023: 18–32. https://doi.org/10.1145/3582016.3582064.

[2] Wen W, Wu C, Wang Y, et al. Learning structured sparsity in deep neural networks[C]// NIPS'16: Proceedings of the 30th International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc., 2016: 2082–2090.

[3] Provost F, Kohavi R. On applied research in machine learning[J]. MACHINE LEARNING-BOSTON-, 1998, 30: 127-132.

[4] Qiu S, You L, Wang Z. Optimizing sparse matrix multiplications for graph neural networks [C]//Li X, Chandrasekaran S. Languages and Compilers for Parallel Computing. Cham: Springer International Publishing, 2022: 101-117.

[5] Hu Z, Dong Y, Wang K, et al. Gpt-gnn: Generative pre-training of graph neural networks[C]// Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. 2020: 1857-1867.

[6] Aktulga H M, Buluç A, Williams S, et al. Optimizing sparse matrix-multiple vectors multiplication for nuclear configuration interaction calculations[C]//2014 IEEE 28th International Parallel and Distributed Processing Symposium. 2014: 1213-1222. DOI: 10.1109/IPDPS.2014.125.

[7] Vuduc R, Demmel J W, Yelick K A, et al. Performance optimizations and bounds for sparse matrix-vector multiply[C]//SC'02: Proceedings of the 2002 ACM/IEEE Conference on Supercomputing. IEEE, 2002: 26-26.

[8] Ng M K, Zhu Z. Sparse matrix computation for air quality forecast data assimilation[J]. Numerical Algorithms, 2019, 80: 687-707.

[9] Lee S, Park S, Lee M I, et al. Air quality forecasts improved by combining data assimilation and machine learning with satellite aod[J]. Geophysical Research Letters, 2022, 49(1): e2021GL096066.

[10] Vuduc R, Demmel J, Yelick K. Oski: A library of automatically tuned sparse matrix kernels[J]. Journal of Physics Conference Series, 2005, 16: 521-530. DOI: 10.1088/1742-6596/16/1/071.

[11] Gale T, Zaharia M, Young C, et al. Sparse gpu kernels for deep learning[C]//SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. 2020: 1-14. DOI: 10.1109/SC41405.2020.00021.

[12] Isaac–Chassande V, Evans A, Durand Y, et al. Dedicated hardware accelerators for processing of sparse matrices and vectors: A survey[J]. ACM Trans. Archit. Code Optim., 2024, 21(2). https://doi.org/10.1145/3640542.

[13] Pal S, Beaumont J, Park D H, et al. Outerspace: An outer product based sparse matrix multiplication accelerator[C]//2018 IEEE International Symposium on High Performance Computer Architecture (HPCA). 2018: 724-736. DOI: 10.1109/HPCA.2018.00067.

[14] 刁文麒. 基于延迟限定型异步电路的矩阵计算架构研究及实现[D]. 兰州大学, 2023.

[15] Ravi S, Trehan S, Jain M, et al. High performance clock path elements for clock skew reduction[C]//2019 2nd International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT): volume 1. 2019: 1663-1670. DOI: 10.1109/ICICICT46008.2019.8993375.

[16] Huang C. Advanced platform-level clock jitter and drift analysis[C]//2011 6th International Microsystems, Packaging, Assembly and Circuits Technology Conference (IMPACT). 2011: 111-114. DOI: 10.1109/IMPACT.2011.6117157.

[17] 张鸿锐. 基于 Strassen 融合策略的矩阵计算异步架构研究及实现[D]. 兰州大学, 2024.

[18] Basar T. A new approach to linear filtering and prediction problems[M]. 2001: 167-179. DOI: 10.1109/9780470544334.ch9.

[19] 张龙龙. 基于 FPGA 的矩阵计算协处理器设计与实现[D]. 国防科技大学, 2020.

[20] Jouppi N P, Young C, Patil N, et al. In-datacenter performance analysis of a tensor processing unit[C]//2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA). 2017: 1-12. DOI: 10.1145/3079856.3080246.

[21] Vahdat A. Announcing trillium, the sixth generation of google cloud tpu[EB]. 2024. https://cloud.google.com/blog/products/compute/introducing-trillium-6th-gen-tpus.

[22] Norrie T, Patil N, Yoon D H, et al. Google's training chips revealed: Tpuv2 and tpuv3[C]//2020 IEEE Hot Chips 32 Symposium (HCS). 2020: 1-70. DOI: 10.1109/HCS49909.2020.9220735.

[23] Jouppi N P, Hyun Yoon D, Ashcraft M, et al. Ten lessons from three generations shaped google's tpuv4i : Industrial product[C]//2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA). 2021: 1-14. DOI: 10.1109/ISCA52012.2021.00010.

[24] Jouppi N, Kurian G, Li S, et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings[C]//ISCA '23: Proceedings of the 50th Annual International Symposium on Computer Architecture. New York, NY, USA: Association for Computing Machinery, 2023. https://doi.org/10.1145/3579371.3589350.

[25] Zhang Z, Wang H, Han S, et al. Sparch: Efficient architecture for sparse matrix multiplication[C]//2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). 2020: 261-274. DOI: 10.1109/HPCA47549.2020.00030.

[26] Qin E, Samajdar A, Kwon H, et al. Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training[C]//2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). 2020: 58-70. DOI: 10.1109/HPCA47549.2020.00015.

[27] Srivastava N, Jin H, Liu J, et al. Matraptor: A sparse-sparse matrix multiplication accelerator based on row-wise product[C]//2020 53rd Annual IEEE/ACM International Symposium on

Microarchitecture (MICRO). 2020: 766-780. DOI: 10.1109/MICRO50266.2020.00068.

[28] Zhang G, Attaluri N, Emer J S, et al. Gamma: leveraging gustavson's algorithm to accelerate sparse matrix multiplication[C]//ASPLOS '21: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. New York, NY, USA: Association for Computing Machinery, 2021: 687–701. https://doi.org/10.1145/3445814.3446702.

[29] Pavon J, Valdivieso I V, Barredo A, et al. Via: A smart scratchpad for vector units with application to sparse matrix computations[C]//2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2021: 921-934. DOI: 10.1109/HPCA51647.2021.00081.

[30] Gerogiannis G, Yesil S, Lenadora D, et al. Spade: A flexible and scalable accelerator for spmm and sddmm[C]//ISCA '23: Proceedings of the 50th Annual International Symposium on Computer Architecture. New York, NY, USA: Association for Computing Machinery, 2023. https://doi.org/10.1145/3579371.3589054.

[31] Shabani H, Singh A, Youhana B, et al. Hirac: A hierarchical accelerator with sorting-based packing for spgemms in dnn applications[C]//2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2023: 247-258. DOI: 10.1109/HPCA56546.2023.10070977.

[32] Zhu Z, Zhou X, Wang C, et al. Bit-sparsity aware acceleration with compact csd code on generic matrix multiplication[J]. IEEE Transactions on Computers, 2025, 74(2): 414-426. DOI: 10.1109/TC.2024.3483632.

[33] Sadi F, Sweeney J, Low T M, et al. Efficient spmv operation for large and highly sparse matrices using scalable multi-way merge parallelization[C]//MICRO '52: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. New York, NY, USA: Association for Computing Machinery, 2019: 347–358. https://doi.org/10.1145/3352460.3358330.

[34] Asgari B, Hadidi R, Krishna T, et al. Alrescha: A lightweight reconfigurable sparse-computation accelerator[C]//2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). 2020: 249-260. DOI: 10.1109/HPCA47549.2020.00029.

[35] Xie X, Liang Z, Gu P, et al. Spacea: Sparse matrix vector multiplication on processing-in-memory accelerator[C]//2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2021: 570-583. DOI: 10.1109/HPCA51647.2021.00055.

[36] Feng S, Sun J, Pal S, et al. Cosparse: A software and hardware reconfigurable spmv framework for graph analytics[C]//2021 58th ACM/IEEE Design Automation Conference (DAC). 2021: 949-954. DOI: 10.1109/DAC18074.2021.9586114.

[37] Lu Y, Liu W. Dasp: Specific dense matrix multiply-accumulate units accelerated general sparse matrix-vector multiplication[C]//SC '23: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. New York, NY, USA: Association for Computing Machinery, 2023. https://doi.org/10.1145/3581784.3607051.

[38] Hwang S, Baek D, Park J, et al. Cerberus: Triple mode acceleration of sparse matrix and vector multiplication[J]. ACM Trans. Archit. Code Optim., 2024, 21(2). https://doi.org/10.1145/3653020.

[39] Strassen V. Gaussian elimination is not optimal[J]. Numer. Math., 1969, 13(4): 354–356. https://doi.org/10.1007/BF02165411.

[40] Cormen T H, Leiserson C E, Rivest R L, et al. Introduction to algorithms, fourth edition[M]. 4th ed. The MIT Press, 2022.

[41] Ohtaki Y, Takahashi D, Boku T, et al. Parallel implementation of strassen's matrix multiplication algorithm for heterogeneous clusters[C]//18th International Parallel and Distributed Processing Symposium, 2004. Proceedings. 2004: 112-. DOI: 10.1109/IPDPS.2004.1303066.

[42] Gustavson F G. Two fast algorithms for sparse matrices: Multiplication and permuted transposition[J]. ACM Trans. Math. Softw., 1978, 4(3): 250–269. https://doi.org/10.1145/355791.355796.

[43] Cannon L E. A cellular computer to implement the kalman filter algorithm[M]. Montana State University, 1969.

[44] Langr D, Tvrdík P. Evaluation criteria for sparse matrix storage formats[J]. IEEE Transactions on Parallel and Distributed Systems, 2016, 27(2): 428-440. DOI: 10.1109/TPDS.2015.2401575.

[45] Greathouse J L, Daga M. Efficient sparse matrix-vector multiplication on gpus using the csr storage format[C]//SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2014: 769-780. DOI: 10.1109/SC.2014.68.

[46] Rice J R, Boisvert R F. Solving elliptic problems using ellpack[M]. Berlin, Heidelberg: Springer-Verlag, 1985.

[47] Sun X, Zhang Y, Wang T, et al. Optimizing spmv for diagonal sparse matrices on gpu[C]//2011 International Conference on Parallel Processing. 2011: 492-501. DOI: 10.1109/ICPP.2011.53.

[48] Bell N, Garland M. Implementing sparse matrix-vector multiplication on throughput-oriented processors[C]//Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis. 2009: 1-11. DOI: 10.1145/1654059.1654078.

[49] Smith S C, Di J. 基于归零逻辑 (NCL) 的异步电路设计方法[M]. 何安平, 冯志华, 译. 电子工业出版社, 2023.

[50] Muller D. A theory of asynchronous circuits[C]//Proc. Int. Symp. on Theory of Switching, 1959: volume 29. 1959: 204-243.

[51] Peeters A, te Beest F, de Wit M, et al. Click elements: An implementation style for data-driven compilation[C]//2010 IEEE Symposium on Asynchronous Circuits and Systems. 2010: 3-14. DOI: 10.1109/ASYNC.2010.11.

[52] Sutherland I, Fairbanks S. Gasp: a minimal fifo control[C]//Proceedings Seventh International Symposium on Asynchronous Circuits and Systems. ASYNC 2001. 2001: 46-53. DOI: 10.1109/ASYNC.2001.914068.

[53] Sparsø J, Furber S B. Principles of asynchronous circuit design[C]//2001. https://api.semant

icscholar.org/CorpusID:62931202.

[54] 万宝霞. 一种支持动态转向的全异步 Crossbar 片上网络架构研究[D]. 兰州大学, 2024.

[55] Fant K, Brandt S. Null convention logic/sup tm/: a complete and consistent logic for asynchronous digital circuit synthesis[C]//Proceedings of International Conference on Application Specific Systems, Architectures and Processors: ASAP '96. 1996: 261-273. DOI: 10.1109/ASAP.1996.542821.

[56] Sutherland I E. Micropipelines[J]. Commun. ACM, 1989, 32(6): 720–738. https://doi.org/10.1145/63526.63532.

[57] He A, Feng G, Zhang J, et al. Click-based asynchronous mesh network with bounded bundled data[C]//ICPP '18: Proceedings of the 47th International Conference on Parallel Processing. New York, NY, USA: Association for Computing Machinery, 2018. https://doi.org/10.1145/3225058.3225118.

[58] Nori V, Chauviere B, Wibbels M J, et al. A novel asynchronous network-on-chip based on source asynchronous signaling[C]//2023 28th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC). 2023: 71-77. DOI: 10.1109/ASYNC58294.2023.10239625.

[59] 梁钰清. 异步 RSA 算法芯片的研究与实现[D]. 兰州大学, 2021.

[60] 孙若云. 融合 RSA 和 ECC 安全密码算法的全异步防侧信道攻击芯片研究[D]. 兰州大学, 2024.

[61] 王天立. 基于 CrossBar 的延迟限定型异步粗粒度可重构架构研究及实现[D]. 兰州大学, 2024.

[62] Huo D, Zhang J, Zhang J, et al. A 28nm energy-efficient asynchronous snn accelerator with on-chip learning for gas recognition[C]//2023 28th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC). 2023: 42-47. DOI: 10.1109/ASYNC58294.2023.10239577.

[63] Nelson S, Kim S Y, Di J, et al. Reconfigurable asic implementation of asynchronous recurrent neural networks[C]//2021 27th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC). 2021: 48-54. DOI: 10.1109/ASYNC48570.2021.00015.

[64] 温立. 面向双目事件相机的 SNN 三维重建系统设计及异步电路实现[D]. 兰州大学, 2021.

[65] 程海波. 基于异步方法的卷积神经网络计算系统的设计与分析[D]. 兰州大学, 2019.

[66] Davies M, Srinivasa N, Lin T H, et al. Loihi: A neuromorphic manycore processor with on-chip learning[J]. IEEE Micro, 2018, 38(1): 82-99. DOI: 10.1109/MM.2018.112130359.

[67] Akopyan F, Sawada J, Cassidy A, et al. Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2015, 34(10): 1537-1557. DOI: 10.1109/TCAD.2015.2474396.

[68] 赵康利. 基于 RISC-V 的异步超标量处理器架构的研究[D]. 兰州大学, 2023.

[69] 付桐. 面向边缘计算的 RISC-V 指令扩展与片上系统架构研究[D]. 兰州大学, 2024.

[70] Roncken M, Sutherland I, Chen C, et al. How to think about self-timed systems[C]//2017 51st Asilomar Conference on Signals, Systems, and Computers. 2017: 1597-1604. DOI: 10.1109/ACSSC.2017.8335628.

[71] Mallon W. Bounded bundled data[J]. Asynchronous Research Center, Portland State University, Tech. Rep, 2012.

[72] Ahrens W, Xu H, Schiefer N. A fill estimation algorithm for sparse matrices and tensors in blocked formats[C]//2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS). 2018: 546-556. DOI: 10.1109/IPDPS.2018.00064.

[73] Davis T A, Hu Y. The university of florida sparse matrix collection[J]. ACM Trans. Math. Softw., 2011, 38(1). https://doi.org/10.1145/2049662.2049663.

[74] Zhang L, Hu R, Zhang H, et al. Hsamm: A hybrid-strassen algorithm-based asynchronous architecture for sparse matrix multiplication[C]//2024 IEEE International Conference on High Performance Computing. 2024.

[75] Fog A. The microarchitecture of intel, amd, and via cpus[EB]. 2024. https://www.agner.org/optimize/microarchitecture.pdf.

[76] Intel. Intel quartus prime pro edition user guide: Power analysis and optimization[EB]. 2021. https://www.intel.com/content/www/us/en/docs/programmable/683174/21-3/power-analysis.html.

[77] Intel. Intel math kernel library[EB]. https://www.intel.cn/content/www/cn/zh/developer/tools/oneapi/mkl-legacy-documentation.html?wapkw=mkl.

[78] Muñoz Martínez F, Garg R, Pellauer M, et al. Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing[C]//ASPLOS 2023: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. New York, NY, USA: Association for Computing Machinery, 2023: 252–265. https://doi.org/10.1145/3582016.3582069.

在学期间的研究成果

一、发表论文

1. HSAMM: A Hybrid-Strassen Algorithm-Based Asynchronous Architecture for Sparse Matrix Multiplication[C]//2024 IEEE International Conference on High Performance Computing & Communications. IEEE, 2024.（已录用，第一作者）

2. An Asynchronous Matrix Multiplication Accelerator[C]// European Conference on Parallel Processing Workshops. 2024.（已录用，第一作者）

3. livp：基于同异步混合机制的 RISC-V 处理器架构 [C]. //第二十七届计算机工程与工艺学术年会暨第十三届“微处理器”论坛, 2023:574-583.（已录用，第一作者）

二、参与课题

1. 基于 RISC-V 指令集的全异步低功耗 SoC 研发与应用示范（XX 省科技重大项目，编号：22ZD6G016，2022.11-2025.11）

2. 异步集成电路芯片研究团队（XX 大学中央高校优秀青年团队项目，编号：lzuibky-2024-eyt02，2024.01-2025.12）

3. 基于 Link-Joint 技术的全异步神经网络芯片设计与 FPGA 验证研究（中央引导地方科技发展资金项目，编号：25ZYJA016，2025.01-2027.02）

三、获奖成果

1. 2024 年 7 月，第八届全国大学生集成电路创新创业大赛西北赛区三等奖（队长）

2. 2023 年 7 月，第七届全国大学生集成电路创新创业大赛西北赛区二等奖（队员）

3. 2023 年 7 月，第十四届“挑战杯”XX 省大学生课外学术科技作品竞赛一等奖（队员）

4. 2023 年 8 月，“中国光谷·华为杯”第六届中国研究生创“芯”大赛团队优秀奖（队员）

5. 2022-2023 学年研究生学业二等奖学金

6. 2023-2024 学年研究生学业一等奖学金

致　　谢

感谢一路走来，陪伴我一路成长的老师、同门与朋友们。